Step 1: Defining the Problem and assembling a dataset
Step 2: Choosing a measure of success and metrics to monitor
Step 3: Deciding on an evaluation protocol
Step 4: Data Preparation
Step 5: Developing an underfitting model with statistical power
Step 6: Scaling - Developing a model that overfits
Step 7: Model Regularization and hyperparameter tuning
Retrain and testing of final model
Results
Evaluation
Appendices
- Appendix A: Code to implement SMOTETomek
- Appendix B: Comparing the performance of a model trained on non-SMOTETomek data (before removal of class 7)
References

Introduction to the Dataset: The dataset I have chosen for this coursework submission [002] is the 'EMG Data for Gestures' dataset from the UC Irvine Machine Learning Repository. The original dataset (published on UCI) was published in the form of multiple .csv files (separated by gesture and test subject) ;
I found this dataset published on Kaggle formatted as 1 .csv file) The Data can be viewed at the following sources:
https://archive.ics.uci.edu/dataset/481/emg+data+for+gestures
https://www.kaggle.com/datasets/sojanprajapati/emg-signal-for-gesture-recognition?resource=download

The neural network for this project is trained on the 'EMG data for gestures' dataset from the UC Irvine Machine Learning Repository. The dataset collection method involved placing a 'MYO Thalmic bracelet' on a test subject's forearm; this device houses 8 EMG (electromyography) sensors which monitor the level of electrical activity in response to muscle-nerve stimulation. Test subjects were asked to perform 7 gestures (1- hand at rest, 2 - hand clenched in a fist, 3 - wrist flexion, 4 - wrist extension, 5 - radial deviations, 6 - ulnar deviations, 7 - extended palm). The aim of this project is utilise the 8 EMG channels to classify the gesture performed. Gesture recognition based on the MYO Thalmic bracelet could enable gesture based interaction with nearby devices (perhaps a nearby smart tv).

Step 1: Defining the Problem and assembling a dataset

Input data is in the following forms:

- EMG signals across 8 channels ; research paper 'Techniques of EMG signal analysis: detection, processing, classification and applications' defines EMG as a 'biomedical signal that measures electrical currents generated in muscles during its contraction representing neuromuscular activities.' (Mohd, 2006)
- a label (identifying a study participant),
- a timestamp.

Output / Target:

The model will aim to classify the gesture based on the input data; gestures are classified into the following groups: (1- hand at rest, 2 - hand clenched in a fist, 3 - wrist flexion, 4 – wrist extension, 5 - radial deviations, 6 - ulnar deviations, 7 - extended palm)

Each data input corresponds to 1 gesture ; this defines the problem as 'single label multiclass classification'

Step 2: Choosing a measure of success and metrics to monitor

During the initial phases of this coursework, I uncovered an imbalance in the class distribution with respect to classes '0' and '7') - this can be seen in Step 4: Data Preparation). I researched techniques to resolve class imbalance and decided to use a combination of SMOTE (synthetic minority oversampling technique) and Tomek Links (under sampling). I selected SMOTETomek over random oversampling as random oversampling can decrease the variance of the dataset which can potentially lead to an overfitting model, whilst SMOTE [developed by Chawla et al. (2002)] 'generates a new observation that exists on a line between observations'. (Pao, 2021) The advantage of combining SMOTE with Tomek links is that it effectively 'cleans' the dataset ; ambiguous points 'Tomek Links' are removed. (Truong, 2022) Further explanation and the code used to implement SMOTETomek can be seen in Appendix A

I read recommendations to not apply SMOTETomek to the testing / validation data as it will introduce artifical data into the test set. (Nair, 2022) Once I reached the validation stage, the validation data was of a different class distribution to the training set ; this resulted in noise / fluctuations in the validation loss. The fluctuations made it difficult to categorise the overall pattern of the learning curve (eg underfitting or overfitting). On realising this, I modified my approach to remove both minority classes 0 and 7 (0 is 'unlabelled data' and 7 was noted in the data documentation as only recorded for some test participants).

I utilised the 'stratify' parameter of scikit learn's train_test_split method to perform a stratified split of the data ; this ensured that the class distribution of train, valiation and test data were balanced. I felt that a stratified split was important as an imbalanced class distribution can skew the accuracy of some evaluation metrics; a random model could potentially acheive a high accuracy by predicting only the majority class (Brownlee, 2021).

I selected the metrics below for the following reasons:

- F1 Score: as the F1 score is calculated as the harmonic mean of precision and recall; maximizing the F1 score simultaneously maximizes both precision and recall (Logunova & guide, 2023)
- Cohen Kappa ; an advantage of the Cohen Kappa metric is that it compensates for classifications that may be due to chance (Arie, 2008). Cohen Kappa scores range from [-1, 1]. Negative scores indicate a disagreement between two raters; positive scores indicate agreement between. A score of 0 indicates agreement by chance. (TensorFlow, 2023)

I decided to use both (F1 score and Cohen Kappa) to evaluate performance as a model can indicate good performance on one metric, but indicate suboptimal performance on another (Buhl, 2023).

Step 3: Deciding on an evaluation / validation protocol

Following the recommendation of the textbook 'Deep Learning with Python' (Chapter 4.5.3) (Chollet, 2018) I will use the hold out validation technique as there seems to be plenty of data available (over 1 million data samples).
[The textbook notes that K-fold cross validation and K-fold validation are often used in the context of small volumes of data)

An article published by Machine Learning Platform 'Saturn Cloud' recommends using the 'random_state' parameter when performing an sklearn train-test split. The use of random_state ensures that experiment results are reproducible and comparable ; 'fixing' the train-test split ensures that this is not a cause of performance deterioration.(Saturn Cloud, 2023)

Step 4: Data Preparation

In [26]:
```
import pandas as pd
#https://www.kaggle.com/datasets/sojanprajapati/emg-signal-for-gesture-recognition?resource=download
#https://archive.ics.uci.edu/dataset/481/emg+data+for+gestures
data = pd.read_csv('./Data/EMG-data.csv', sep=',')
data
```

Out[26]:

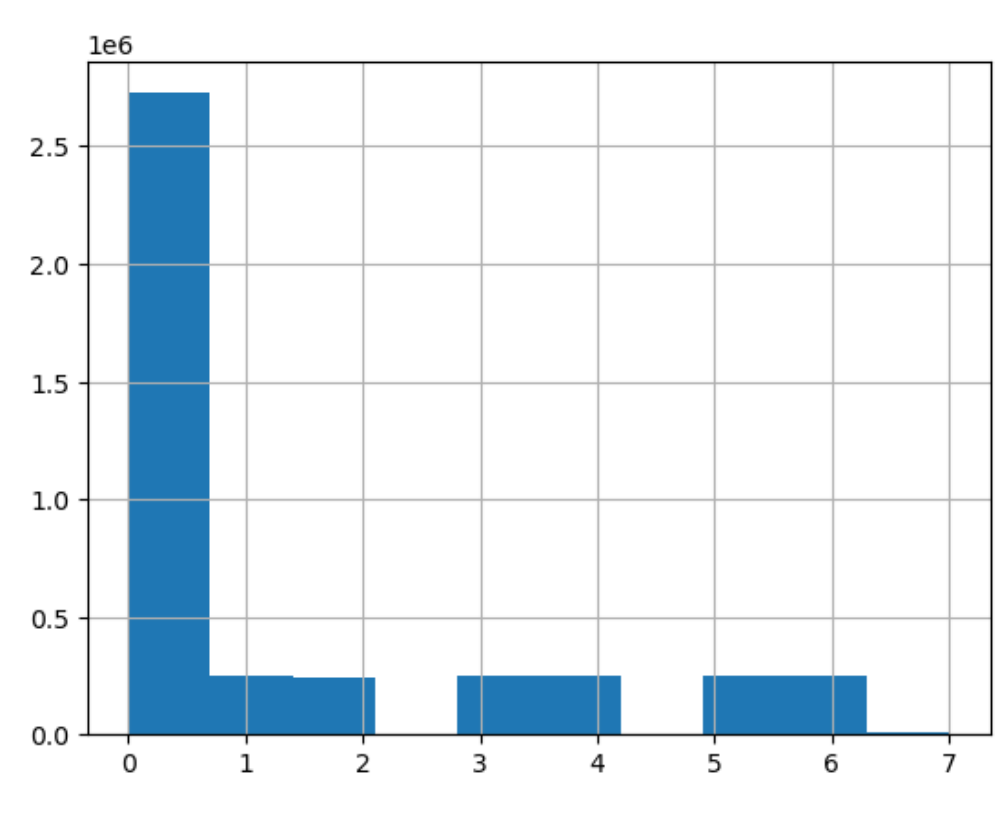
	time	channel1	channel2	channel3	channel4	channel5	channel6	channel7	channel8	class	label
0	1	0.00001	-0.00002	-0.00001	-0.00003	0.00000	-0.00001	0.00000	-0.00001	0	1
1	5	0.00001	-0.00002	-0.00001	-0.00003	0.00000	-0.00001	0.00000	-0.00001	0	1
2	6	-0.00001	0.00001	0.00002	0.00000	0.00001	-0.00002	-0.00001	0.00001	0	1
3	7	-0.00001	0.00001	0.00002	0.00000	0.00001	-0.00002	-0.00001	0.00001	0	1
4	8	-0.00001	0.00001	0.00002	0.00000	0.00001	-0.00002	-0.00001	0.00001	0	1
...	...	...	...	...	...	...	...	...	...	...	...
4237902	50962	0.00001	-0.00001	-0.00002	-0.00004	-0.00012	0.00000	0.00002	0.00002	0	36
4237903	50963	0.00001	-0.00001	-0.00002	-0.00004	-0.00012	0.00000	0.00002	0.00002	0	36
4237904	50964	0.00001	-0.00001	-0.00002	-0.00004	-0.00012	0.00000	0.00002	0.00002	0	36
4237905	50965	0.00001	-0.00001	-0.00002	-0.00004	-0.00012	0.00000	0.00002	0.00002	0	36
4237906	50966	0.00001	-0.00001	-0.00002	-0.00004	-0.00012	0.00000	0.00002	0.00002	0	36

4237907 rows × 11 columns

Assessment of class distribution:

In [27]:
```
data['class'].hist()
```

Out[27]:
```
<Axes: >
```



The class distribution in the histogram is unbalanced ; looking at the data documentation class '0' signifies 'unmarked data'. I will remove all data with class '0' to resolve this imbalance.

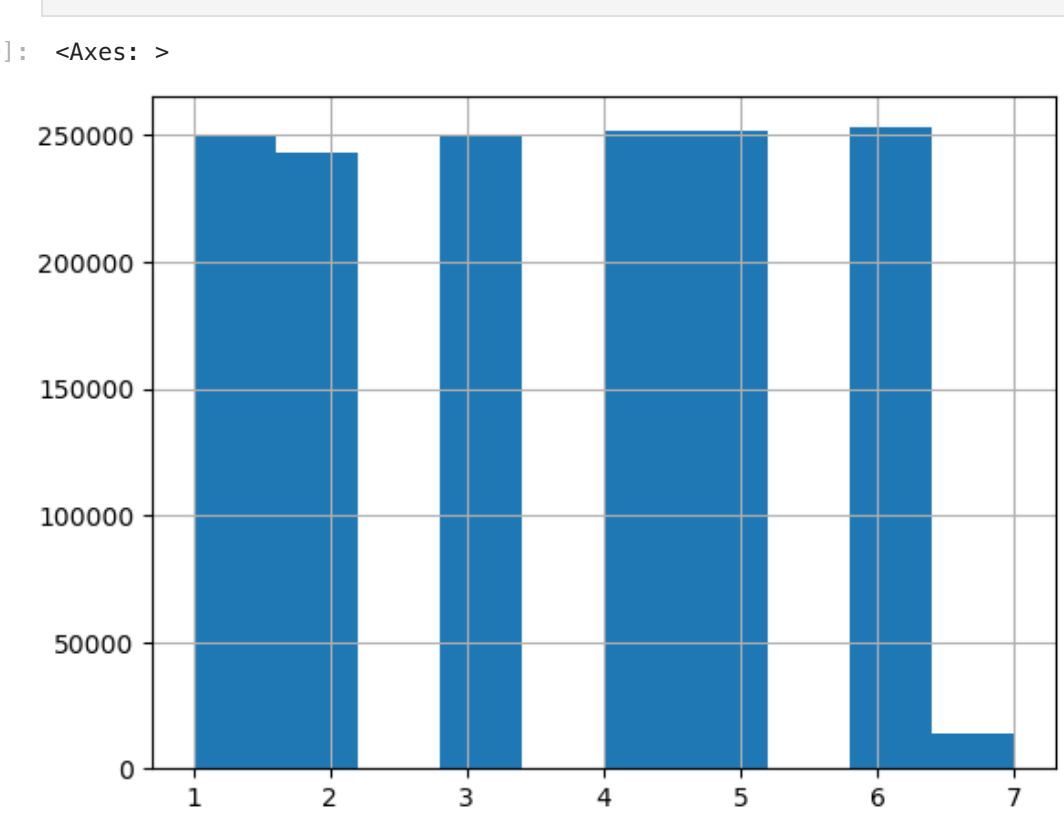
In [28]:
```
df = data.drop(data[data['class'] == 0].index)
print(data.shape)
print(df.shape)
```

```
(4237907, 11)
(1512750, 11)
```

Removal of data points labelled class 0 has left 1,512,750 data samples ; a sufficient volume for training.

In [29]:
```
df['class'].hist()
```

Out[29]:
```
<Axes: >
```



The distribution of classes 1-6 appear to be relatively balanced ; however class 7 has fewer samples. I shall further examine this using the pandas .value_counts() method:

In [30]:
```
import math

print('Average number of class samples:')
average_class_count = math.ceil(df['class'].value_counts().mean())
print(average_class_count)
print('Number of class 7 samples:')
class_7_count = df['class'].value_counts()[7]
print(class_7_count)
print("Difference in samples size:")
print(average_class_count-class_7_count)
print("% of class samples that are class 7:")
total_class_count = df['class'].value_counts().sum()
print(class_7_count/total_class_count * 100)
```

```
Average number of class samples:
216108
Number of class 7 samples:
13696
Difference in samples size:
202412
% of class samples that are class 7:
0.9053710138556933
```

Class 7 Samples consitiute less than 1% of total class samples; this suggests a severe imbalance in the class. This discrepancy is noted in the data documentation; '7 - extended palm (the gesture was not performed by all subjects).' (UC Irvine Machine Learning Repository, 2019). In the initial phases of this coursework I split the data and implemented SMOTETomek Links to resolve the class imbalance in the training data (this can be seen in Appendix A). SMOTETomek links was not applied to the validation and test set (as it would introduce synthetic data into the model evaluation procedure) ; this resulted in noise in the learning curves that made it difficult to assess overall trend (ie underfitting or overfitting). My next approach was to remove the minority class '7' from the dataset as it did not contain samples from every test participant:

In [31]:
```
mask = df['class'] != 7
df_2 = df[mask]
y = df_2.pop('class')
x = df_2.drop(columns=['time'])
```

Considering whether the 'label' feature should be removed:

The 'label' column identifies the subject who performed the gesture ; this feature could prove useful in making accurate, personalised gesture recognition:
The following quote from the research paper 'Techniques of EMG signal analysis: detection, processing, classification and applications' suggests it may be useful to keep the label: EMG 'is dependent on the anatomical and physiological properties of muscles. EMG signal acquires noise while traveling through different tissues.' (Mohd, 2006) This suggests that EMG signals could have unique variations and be useful in making accurate personalised predictions.

In [32]:
```
from sklearn.model_selection import train_test_split

# I originally trained the model on the entire dataset (1,499,054) however the time to train the model
#(both locally and on Google Colab V100 GPU) became too large during hyperparameter tuning
#the updated training test split, using roughly 1/8 of the data, can be seen below

# x_train, x_remaining = train_test_split(x, train_size=0.7, random_state=42)
# y_train, y_remaining = train_test_split(y, train_size=0.7, random_state=42)

# x_val, x_test = train_test_split(x_remaining, test_size=0.5, random_state=42)
# y_val, y_test = train_test_split(y_remaining, test_size=0.5, random_state=42)



#80/10/10 split ;
x_train, x_test, y_train, y_test = train_test_split(x,y, train_size=0.12, test_size=0.012, random_state=42, stratify=y)

x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.1, random_state=42, stratify=y_train)

#confirm that split was performed correctly
# print(len(x_train))
# print(len(x_val))
# print(len(x_test))
# sum = len(x_train) + len(x_val) + len(x_test)
# print(len(x_val)/sum)*100)
```

In [33]:
```
#store class counts (to calculate statistical baseline) before one hot encoding transforms the data
y_train_class_count = y_train.value_counts()
y_val_class_count = y_val.value_counts()
y_test_class_count = y_test.value_counts()
```

Data Normalisation

The following reaserch paper published on PubMed Central (National Library of Medicine) suggests that the dataset should be normalized: 'Because of the inherent EMG signal variability, clinical interpretation of surface EMG signals requires normalization of the signal for physiologic interpretation and for comparison between bilateral muscles and between the same muscle on different days and between different subjects.' (McGill, 1999)

In [34]:
```
from sklearn.preprocessing import MinMaxScaler
#scaler = MinMaxScaler(feature_range=(-1, 1))
scaler = MinMaxScaler()
#scale validation data
x_val_scaled = scaler.fit_transform(x_val)
#scale testing data
x_test_scaled = scaler.fit_transform(x_test)
#scale non-SMOTETomek training data ; this will be used to compare model performance trained on SMOTETomek vs non-SMOTETomek
x_train_scaled = scaler.fit_transform(x_train)
```

Convert data to tensor form

In [35]:
```
import tensorflow as tf
#convert validation data to tensor form
x_val_scaled = tf.convert_to_tensor(x_val_scaled)
#convert testing data to tensor form
x_test_scaled = tf.convert_to_tensor(x_test_scaled)
#convert training data to tensor form
x_train_scaled = tf.convert_to_tensor(x_train_scaled)
```

One hot encoding

The textbook 'Deep Learning with Python' Chapter 3.5.9 suggested two approaches to encoding the labels in single label, multi class classification:

- one-hot encoding, categorical_crossentropy loss function
- encoding labels as integers, sparse_categorical_crossentropy loss function

According to (Crêteur, 2022) one-hot encoding allows the model to learn more effectively ; as 'integer encoding can introduce a bias into the data by implying a natural ordering between the categories' (Crêteur, 2022). With this in mind, I chose to utilize one-hot encoding for this project.
(One hot encoding was implemented after the train-test split to mitigate train-test contamination.)

In [36]:
```
from tensorflow.keras.utils import to_categorical
num_classes = len(y.value_counts())
```

The tensorflow documentation states that to_categorical infers the number of classes as max(y) + 1
As the classes in the current dataframe are (1,2,3,4,5,6) keras infers that there are 7 classes - to resolve this I will subtract 1 from every value in the classes column ; making class numbers (0,1,2,3,4,5) - this procedure was also applied to the validation and test set:

In [37]:
```
new_train_classes = [x-1 for x in list(y_train)]
new_val_classes = [x-1 for x in list(y_val)]
new_test_classes = [x-1 for x in list(y_test)]
```

In [38]:
```
#one hot encode validation data
y_val = to_categorical(new_val_classes)
y_val = tf.convert_to_tensor(y_val)

#one hot encode testing data
y_test = to_categorical(new_test_classes)
y_test = tf.convert_to_tensor(y_test)

#one hot encode non-SMOTETomek training data;
#this will be used to compare model performance trained on SMOTETomek vs non-SMOTETomek
y_train = to_categorical(new_train_classes)
y_train = tf.convert_to_tensor(y_train)
```

Step 5: Developing an underfitting model with statistical power

Section 5: Table of F1 Outcomes:

In [53]:
```
stage_5_results.sort_values(by='val_f1_score', ascending=True)
```

Out[53]:

	num_hidden_layers	num_units	optimizer	learning_rate	activation_function	batch_size	num_epochs	regularization	val_f1_score
0	0	2	rmsprop	0.001	relu	128	5	False	0.157849
0	0	4	rmsprop	0.001	relu	128	5	False	0.231503
0	0	8	rmsprop	0.001	relu	128	5	False	0.269033
0	0	6	rmsprop	0.001	relu	128	5	False	0.298736
0	0	10	rmsprop	0.001	relu	128	5	False	0.334771
2	2	10	rmsprop	0.001	relu	128	5	False	0.352718
3	3	10	rmsprop	0.001	relu	128	5	False	0.378242
1	1	10	rmsprop	0.001	relu	128	5	False	0.396564
4	0	10	rmsprop	0.001	relu	128	10	False	0.432506
5	1	10	rmsprop	0.001	relu	128	10	False	0.455896
8	0	10	rmsprop	0.001	relu	128	15	False	0.464227
7	3	10	rmsprop	0.001	relu	128	10	False	0.480803
6	2	10	rmsprop	0.001	relu	128	10	False	0.483957
12	0	10	rmsprop	0.001	relu	128	20	False	0.489171
10	2	10	rmsprop	0.001	relu	128	15	False	0.527253
11	3	10	rmsprop	0.001	relu	128	15	False	0.529207
14	2	10	rmsprop	0.001	relu	128	20	False	0.536040
23	3	10	rmsprop	0.001	relu	128	30	False	0.537457
15	3	10	rmsprop	0.001	relu	128	20	False	0.542709
9	1	10	rmsprop	0.001	relu	128	15	False	0.544056
16	0	10	rmsprop	0.001	relu	128	25	False	0.547732
19	3	10	rmsprop	0.001	relu	128	25	False	0.548413
18	2	10	rmsprop	0.001	relu	128	25	False	0.560880
20	0	10	rmsprop	0.001	relu	128	30	False	0.564644
13	1	10	rmsprop	0.001	relu	128	20	False	0.571100
21	1	10	rmsprop	0.001	relu	128	30	False	0.587181
22	2	10	rmsprop	0.001	relu	128	30	False	0.588983
17	1	10	rmsprop	0.001	relu	128	25	False	0.599177

Section 5: Summary of F1 outcomes:

The table above shows the hyperparameters I tested in this section to identify the smallest capacity model that beat the statistical baseline F1 score of 0.25 (see calculation below). The smallest capacity model identified in this section can be seen on row 3 in the table above (0 hidden layers, 8 units in the input layer, trained for 5 epochs)

Section 5 Training Log:

Calculation of Majority Class Baseline:

In [18]:
```
train_baseline = y_train_class_count[y_train_class_count.idxmax()]/ sum(y_train_class_count) *100
print(f'Training data statistical baseline: {train_baseline}')
val_baseline = y_val_class_count[y_val_class_count.idxmax()]/ sum(y_val_class_count) *100
print(f'Validation data statistical baseline: {val_baseline}')
test_baseline = y_test_class_count[y_test_class_count.idxmax()]/ sum(y_test_class_count) *100
print(f'Test data statistical baseline: {test_baseline}')
```

```
Training data statistical baseline: 16.878015034250172
Validation data statistical baseline: 16.876980376896995
Test data statistical baseline: 16.876980376896995
```

On reflection and further research I realised that the above was not directly comparabale to the F1 evaluation of the model; Further research identified that the common sense baseline is calculated on the test set (Brownlee, 2020)

With this in mind, I calculated the following common sense baseline:

[Baseline models were trained on the preprocessed data - not the raw data - this decision was influenced by the following: 'A healthy baseline begins with clean and well-preprocessed data...Ensuring data quality lays the groundwork for meaningful baseline comparisons.' (iguazio, 2024)]

In [23]:
```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import precision_recall_fscore_support as score
#random baseline: calculated on validation data
dummy_clf = DummyClassifier(strategy="uniform")
#the sklearn docs define 'uniform' as follows:
#'generates predictions uniformly at random from the list of unique classes observed in y, i.e. each class has equal probability.'
#(scikit learn, 2024)
dummy_clf.fit(x_train_scaled, y_train)
y_pred_dummy = dummy_clf.predict(x_val)
precision,recall,fscore,support=score(y_val,y_pred_dummy,average='macro')
val_common_sense_baseline = fscore
val_common_sense_baseline
```

Out[23]:
```
0.250846781011201
```

In [24]:
```
#random baseline: calculated on test data
dummy_clf = DummyClassifier(strategy="uniform")
#the sklearn docs define 'uniform' as follows:
#'generates predictions uniformly at random from the list of unique classes observed in y, i.e. each class has equal probability.'
#(scikit learn, 2024)
dummy_clf.fit(x_train_scaled, y_train)
y_pred_dummy = dummy_clf.predict(x_test)
precision,recall,fscore,support=score(y_test,y_pred_dummy,average='macro')
test_common_sense_baseline = fscore
test_common_sense_baseline
```

Out[24]:
```
0.24845913243835804
```

This identifies:

- a validation data 'common sense baseline' (random baseline, f1 score) of 0.25
- a test data 'common sense baseline' (random baseline, f1 score) of 0.25

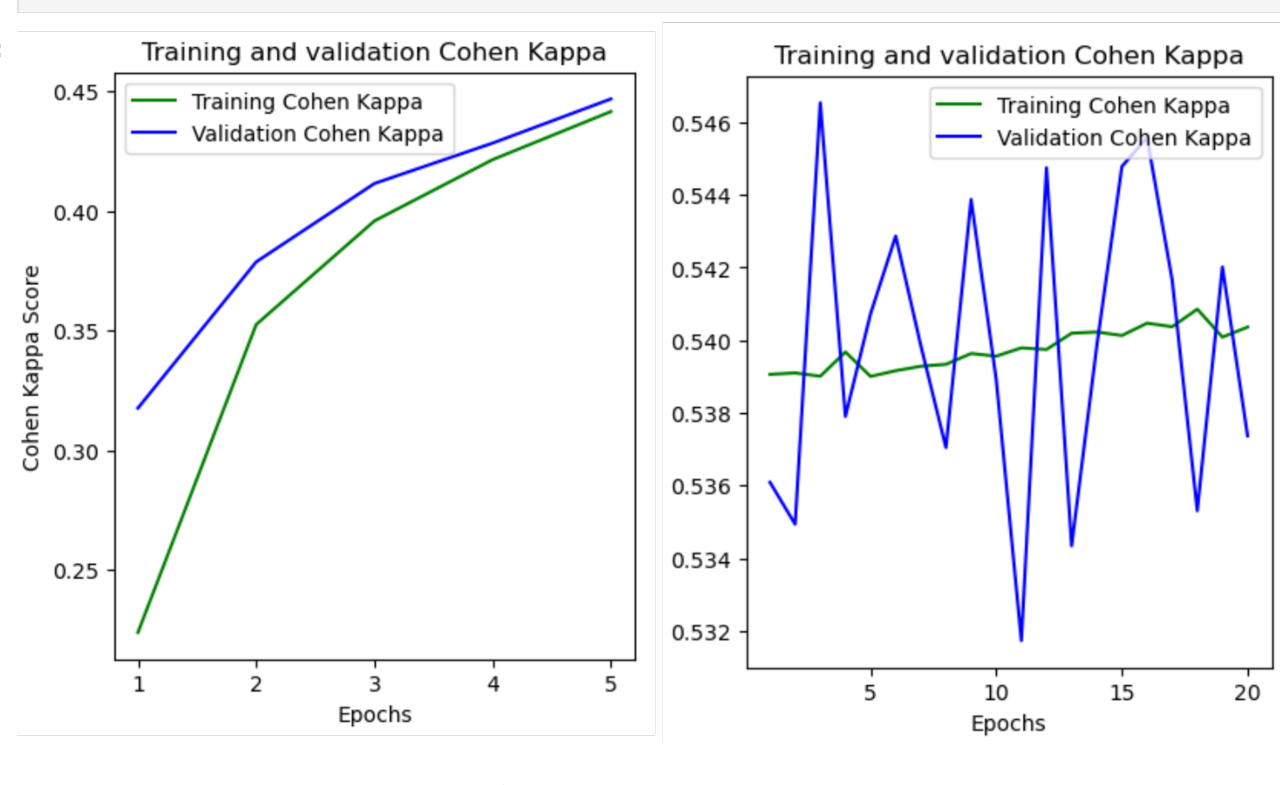
I defined a custom callback to pass to the 'callbacks' parameter of keras.model.fit ; this custom callback displays a plot of the training/validation loss and the training/validation evaluation metrics (F1 and Cohen Kappa):

In [40]:
```
from keras.callbacks import Callback
import numpy as np
import matplotlib.pyplot as plt

class CustomCallback(Callback):

    def __init__(self):
          fig, (ax1, ax2, ax3, ax4) = plt.subplots(1,4, figsize=(15,5))
          self.ax1 = ax1
          self.ax2 = ax2
          self.ax3 = ax3
          self.ax4 = ax4

    def on_train_end(self, logs=None):
        plt.tight_layout()
        self.plotLossOverview()
        self.plotLossDetail()
        self.plotAccuracy()
        self.plotKappa()

    #the following function 'plotLoss' has been modified sourced from the textbook "Deep Learning with Python"
    #graphs are not plotted on sub axis
    def plotLossOverview(self):
        loss = self.model.history.history['loss']
        val_loss = self.model.history.history['val_loss']
        epochs = range(1, len(loss) + 1)
        self.ax1.plot(epochs, loss, 'g', label='Training loss')
        self.ax1.plot(epochs, val_loss, 'b', label='Validation loss')
        self.ax1.set_title('Training and validation loss - Overview')
        self.ax1.set_xlabel('Epochs')
        self.ax1.set_ylabel('Loss')
        #specify y axis to make the results comparable
        self.ax1.set_ylim(0,1.7)
        self.ax1.legend()

    def plotLossDetail(self):
        loss = self.model.history.history['loss']
        val_loss = self.model.history.history['val_loss']
        epochs = range(1, len(loss) + 1)
        self.ax2.plot(epochs, loss, 'g', label='Training loss')
        self.ax2.plot(epochs, val_loss, 'b', label='Validation loss - Detail')
        self.ax2.set_title('Training and validation loss')
        self.ax2.set_xlabel('Epochs')
        self.ax2.set_ylabel('Loss')
        self.ax2.legend()

        #add an identifier ready to compare results
        caption_txt = f'^ {len(self.model.layers)-2} Hidden layers, {max(epochs)} epochs, {[self.model.layers[x].units for x in range(0, len(self.model.layers)-1) if self.model.layers[x].__class_
        self.ax2.text(0.9, -0.2, caption_txt, size=12,
                   transform=self.ax2.transAxes, horizontalalignment='right')

    #the following function 'plotAccuracy' has been modified from the textbook "Deep Learning with Python"
    def plotAccuracy(self):
        f1 = self.model.history.history['f1_score']
        val_f1 = self.model.history.history['val_f1_score']
        #each item in the history[f1_score] is an array of values - the summary during training displays this array as the mean
        f1 = [np.mean(x) for x in f1]
        val_f1 = [np.mean(x) for x in val_f1]
        loss = self.model.history.history['loss']
        epochs = range(1, len(loss) + 1)
        self.ax3.plot(epochs, f1, 'g', label='Training F1')
        self.ax3.plot(epochs, val_f1, 'b', label='Validation F1')
        self.ax3.set_title('Training and validation F1 Score')
        self.ax3.set_xlabel('Epochs')
        self.ax3.set_ylabel('F1 Score')
        self.ax3.legend()
        self.ax3.set_ylim(0,1)

    def plotKappa(self):
        kappa = self.model.history.history['cohen_kappa']
        val_kappa = self.model.history.history['val_cohen_kappa']
        loss = self.model.history.history['loss']
        epochs = range(1, len(loss) + 1)
        self.ax4.plot(epochs, kappa, 'g', label='Training Cohen Kappa')
        self.ax4.plot(epochs, val_kappa, 'b', label='Validation Cohen Kappa')
        self.ax4.set_title('Training and validation Cohen Kappa')
        self.ax4.set_xlabel('Epochs')
        self.ax4.set_ylabel('Cohen Kappa Score')
        self.ax4.legend()
        self.ax4.set_ylim(-1,1)
```

The custom callback above originally used matplotlib's autoscaling axis feature; however during experimentation I realised that this resulted in learning curves that were difficult to compare: (an example is provided below)

In [41]:
```
from IPython.display import Image
Image(filename="./images/learning_curve_scale_issue.png", width='700')
```

Out[41]:



In order to make the results (learning curves) more comparable I updated the custom callback above to specify axis size. I added an overview and a detailed plot of the training validation loss as this may help to compare losses and identify overfitting.

In [42]:
```
from keras import models
from keras import layers
from keras import metrics
import tensorflow_addons as tfa
import tensorflow as tf

class NeuralNet:
    #layers should be an array of keras.layers.Dense / keras.layers.Dropout
    def __init__(self, layersArr):
        self.model = models.Sequential(layersArr)
        self.call_back_functions = [CustomCallback()]
        self.hyperparameters = {
            "optimizer": None,
            "learning_rate": None,
            "batch_size": None,
            "epochs_number": None,
            "loss_function": None,
            #weight initialisation technique

            #the following 2 are covered by layersArr ?
            #number of hidden layer
            #number of neurons in hidden layer
            #activation functions
        }
        #as the evaluation metrics are predefined to compare models - I shall assign them at initialisation
        #precision and recall are only for binary classfication?
        #tf.keras.metrics.Precision(), tf.keras.metrics.Recall(),
        self.metricsArr = [tf.keras.metrics.F1Score(), tfa.metrics.CohenKappa(num_classes=num_classes)],

    def compile_model(self):
        self.model.compile(loss=self.hyperparameters['loss_function'],
                                        optimizer=self.hyperparameters['optimizer'],
                                        metrics=self.metricsArr)

    def fit(self, x_train, y_train, validation_data, verboseParam=1, isCallback=True):
        if (isCallback):
            self.history = self.model.fit(x_train, y_train,
                                        validation_data = validation_data,
                                        epochs=self.hyperparameters['epochs_number'],
                                        batch_size=self.hyperparameters['batch_size'],
                                        callbacks=self.call_back_functions,
                                        verbose=verboseParam
                                        )

        else:
            self.history = self.model.fit(x_train, y_train,
                                        validation_data = validation_data,
                                        epochs=self.hyperparameters['epochs_number'],
                                        batch_size=self.hyperparameters['batch_size'],
                                        verbose=verboseParam
                                        )

            plt.close()

    def test(self, x_test, y_test):
        #LOOK INTO ANY MORE PARAMETERS FOR THIS
        #NEED TO RETURN THE RESULTS OR ASSIGN THEM TO A CLASS PROPERTY ?
        test_loss, test_acc, *remaining = self.model.evaluate(x_test, y_test)
        return (test_loss, test_acc, *remaining)

    #Setter functions
    def set_hyperparameters(self, new_hyperparameters_obj):
        for key, value in new_hyperparameters_obj.items():
            self.hyperparameters[key] = value


    def add_call_back_function(self, newFunc):
        self.call_back_functions.append(newFunc)
```

Model definition 1: trained on SMOTETomek data

Experimentation with different hyperparameters to find the smallest capacity model that beats the statistical baseline:

I created a file to keep track of the values used in the experiment - this also allowed me to calculate the mean for each row in the table
This table will be used to identify the smallest capacity model with statistical power:

The initial model was created in line with the suggestions from chapter 4 of the textbook 'Deep Learning with Python'; the last layer activation function is softmax and the loss function is categorical cross entropy:

In [63]:
```
from IPython.display import Image
Image(filename="./images/textbook_architecture.png", width='500')
```

Out[63]:

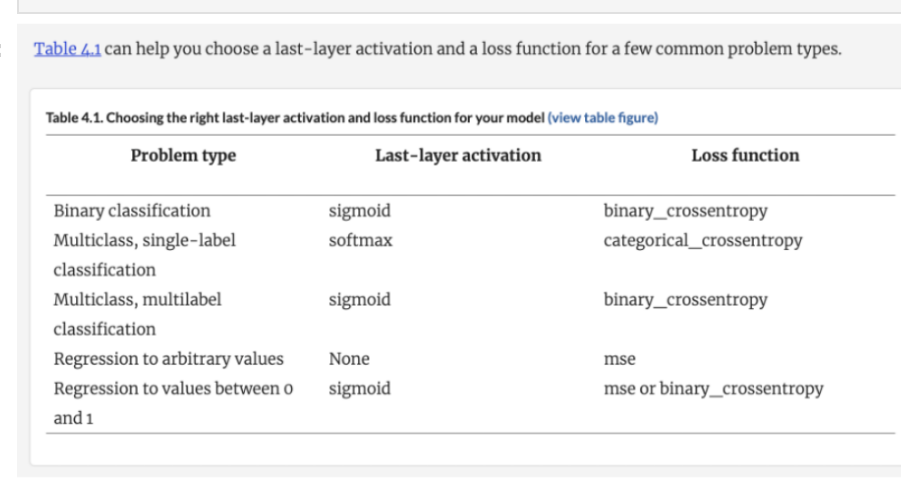


Table 4.1 can help you choose a last-layer activation and a loss function for a few common problem types.

Table 4.1. Choosing the right last-layer activation and loss function for your model (view table figure)

Problem type	Last-layer activation	Loss function
Binary classification	sigmoid	binary_crossentropy
Multiclass, single-label classification	softmax	categorical_crossentropy
Multiclass, multilabel classification	sigmoid	binary_crossentropy
Regression to arbitrary values	None	mse
Regression to values between 0 and 1	sigmoid	mse or binary_crossentropy

Image source: (Chollet, 2018)
[The problem type of this coursework is multiclass, single-label classification]

I selected the relu activation function for the hidden layer(s) of the initial model ; this decision factored in that the alternatives sigmoid and Tanh are susceptible to the vanishing gradients problem (Brownlee, 2021). A small (vanishing) gradient can negatively impact how effectively the weights and biases are updated after each epoch (Wang, 2019) ; this could result in slower convergence (Engati, 2024). Another potential advantage of the relu activation function is that it is non-saturating (compared to sigmoid which can saturate at -1 and 1) (Turp, 2022).

In [43]:
```
#code used to initialise the file
stage_5_results = pd.DataFrame(columns=['num_hidden_layers', 'num_units', 'optimizer', 'learning_rate',
                                        'activation_function', 'batch_size', 'num_epochs', 'regularization', 'val_f1_score'])
stage_5_results.to_csv('stage_5_results', index=False)

#load the results file ready to add to it
stage_5_results = pd.read_csv('stage_5_results')
```

In line with the aim of finding the smallest capacity model that beats the statistical baseline - I opted to use 10 units for each hidden layer. A member of the TensorFlow Team commented in the TensorFlow Forum that 'It is generally advisable to start with a small number of units and gradually increase complexity if necessary.' (Ch, 2023)

Try a combination of number of layers and epochs to find an underfitting model: Keep the number of hidden layer units relatively low as perhaps more likely to result in an overfitting model:

In [44]:
```
epochs = [5,10,15,20,25,30]
num_units = 10

layersArr = [
    [
    layers.Dense(num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ],

    [
    layers.Dense(num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_units, activation='relu'), #1 hidden layer
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ],

    [
    layers.Dense(num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_units, activation='relu'), #1 hidden layer
    layers.Dense(num_units, activation='relu'), #2 hidden layer
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ],

    [
    layers.Dense(num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_units, activation='relu'), #1 hidden layer
    layers.Dense(num_units, activation='relu'), #2 hidden layer
    layers.Dense(num_units, activation='relu'), #3 hidden layer
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ],

]

for epoch_num in epochs:
    for layer in layersArr:

        initial_hyperparameters = {
            "optimizer": 'rmsprop',
            "learning_rate": None,
            "batch_size": 128,
            "epochs_number": epoch_num,
            "loss_function": 'categorical_crossentropy',
            #use categorical cross entropy as targets are one-hot encoded
        }

        model_1 = NeuralNet(layer)
        model_1.set_hyperparameters(initial_hyperparameters)
        model_1.compile_model()
        model_1.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0, isCallback=False)

        results = pd.DataFrame({
            'num_hidden_layers':[len(model_1.model.layers)-2],
            'num_units': num_units,
            'optimizer': 'rmsprop',
            'learning_rate': 0.001, #currently using the keras default rmsprop learning rate
            'activation_function': 'relu',
            'batch_size': 128,
            'num_epochs': epoch_num,
            'regularization': False,
            'val_f1_score':[np.mean(model_1.history.history['val_f1_score'][-1])]
        })
        stage_5_results = pd.concat([stage_5_results, results])

stage_5_results.to_csv('stage_5_results', index=False)
```

In [45]:
```
stage_5_results.sort_values('val_f1_score', ascending=False)
```

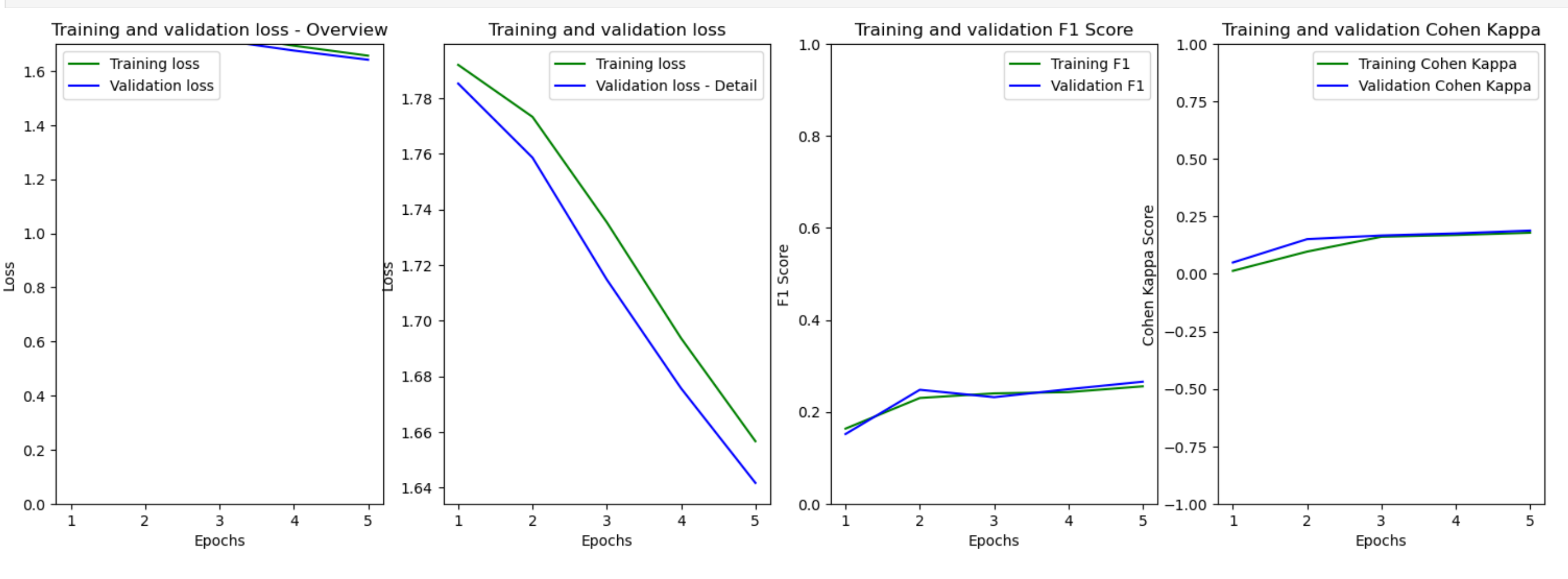
Out[45]:

	num_hidden_layers	num_units	optimizer	learning_rate	activation_function	batch_size	num_epochs	regularization	val_f1_score
0	1	10	rmsprop	0.001	relu	128	25	False	0.599177
0	2	10	rmsprop	0.001	relu	128	30	False	0.588983
0	1	10	rmsprop	0.001	relu	128	30	False	0.587181
0	1	10	rmsprop	0.001	relu	128	20	False	0.571100
0	0	10	rmsprop	0.001	relu	128	30	False	0.564644
0	2	10	rmsprop	0.001	relu	128	25	False	0.560880
0	3	10	rmsprop	0.001	relu	128	25	False	0.548413
0	0	10	rmsprop	0.001	relu	128	25	False	0.547732
0	1	10	rmsprop	0.001	relu	128	15	False	0.544056
0	3	10	rmsprop	0.001	relu	128	20	False	0.542709
0	3	10	rmsprop	0.001	relu	128	30	False	0.537457
0	2	10	rmsprop	0.001	relu	128	20	False	0.536040
0	3	10	rmsprop	0.001	relu	128	15	False	0.529207
0	2	10	rmsprop	0.001	relu	128	15	False	0.527253
0	0	10	rmsprop	0.001	relu	128	20	False	0.489171
0	2	10	rmsprop	0.001	relu	128	10	False	0.483957
0	3	10	rmsprop	0.001	relu	128	10	False	0.480803
0	0	10	rmsprop	0.001	relu	128	15	False	0.464227
0	1	10	rmsprop	0.001	relu	128	10	False	0.455896
0	0	10	rmsprop	0.001	relu	128	10	False	0.432506
0	1	10	rmsprop	0.001	relu	128	5	False	0.396564
0	3	10	rmsprop	0.001	relu	128	5	False	0.378242
0	2	10	rmsprop	0.001	relu	128	5	False	0.352718
0	0	10	rmsprop	0.001	relu	128	5	False	0.334771

The smallest capacity model that beats the statistical baseline is the model in the last row in the table above;
5 training epochs, 0 hidden layers (the model consists of an input and output layer), a val_f1_score of 0.525.
The following cell outputs the Training/Validation Loss Curve; this curve will be used to check that the model is underfitting - as suggested in the Deep Learning Workflow:

In [46]:
```
initial_hyperparameters = {
            "optimizer": 'rmsprop',
            "learning_rate": None,
            "batch_size": 128,
            "epochs_number": 5,
            "loss_function": 'categorical_crossentropy',
            #use categorical cross entropy as targets are one-hot encoded
        }
initial_layers = [
    layers.Dense(10, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ]
```

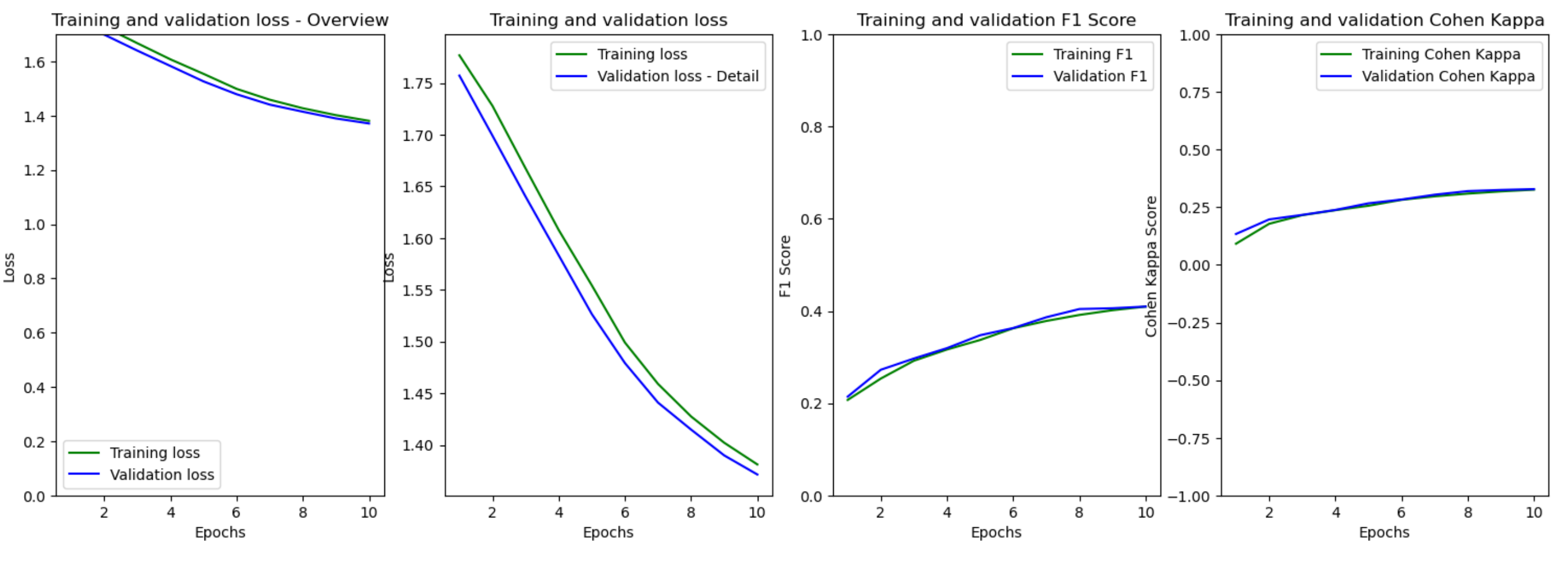
```
model_1 = NeuralNet(initial_layers)
model_1.set_hyperparameters(initial_hyperparameters)
model_1.compile_model()
model_1.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 0 Hidden layers, 5 epochs, [10] units ^

Looking at the graphs above, the validation loss is lower than the training loss - this is an unexpected behaviour given that I have not implemented any form of regularisation / dropout yet (Rosebrock, 2019). Author of the O'Reilly book 'Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow', Aurélien Geron, suggests the following possible cause: 'the training loss is measured during each epoch, whilst the validation loss is measured after each epoch, so on average the training loss is measured 1/2 an epoch earlier.' (Rosebrock, 2019) To further examine this discrepancy between training and validation loss trained the model for 10 epochs:

In [47]:
```
initial_hyperparameters = {
            "optimizer": 'rmsprop',
            "learning_rate": None,
            "batch_size": 128,
            "epochs_number": 10,
            "loss_function": 'categorical_crossentropy',
            #use categorical cross entropy as targets are one-hot encoded
        }
initial_layers = [
    layers.Dense(10, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ]

model_1 = NeuralNet(initial_layers)
model_1.set_hyperparameters(initial_hyperparameters)
model_1.compile_model()
model_1.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



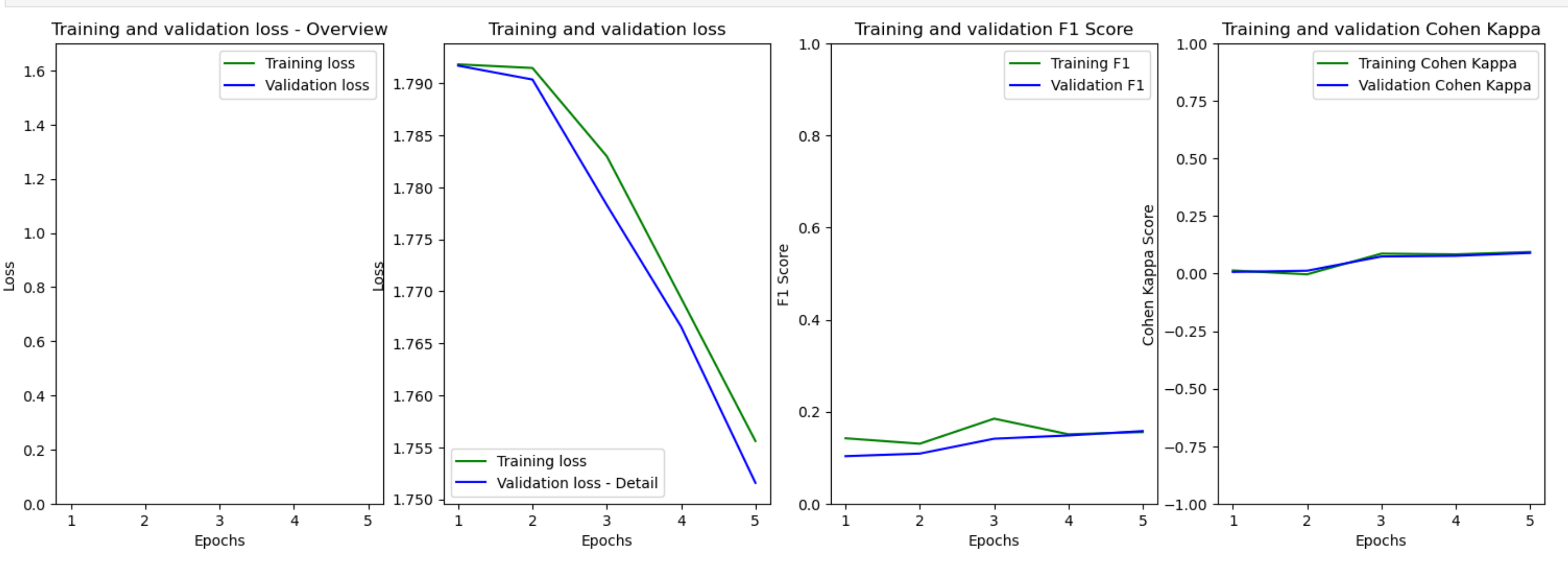
^ 0 Hidden layers, 10 epochs, [10] units ^

The training/validation loss pictured above appears to be congruent with Aurélien Geron's suggestion ; if the training loss were to be shifted 1/2 an epoch to the left (as Geron suggests), the validation loss would no longer be lower than the training loss.

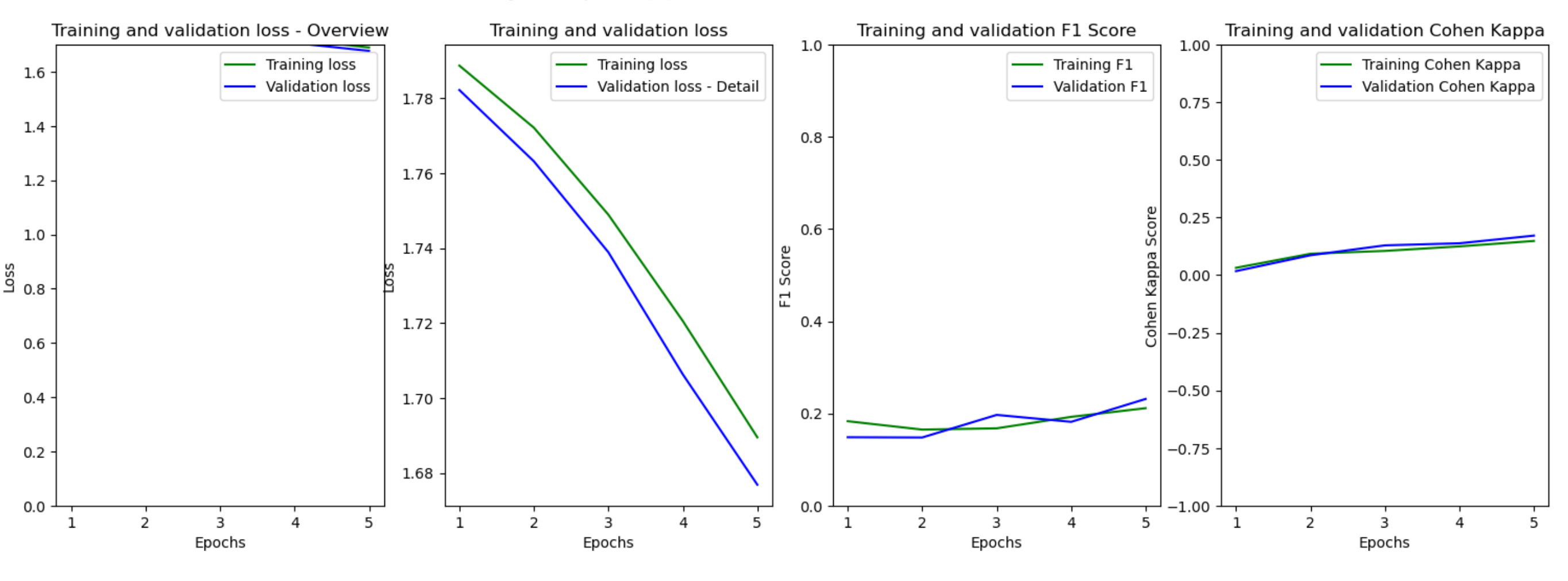
Identification of a smaller capacity model based:

Based on the, a neural network with 0 hidden layers (consisting of only an input and output layer) is sufficient to beat the statistical baseline identified (F1 Score: 0.25). I decided to further explore if I could identify a smaller capacity model (a model with fewer units) that beats the statistical baseline:

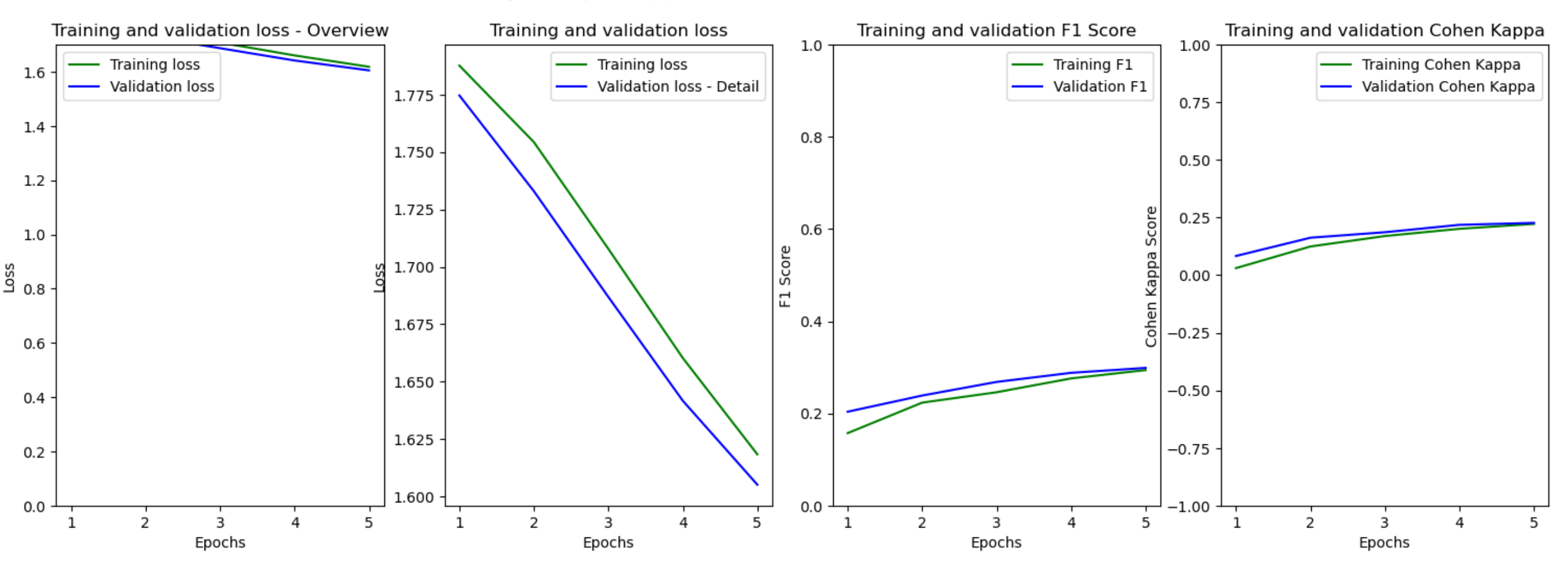
In [48]:
```
units = [2,4,6,8]
stage_5_results = pd.read_csv('stage_5_results')

for unit in units:
    initial_hyperparameters = {
                "optimizer": 'rmsprop',
                "learning_rate": None,
                "batch_size": 128,
                "epochs_number": 5,
                "loss_function": 'categorical_crossentropy',
                #use categorical cross entropy as targets are one-hot encoded
            }
    initial_layers = [
        layers.Dense(unit, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
        layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
        ]

    model_1 = NeuralNet(initial_layers)
    model_1.set_hyperparameters(initial_hyperparameters)
    model_1.compile_model()
    model_1.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)

    results = pd.DataFrame({
                'num_hidden_layers':[len(model_1.model.layers)-2],
                'num_units': unit,
                'optimizer': 'rmsprop',
                'learning_rate': 0.001, #currently using the keras default rmsprop learning rate
                'activation_function': 'relu',
                'batch_size': 128,
                'num_epochs': 5,
                'regularization': False,
                'val_f1_score':[np.mean(model_1.history.history['val_f1_score'][-1])]
            })
    stage_5_results = pd.concat([stage_5_results, results])

stage_5_results.to_csv('stage_5_results', index=False)
```



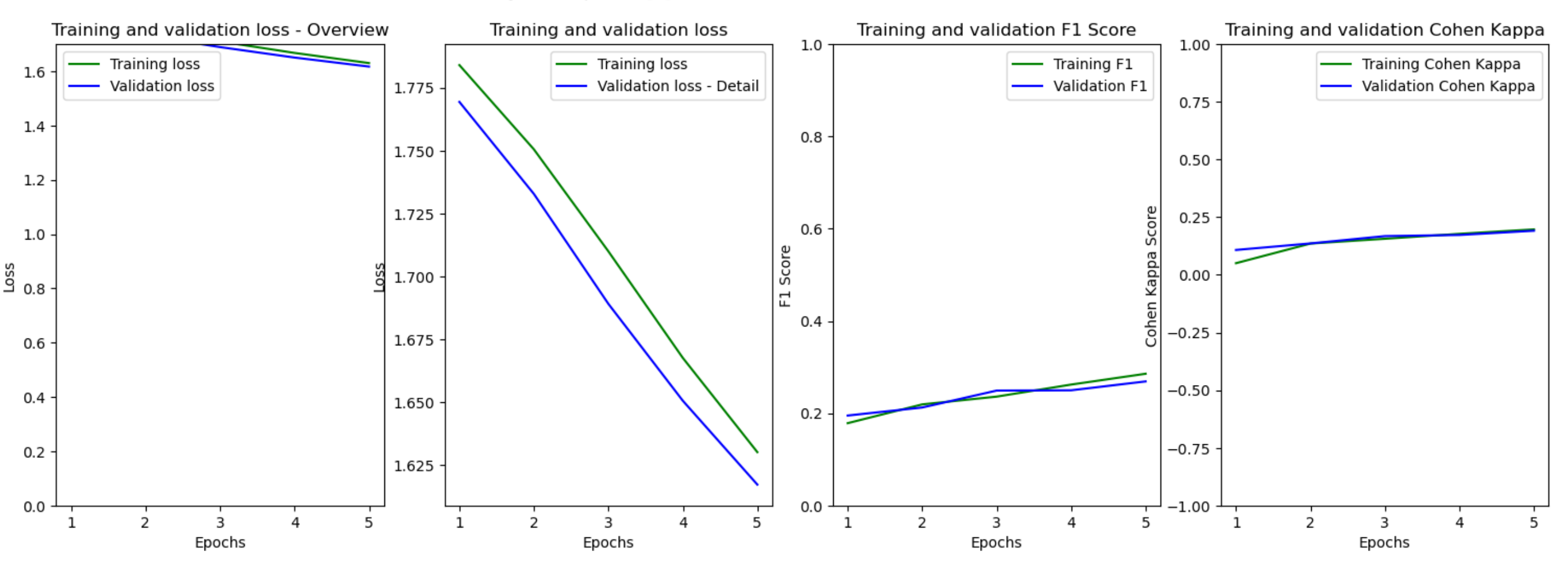
^ 0 Hidden layers, 5 epochs, [2] units ^



^ 0 Hidden layers, 5 epochs, [4] units ^



^ 0 Hidden layers, 5 epochs, [6] units ^



^ 0 Hidden layers, 5 epochs, [8] units ^

The shape of the training loss curve indicates underfitting ; the trainig loss is on a trajectory to continue decreasing if training continues. This suggests the model is capable of further learning. (Brownlee, 2019) The training and validation loss are also relatively high at ~1.6 - this also suggests underfitting (Brownlee, 2019).

In [49]:
```
stage_5_results.sort_values(by='val_f1_score', ascending=True).head()
```

Out[49]:

	num_hidden_layers	num_units	optimizer	learning_rate	activation_function	batch_size	num_epochs	regularization	val_f1_score
0	0	2	rmsprop	0.001	relu	128	5	False	0.157849
0	0	4	rmsprop	0.001	relu	128	5	False	0.231503
0	0	8	rmsprop	0.001	relu	128	5	False	0.269033
0	0	6	rmsprop	0.001	relu	128	5	False	0.298736
0	0	10	rmsprop	0.001	relu	128	5	False	0.334771

A summary of the smallest capacity model beating the 'common sense baseline':

In [56]:
```
np.mean(model_1.model.history.history['val_f1_score'][-1]) #confirm that this model beats the statistical F1 baseline
```

Out[56]:
```
0.26734504
```

In [54]:
```
model_1.model.summary()
```

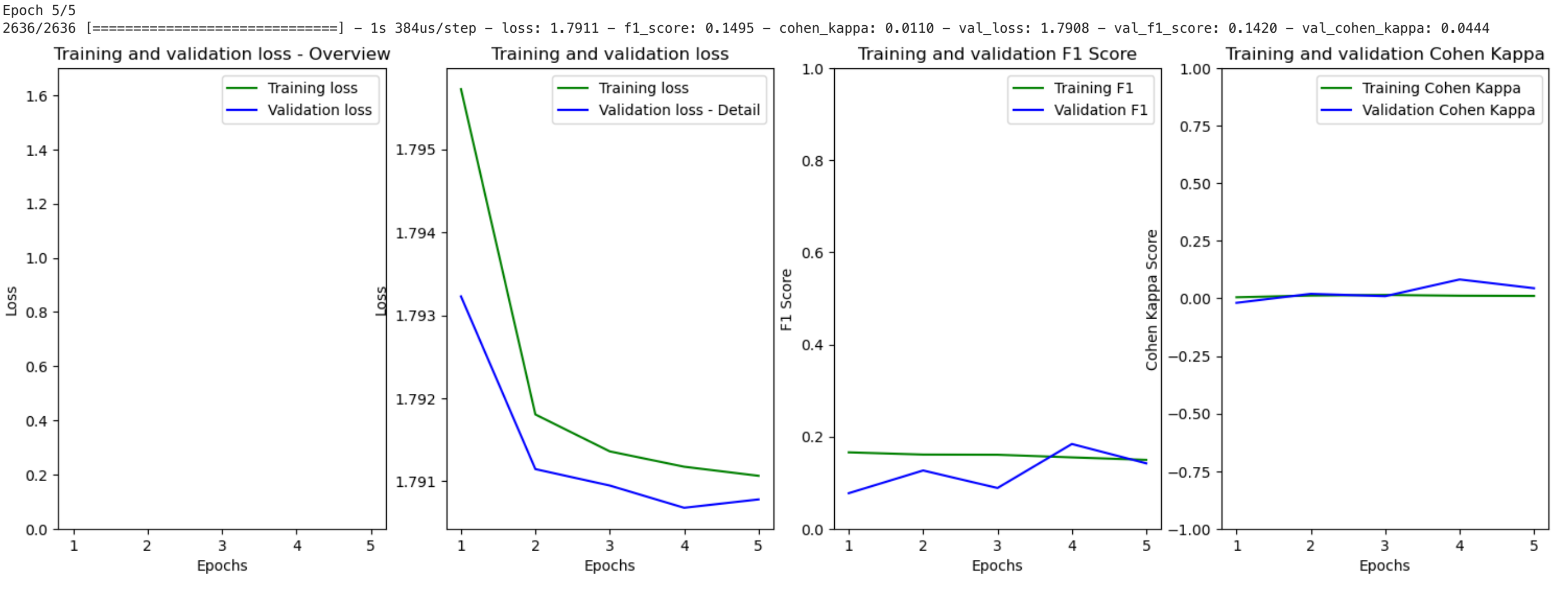
```
Model: "sequential_32"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_32 (Dense)            (None, 4)                 40

 dense_33 (Dense)            (None, 6)                 30

=================================================================
Total params: 70 (280.00 Byte)
Trainable params: 70 (280.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [30]:
```
underfitting_hyperparameters = {
            "optimizer": 'rmsprop',
            "learning_rate": None,
            "batch_size": 128,
            "epochs_number": 5,
            "loss_function": 'categorical_crossentropy',
            #use categorical cross entropy as targets are one-hot encoded
        }
underfitting_layers = [
    layers.Dense(4, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ]
```

Exploring the effect of activation functions of the intial model:

In [31]:
```
underfitting_layers_no_relu = [
    layers.Dense(4, input_shape=(9,)), #input shape == x_train_tensor[0].shape
    layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
    ]

model_1 = NeuralNet(underfitting_layers_no_relu)
model_1.set_hyperparameters(underfitting_hyperparameters)
model_1.compile_model()
model_1.fit(x_train_scaled, y_train, (x_val_scaled, y_val))
```

```
Epoch 1/5
2636/2636 [==============================] - 1s 436us/step - loss: 1.7957 - f1_score: 0.1658 - cohen_kappa: 0.0051 - val_loss: 1.7932 - val_f1_score: 0.0772 - val_cohen_kappa: -0.0190
Epoch 2/5
2636/2636 [==============================] - 1s 405us/step - loss: 1.7918 - f1_score: 0.1610 - cohen_kappa: 0.0126 - val_loss: 1.7911 - val_f1_score: 0.1265 - val_cohen_kappa: 0.0200
Epoch 3/5
2636/2636 [==============================] - 1s 387us/step - loss: 1.7914 - f1_score: 0.1606 - cohen_kappa: 0.0151 - val_loss: 1.7909 - val_f1_score: 0.0884 - val_cohen_kappa: 0.0099
Epoch 4/5
2636/2636 [==============================] - 1s 388us/step - loss: 1.7912 - f1_score: 0.1549 - cohen_kappa: 0.0119 - val_loss: 1.7907 - val_f1_score: 0.1840 - val_cohen_kappa: 0.0825
Epoch 5/5
2636/2636 [==============================] - 1s 384us/step - loss: 1.7911 - f1_score: 0.1495 - cohen_kappa: 0.0110 - val_loss: 1.7908 - val_f1_score: 0.1420 - val_cohen_kappa: 0.0444
```



^ 0 Hidden layers, 5 epochs, [4] units ^

[the graph on the top left appears blank as the training/validation loss is higher than 1.7 - as shown in the more detailed plot in the second column]

It appears the the relu activation had a significant impact on the validation F1 and Cohen Kappa scores ; removing the relu activation function has resulted in a model with no statistical power ; the val_f1_score of this model (0.1322) is lower than the common sense statistical baseline (0.21)

In [32]:
```
stage_5_results.sort_values('val_f1_score', ascending=False).head()
```

Out[32]:

	num_hidden_layers	num_units	optimizer	learning_rate	activation_function	batch_size	num_epochs	regularization	val_f1_score
21	1	10	rmsprop	0.001	relu	128	30	False	0.634199
17	1	10	rmsprop	0.001	relu	128	25	False	0.626490
9	1	10	rmsprop	0.001	relu	128	15	False	0.622296
15	3	10	rmsprop	0.001	relu	128	20	False	0.617044
23	3	10	rmsprop	0.001	relu	128	30	False	0.616332

The top ranking model had 1 hidden layers and was trained for 30 epochs; this may be a good point of comparison as the model is relatively simple (with only 1 hidden layer and 10 units).

Step 6: Scaling - Developing a model that overfits

Section 6 and 7: Table of F1 outcomes:

In [65]:
```
stage_6_results = pd.read_csv('./stage_6_results')
stage_6_results.sort_values(by='val_f1_score', ascending=False)
```

Out[65]:

	num_hidden_layers	num_units	optimizer	learning_rate	activation_function	batch_size	num_epochs	regularization	val_f1_score
11	1	600	rmsprop	0.001	relu	128	100	False	0.702647
14	1	600	rmsprop	0.001	relu	128	100	L2, alpha:1e-4	0.698807
25	4	600	rmsprop	0.001	relu	128	100	False	0.695391
19	4	600	rmsprop	0.001	relu	128	100	L1, alpha:1e-4	0.689719
22	4	600	rmsprop	0.001	relu	128	100	L2, alpha:1e-4	0.688726
24	3	600	rmsprop	0.001	relu	128	50	False	0.688507
26	4	600	rmsprop	0.001	relu	128	50	L1, alpha:1e-4	0.680557
20	4 hidden layers (HeNormal Init), 2 Dropout	600	rmsprop	0.001	relu	128	50	L1, alpha:1e-6	0.677918
10	1	600	rmsprop	0.001	relu	128	100	False	0.674576
9	1	400	rmsprop	0.001	relu	128	200	False	0.667377
8	1	200	rmsprop	0.001	relu	128	100	False	0.662064
12	4 hidden layers (HeUniform Init), 3 Dropout	600	rmsprop	0.001	relu	128	50	L1, alpha:1e-6	0.661044
21	4 hidden layers (HeNormal Init), 4 Dropout	600	rmsprop	0.001	relu	128	50	L1, alpha:1e-6	0.656148
23	4 Dense layers, 4 Dropout Layers	600	rmsprop	0.001	relu	128	50	L1, alpha:1e-6	0.653444
7	1	90	rmsprop	0.001	relu	128	200	False	0.649796
13	4 hidden layers (HeNormal Init), 4 Dropout	600	rmsprop	0.001	relu	128	50	L1, alpha:1e-6	0.649669
15	4 hidden layers (HeUniform Init), 3 Dropout	600	rmsprop	0.001	relu	128	20	L1, alpha:1e-6	0.645878
17	4 hidden layers (HeNormal Init), 3 Dropout	600	rmsprop	0.001	nadam	128	20	L1, alpha:1e-6	0.644136
6	1	90	rmsprop	0.001	relu	128	100	False	0.638381
4	0	300	rmsprop	0.001	relu	128	50	False	0.630724
3	0	90	rmsprop	0.001	relu	128	50	False	0.618967
2	0	70	rmsprop	0.001	relu	128	50	False	0.614982
0	0	30	rmsprop	0.001	relu	128	50	False	0.613648
16	4 hidden layers (HeNormal Init), 3 Dropout	600	rmsprop	0.001	elu	128	20	L1, alpha:1e-6	0.611494
1	0	50	rmsprop	0.001	relu	128	50	False	0.607879
18	4 hidden layers (HeNormal Init), 3 Dropout	600	adam	0.001	relu	128	20	L1, alpha:1e-6	0.588720
5	10	90	rmsprop	0.001	relu	128	50	False	0.412798

Section 6 and 7: Summary of F1 outcomes:

The table above has been sorted by val_f1_score to identify the top performing model during training. The top performing model consisted of 1 hidden layer, 600 units, rmsprop optimization (with a learning rate of 0.001), activation function relu, no regularization and was trained for 100 epochs.

Section 6 Training Log:

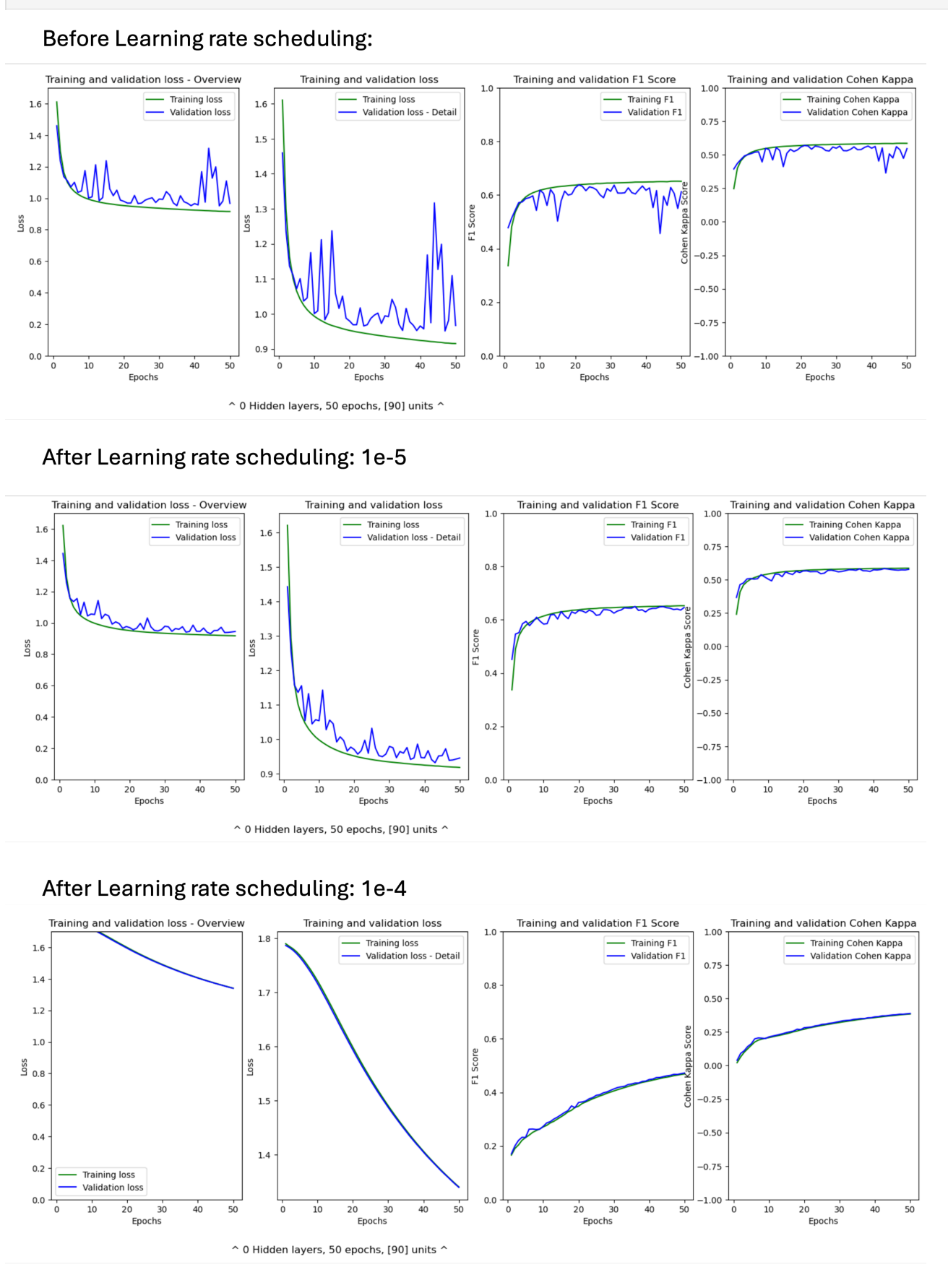
In this section I will modify the following 3 hyperparameters in attempt to achieve overfitting:

1. Number of units per layer
2. Number of hidden layers
3. Number of training epochs

As part of this phase I noticed the validation loss was fluctuating / presenting noise:
I experimented with a learning rate schedule to reduce this noise ; I implemented 'power scheduling' of the learning rate using the 'decay' parameter of tf.keras.optimizer.RMSprop. The decaying learning rate may be beneficial in this context as it may allow the model to search a large / noisy parameter space in the initial stages - and narrow down / reduce of the noise of the hyperparameter space once the correct direction for model weight changes has been identified. (Wang, 2022)

In [50]:
```
from IPython.display import Image
Image(filename="./images/smoothing_val_loss.png", width='700')
```

Out[50]:


Before Learning rate scheduling:

^ 0 Hidden layers, 50 epochs, [90] units ^

After Learning rate scheduling: 1e-5

^ 0 Hidden layers, 50 epochs, [90] units ^

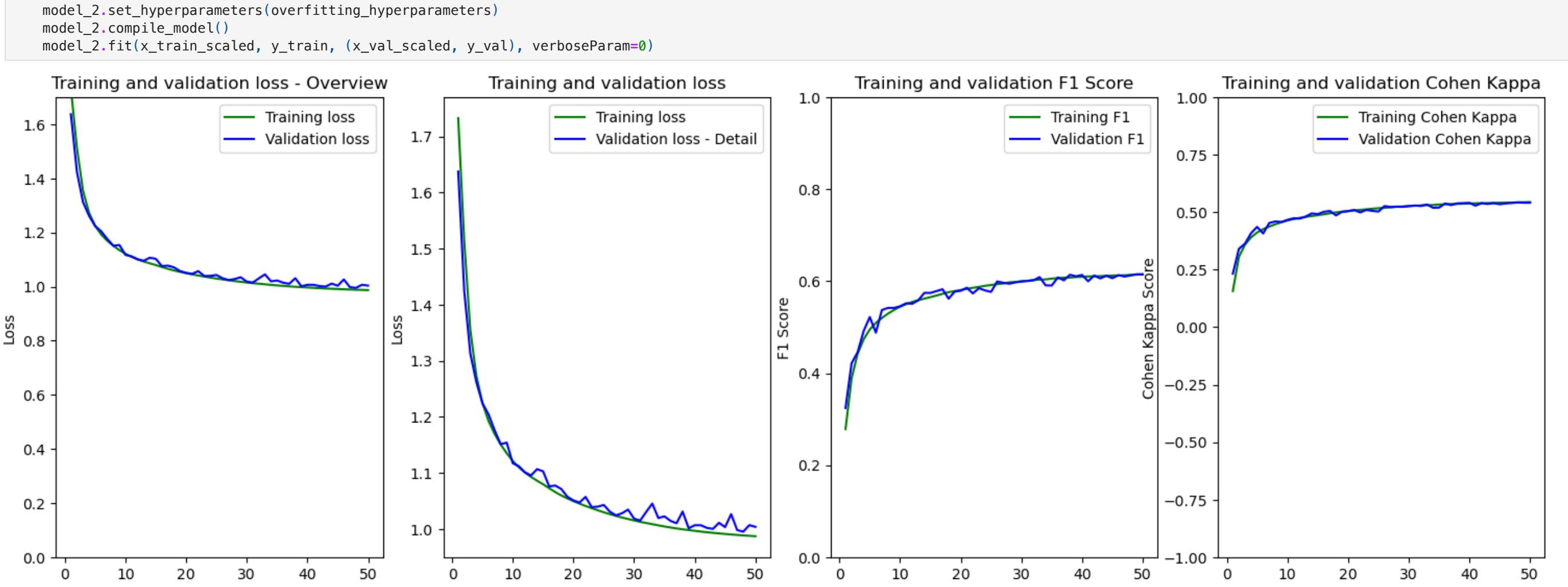
After Learning rate scheduling: 1e-4

^ 0 Hidden layers, 50 epochs, [90] units ^

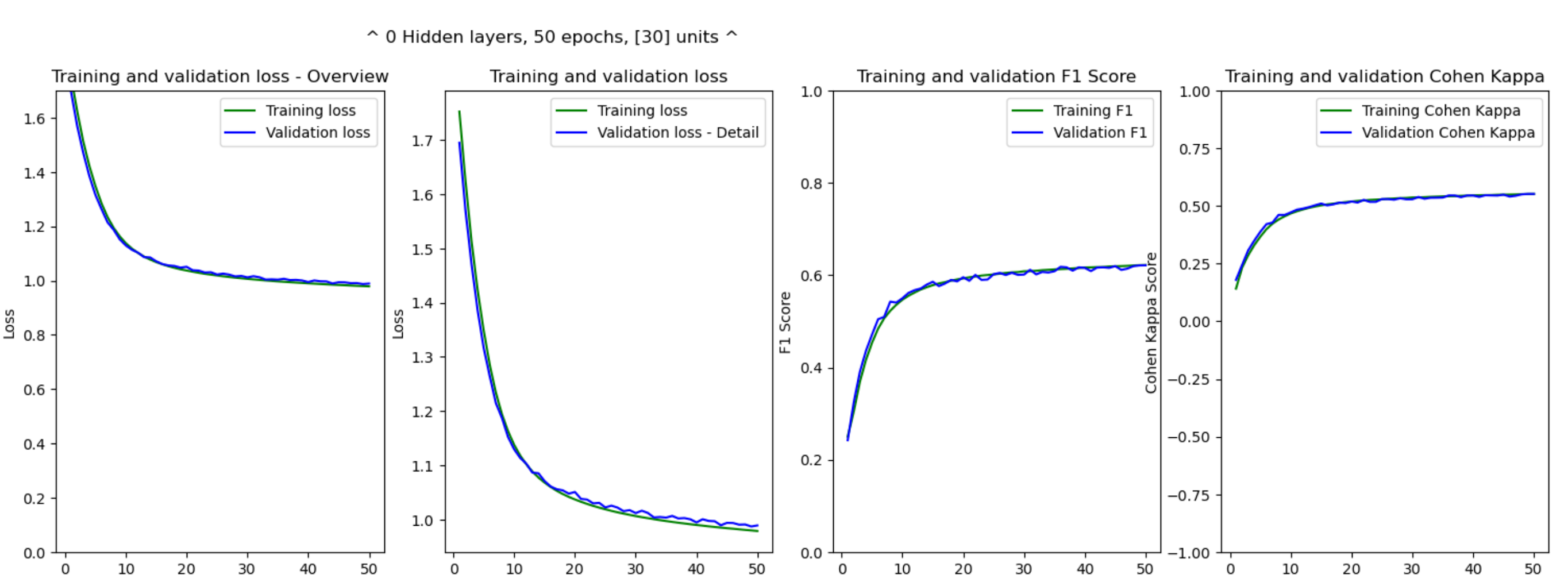
I found setting the RMSProp 'decay' to 1e-5 optimal ; this value reduced noise in the validation loss and did not negatively impact the evaluation metrics. [the decay argument of 1e-4 negatively impacted the evalution metrics - with the F1 score decreasing by ~0.2. It also increased the loss by ~0.4]
I could further experiment with this learning schedule using the 'momentum' parameter

Element 1: Number of units per layer:

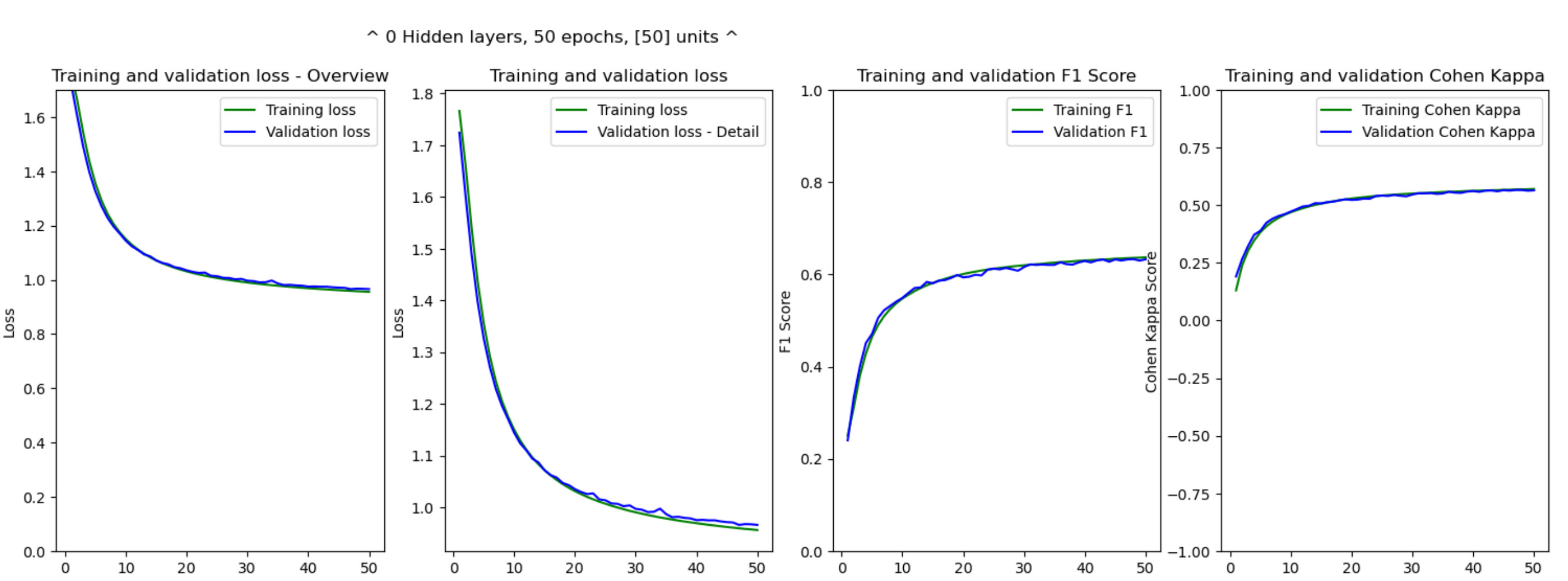
In [49]:
```
unit_options = [30, 50, 70, 90]
overfitting_hyperparameters = {
            #a tensorflow warning suggested using the legacy version of M1/M2 chips
            "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-5),
            "learning_rate": None,
            "batch_size": 128,
            "epochs_number": 50, #in order to better see the overall patten of loss I have increased the no of epochs
                                  #a better overview may be beneficial when attempting to identify overfitting
            "loss_function": 'categorical_crossentropy'
        }

for unit in unit_options:
    overfitting_layers = [
        layers.Dense(unit, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
        layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
        ]

    model_2 = NeuralNet(overfitting_layers)
    model_2.set_hyperparameters(overfitting_hyperparameters)
    model_2.compile_model()
    model_2.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



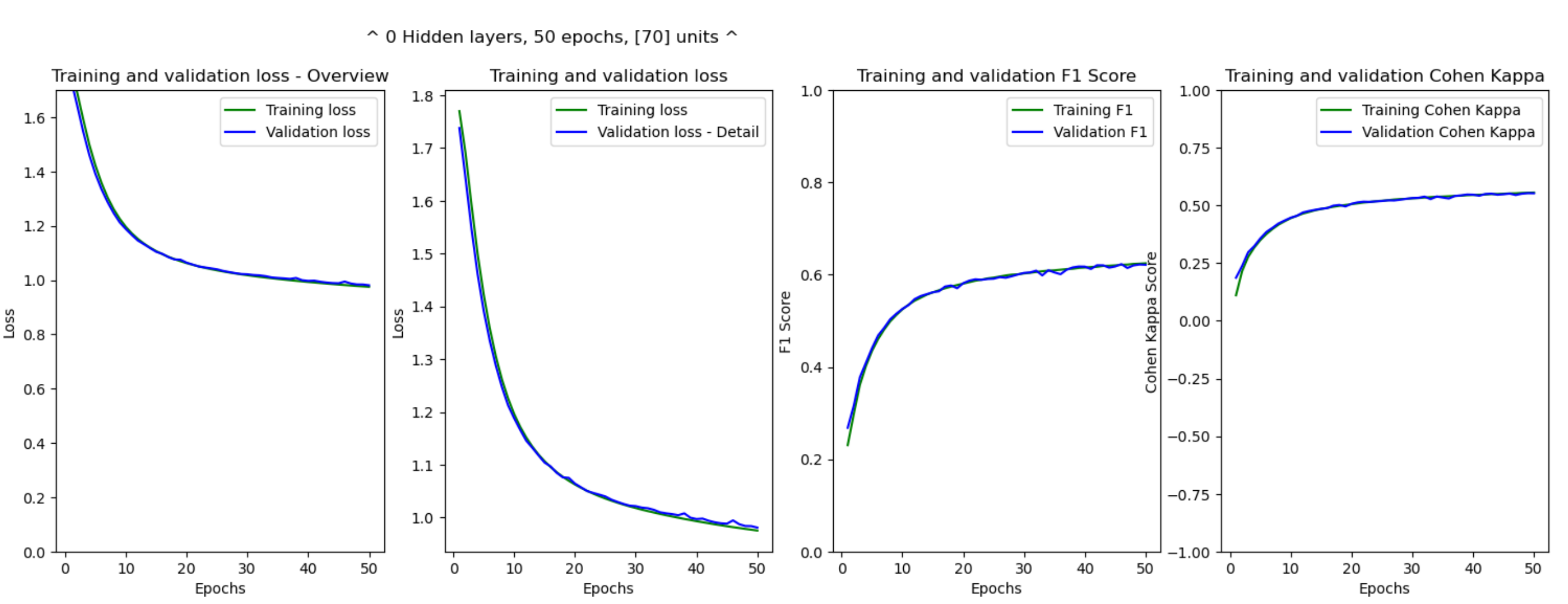
^ 0 Hidden layers, 50 epochs, [30] units ^



^ 0 Hidden layers, 50 epochs, [50] units ^



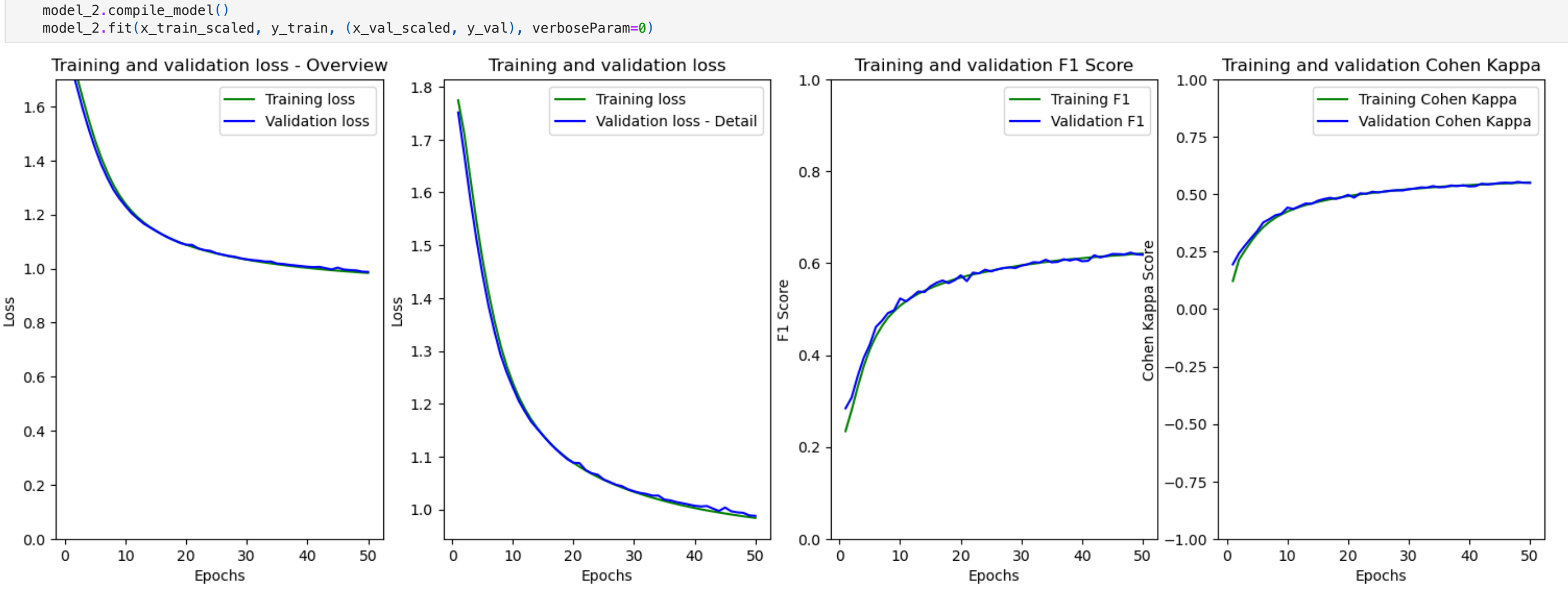
^ 0 Hidden layers, 50 epochs, [70] units ^



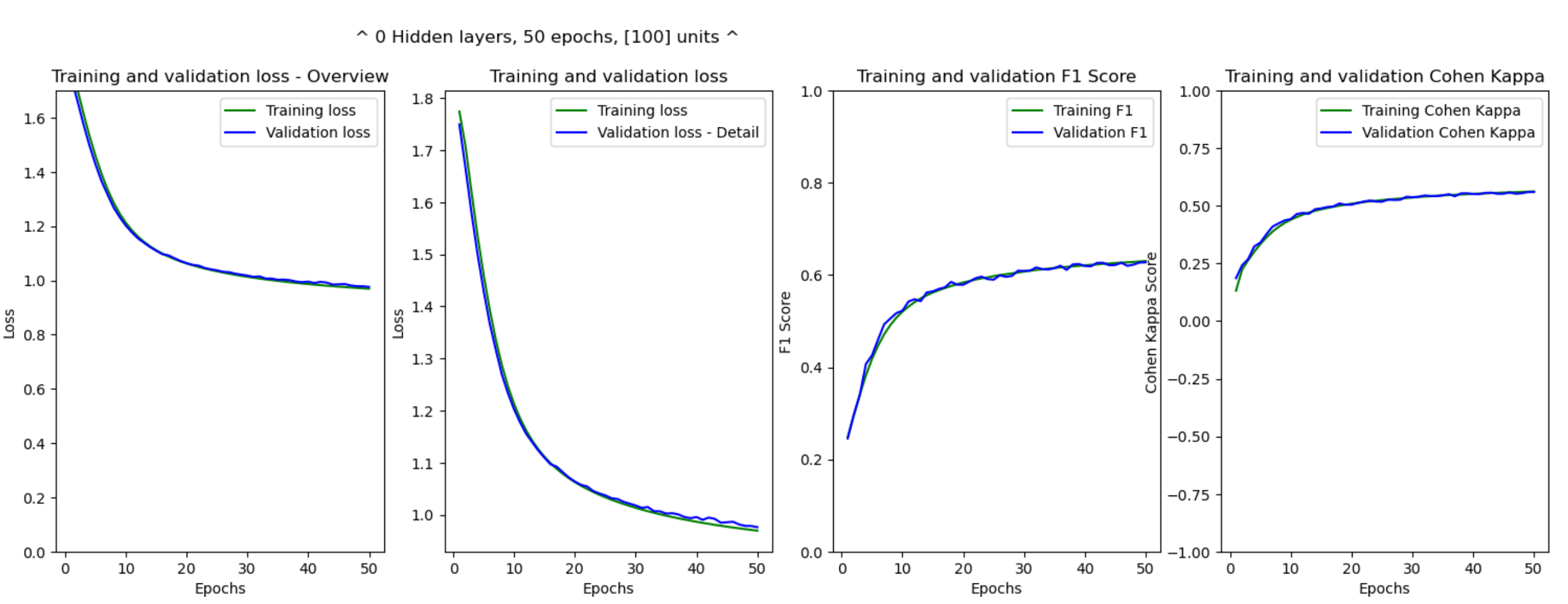
^ 0 Hidden layers, 50 epochs, [90] units ^

Looking at the second column of graphs above, the last row (90 units) acheives the lowest training/validation loss ; I will further increase the number of units to identify if a further increase in the number of units decreases the training/validation loss.

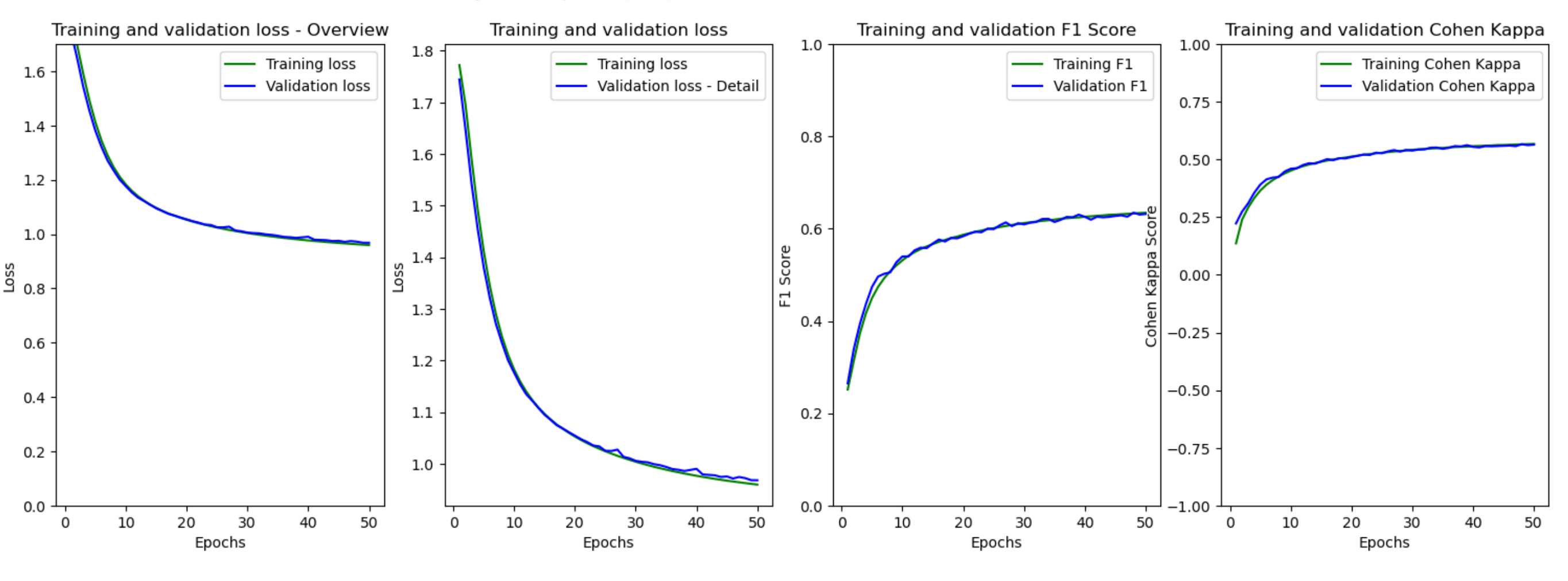
In [51]:
```
unit_options = [100,200,300]

for unit in unit_options:
    overfitting_layers = [
        layers.Dense(unit, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
        layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
        ]

    model_2 = NeuralNet(overfitting_layers)
    model_2.set_hyperparameters(overfitting_hyperparameters)
    model_2.compile_model()
    model_2.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 0 Hidden layers, 50 epochs, [100] units ^



^ 0 Hidden layers, 50 epochs, [200] units ^



^ 0 Hidden layers, 50 epochs, [300] units ^

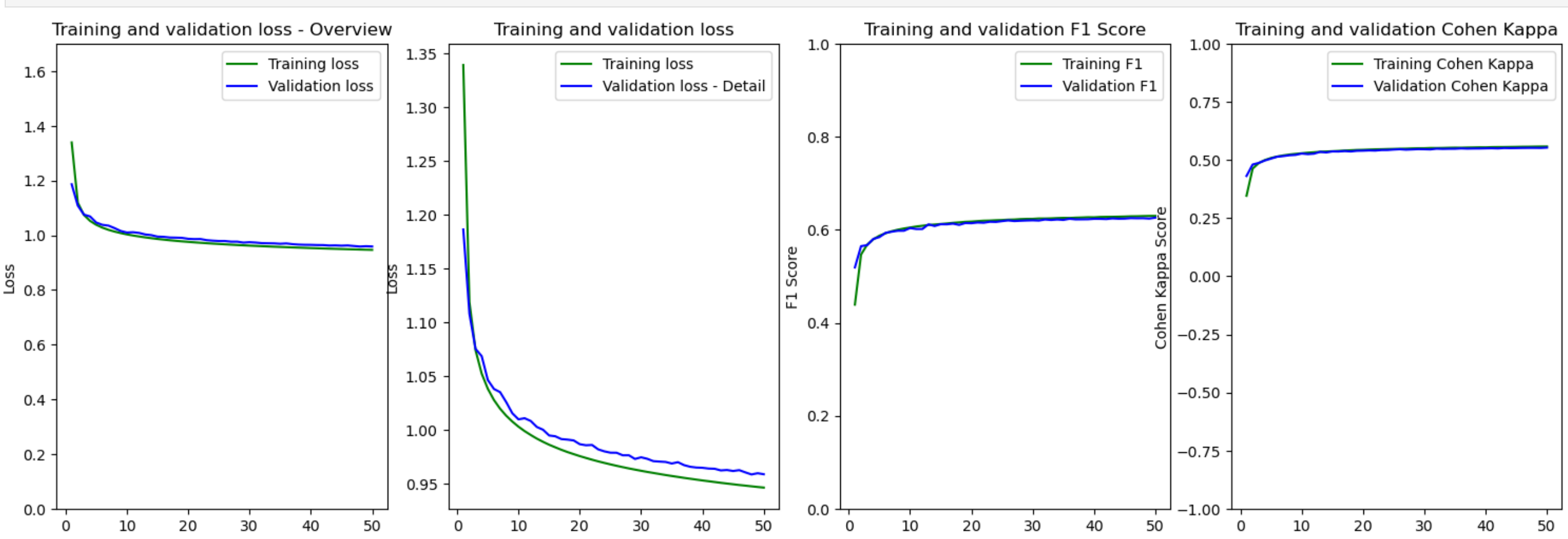
Performance appears to have plateaued at 90 units ; comparing the graphs in the second column, I can see no further decrease in the training loss after 100+ units (similarly I can see no further increase in the evaluation metrics after 100+ units).
I will begin the next set of experiments using the optimal number of units identified so far (300).

In [57]:
```
stage_6_num_units = 90
```

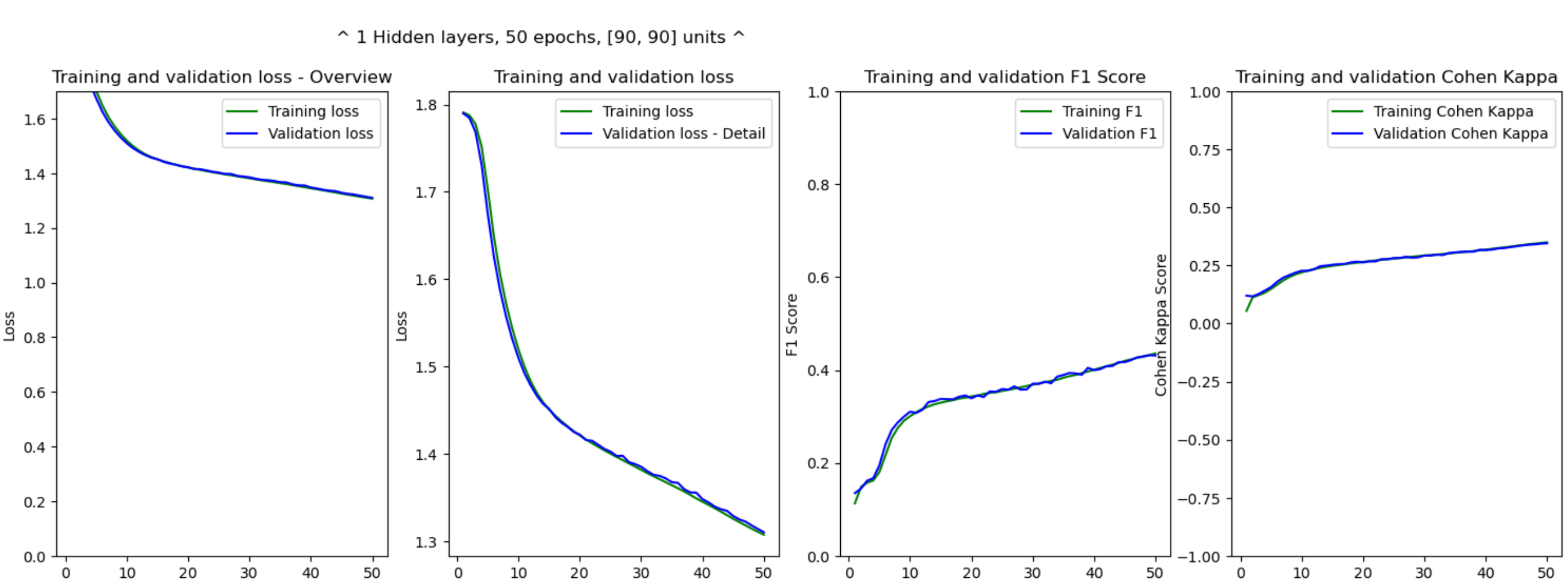
Element 2: Number of Hidden Layers:

When attempting to train a model with a first layer of 300 units, the training time increased by a large amount ; reviewing the benefits gained from increasing from 90 - 300 (a training loss decrease of ~0.1) I realised that this performance gain was outweighed by the large increase in training time.

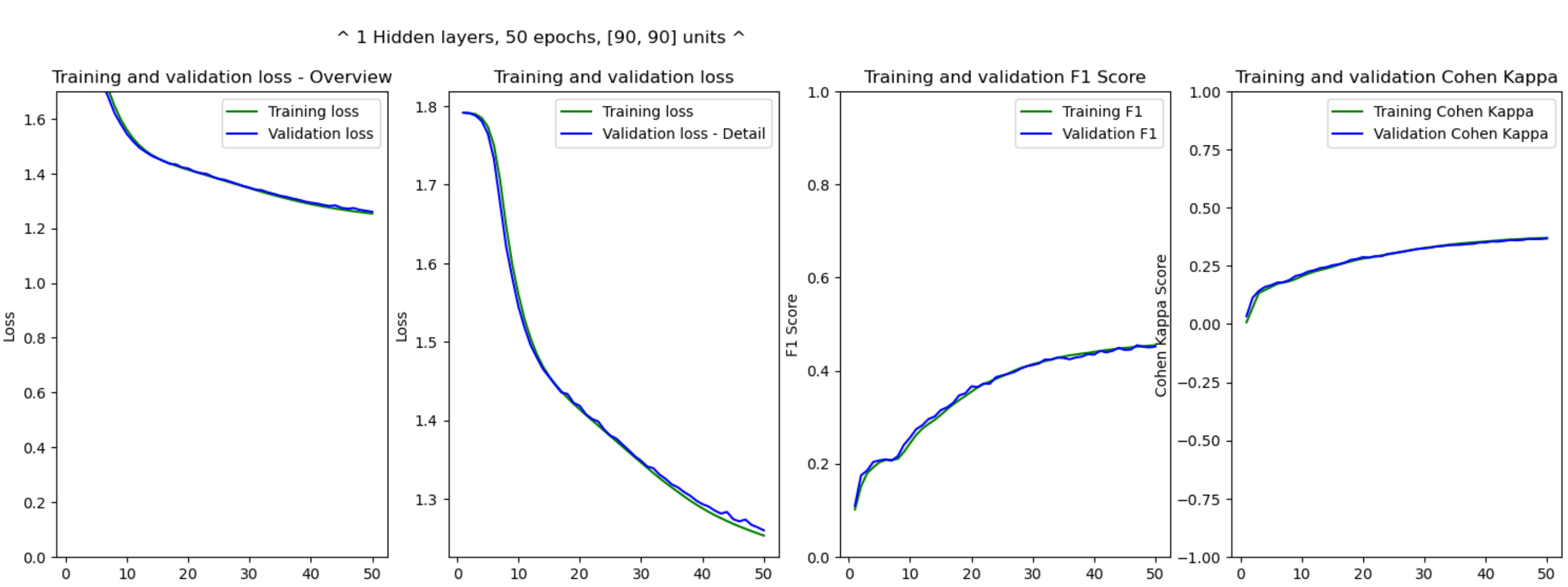
```
In [65]: num_hidden_layers = [2, 6, 10]

         overfitting_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 50,
                     "loss_function": 'categorical_crossentropy'
                 }
         for number in num_hidden_layers:
             layer_trial = [layers.Dense(stage_6_num_units, activation='relu')]*number
             #insert the input layer
             layer_trial.insert(0, layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)))
             #append the output layer
             layer_trial.append(layers.Dense(num_classes, activation='softmax'))
             model_2 = NeuralNet(layer_trial)
             model_2.set_hyperparameters(overfitting_hyperparameters)
             model_2.compile_model()
             model_2.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 1 Hidden layers, 50 epochs, [90, 90] units ^



^ 1 Hidden layers, 50 epochs, [90, 90] units ^

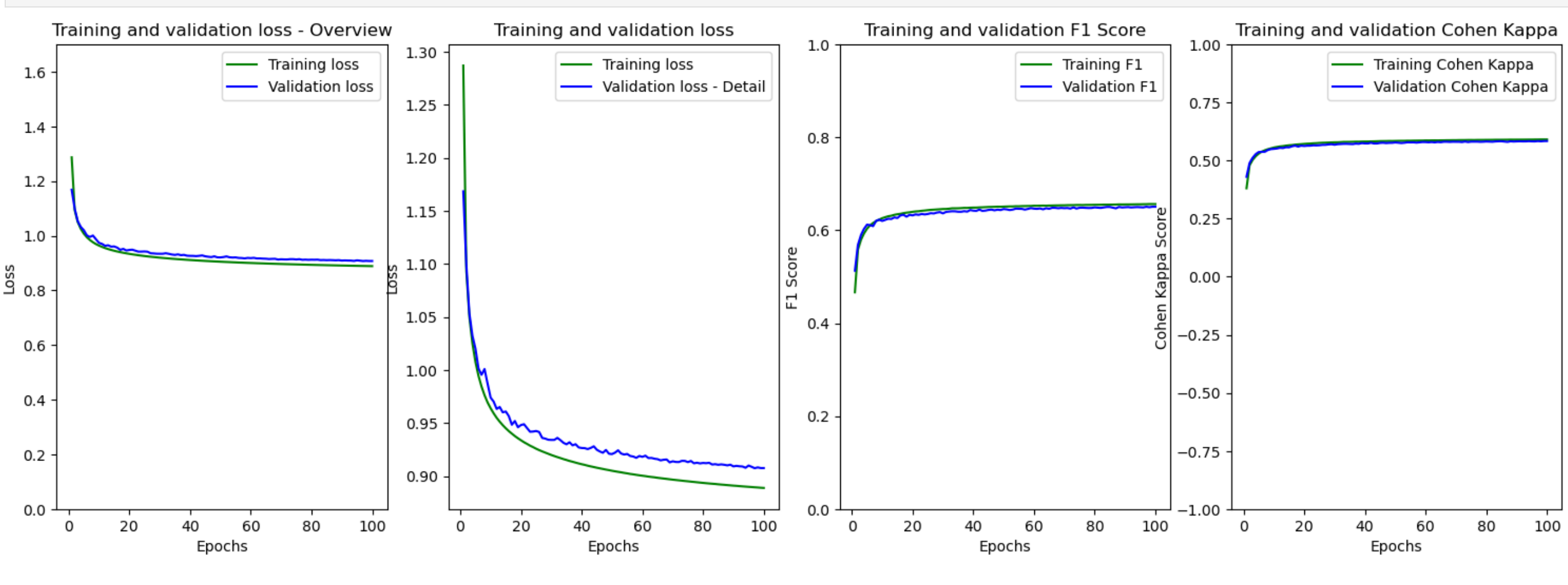


^ 1 Hidden layers, 50 epochs, [90, 90] units ^

Looking at the graphs above, 2 hidden layers resulted in the lowest training/validation loss and the highest evaluation metrics.

Element 3: Number of Epochs:

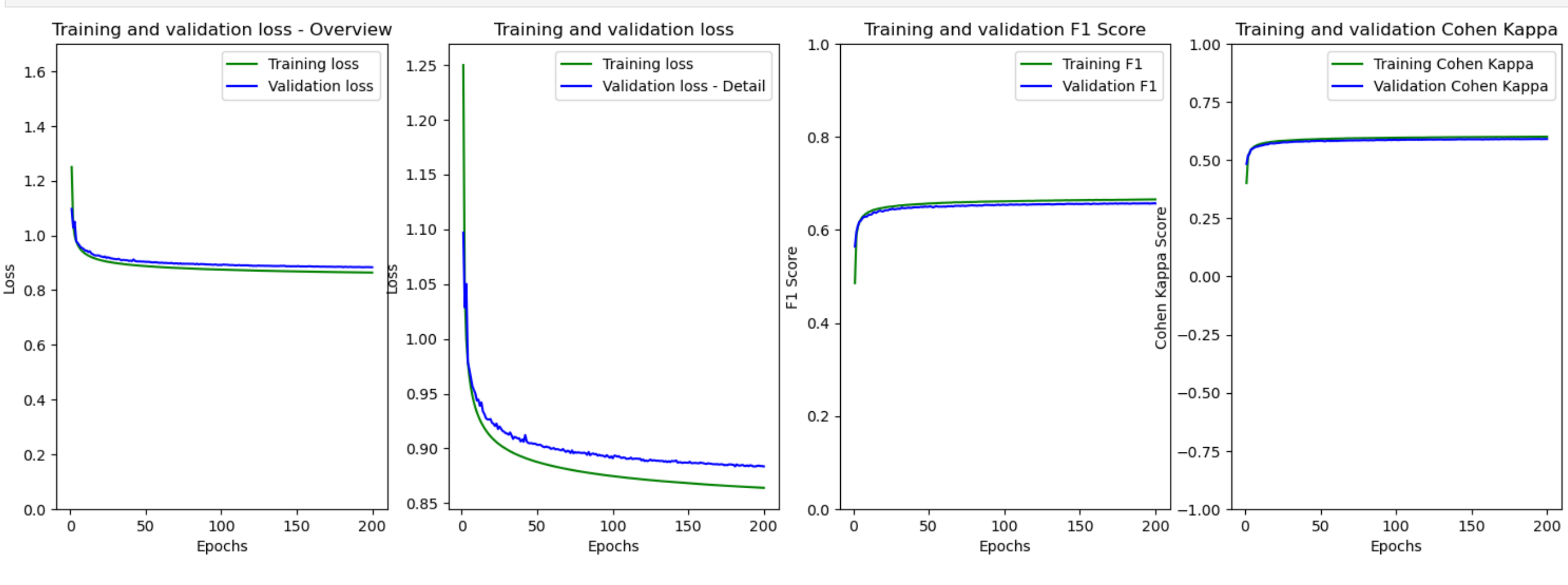
```
In [68]: overfitting_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 100,
                     "loss_function": 'categorical_crossentropy'
                 }


         overfitting_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu'),
                 layers.Dense(stage_6_num_units, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(overfitting_layers)
         model_2_hidden_layers.set_hyperparameters(overfitting_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 2 Hidden layers, 100 epochs, [90, 90, 90] units ^

Doubling the number of training epochs (50 to 100) has decreased the training loss by ~0.4. It has also increased the

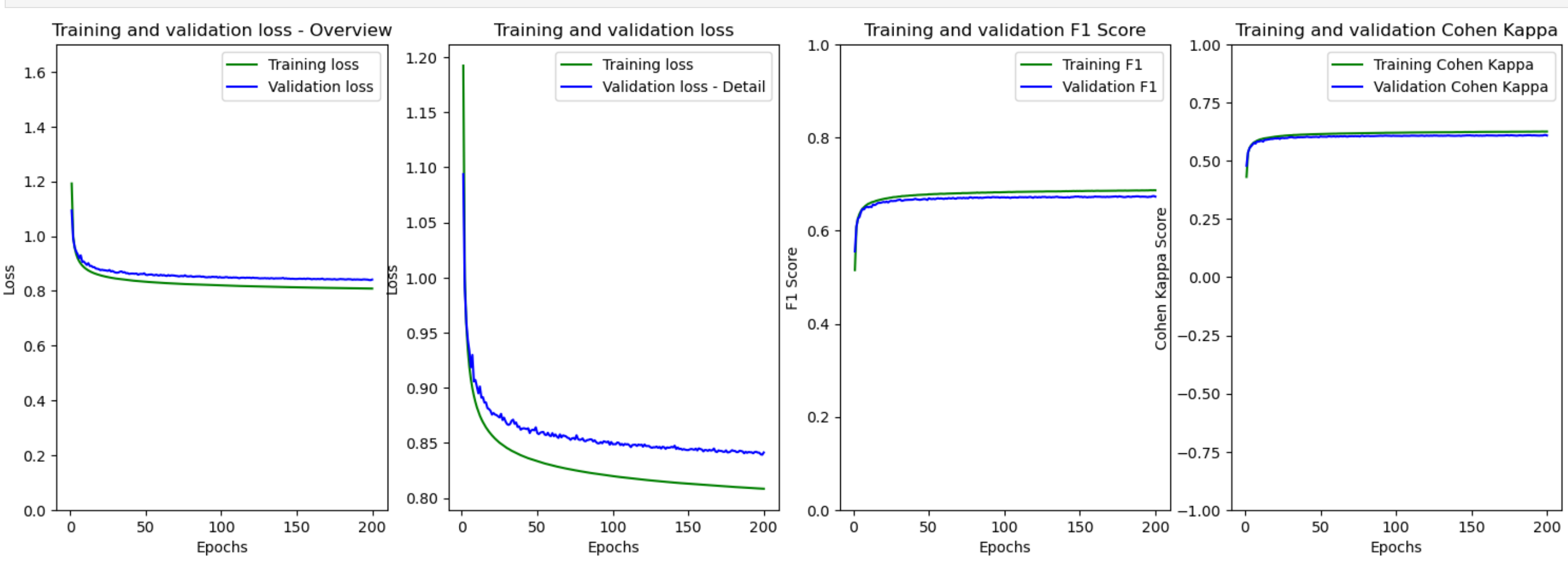
```
In [69]: overfitting_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 200,
                     "loss_function": 'categorical_crossentropy'
                 }


         overfitting_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu'),
                 layers.Dense(stage_6_num_units, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(overfitting_layers)
         model_2_hidden_layers.set_hyperparameters(overfitting_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 2 Hidden layers, 200 epochs, [90, 90, 90] units ^

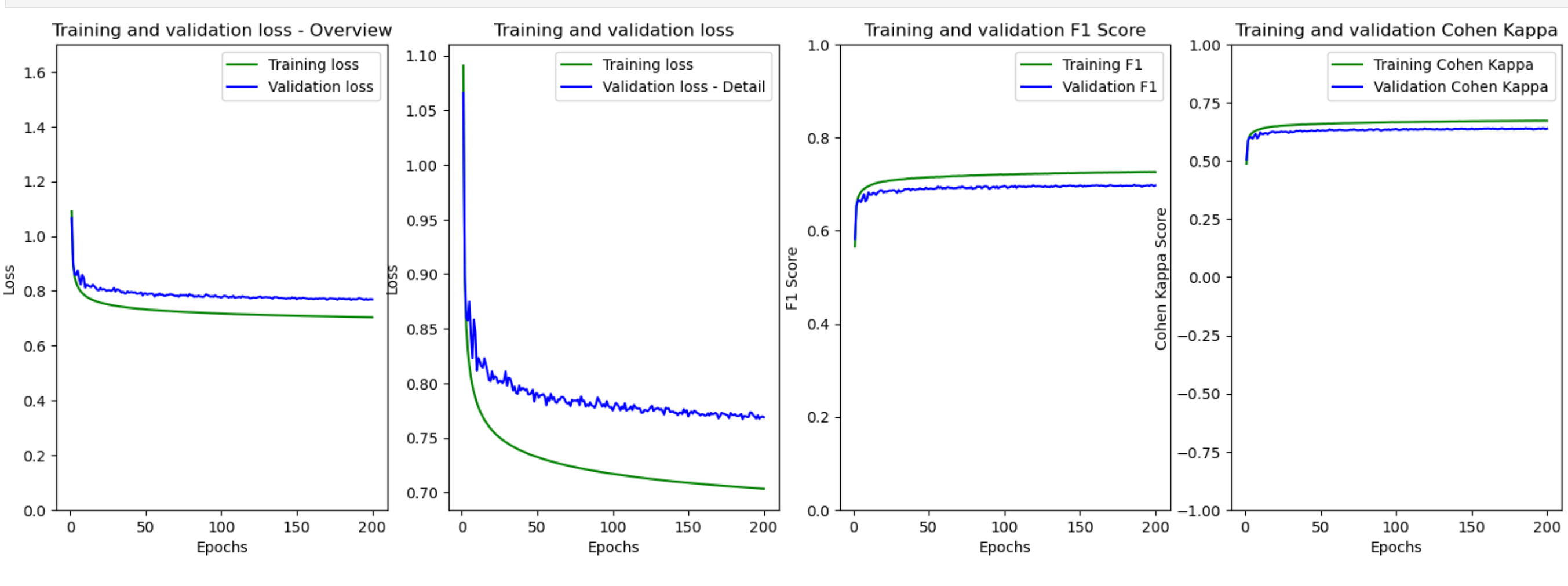
As the model does not seem to be overfitting (based on the graphs above): I will return to Element 1: Number of units:

```
In [72]: overfitting_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 200,
                     "loss_function": 'categorical_crossentropy'
                 }


         overfitting_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(200, activation='relu'),
                 layers.Dense(200, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(overfitting_layers)
         model_2_hidden_layers.set_hyperparameters(overfitting_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



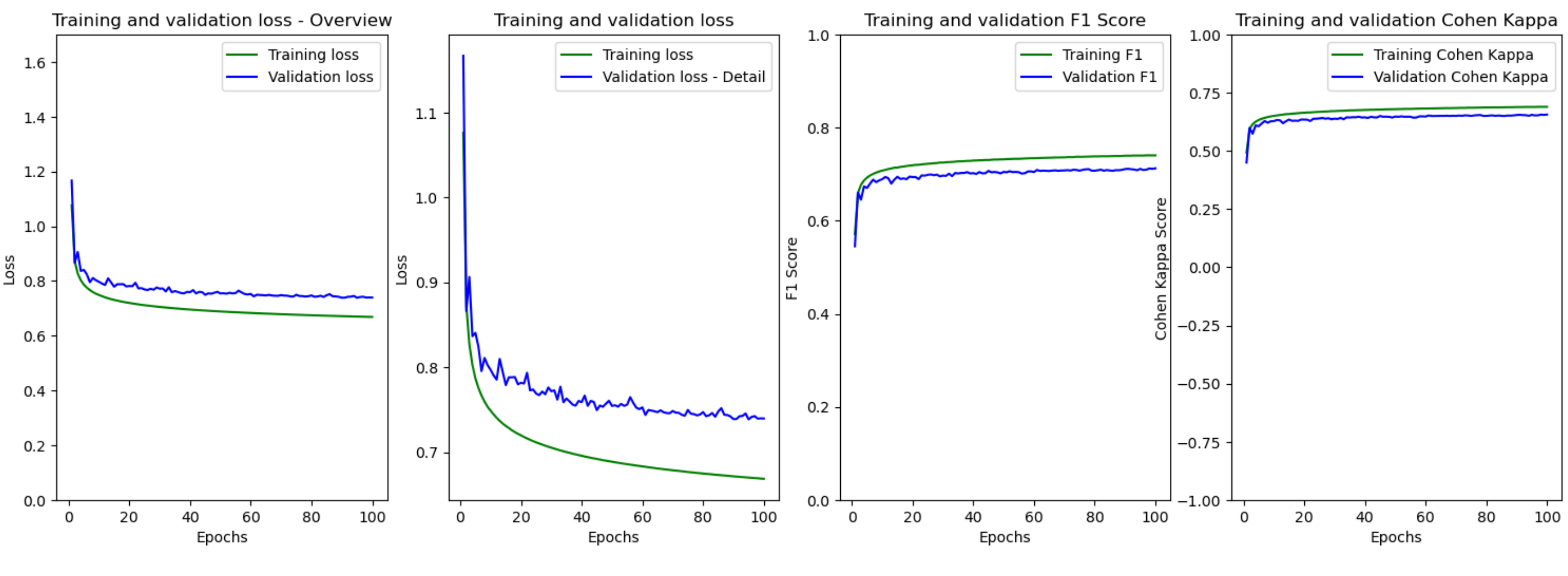
^ 2 Hidden layers, 200 epochs, [90, 200, 200] units ^

```
In [73]: overfitting_hyperparameters = {
                     #a tensorflow warning suggested usinaag the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 200,
                     "loss_function": 'categorical_crossentropy'
                 }


         overfitting_layers = [
                 layers.Dense(400, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(400, activation='relu'),
                 layers.Dense(400, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(overfitting_layers)
         model_2_hidden_layers.set_hyperparameters(overfitting_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 2 Hidden layers, 200 epochs, [400, 400, 400] units ^

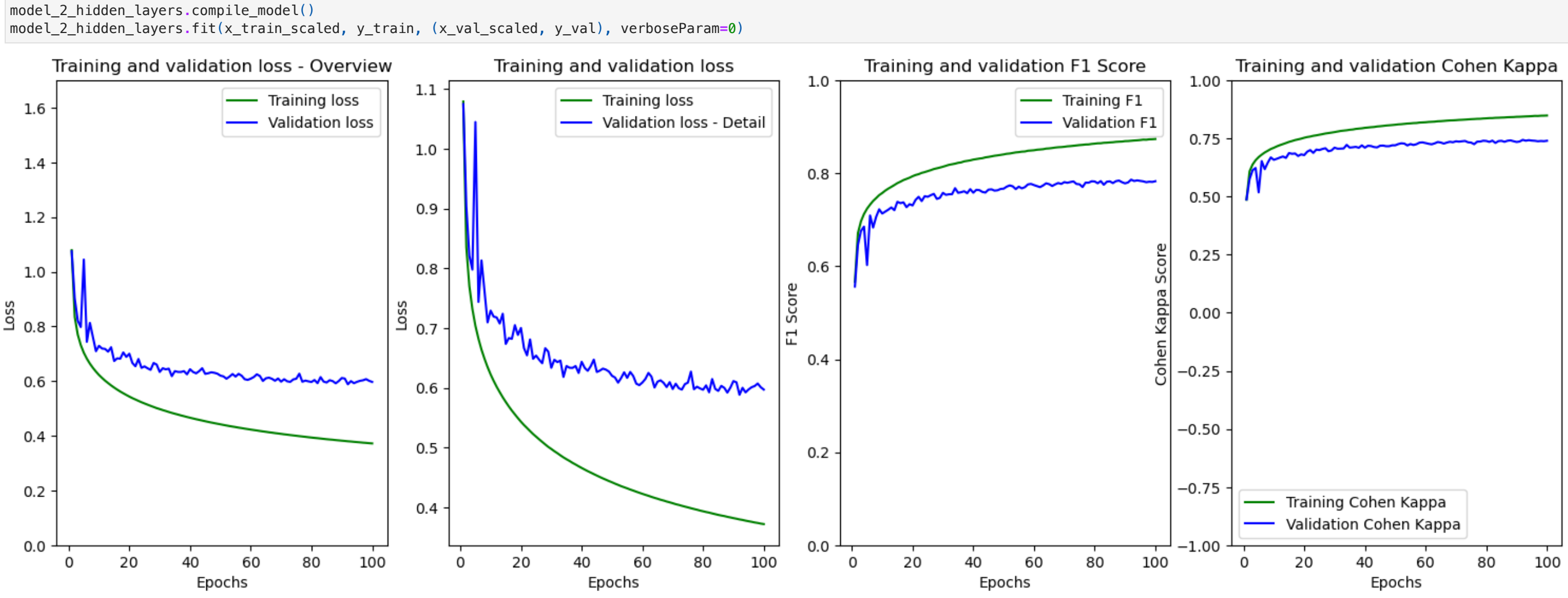
```
In [75]: overfitting_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 100,
                     "loss_function": 'categorical_crossentropy'
                 }


         overfitting_layers = [
                 layers.Dense(600, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(overfitting_layers)
         model_2_hidden_layers.set_hyperparameters(overfitting_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 2 Hidden layers, 100 epochs, [600, 600, 600] units ^

Increasing the number of units from 400 to 600 has resulted in a slight decrease in the training loss (looking at the y axis of the graphs in the second column in the previous 2 cells)

```
In [23]: overfitting_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 100,
                     "loss_function": 'categorical_crossentropy'
                 }


         overfitting_layers = [
                 layers.Dense(600, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(overfitting_layers)
         model_2_hidden_layers.set_hyperparameters(overfitting_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 4 Hidden layers, 100 epochs, [600, 600, 600, 600, 600] units ^

The combination of 600 units and 4 hidden layers has achieved the lowest training loss and highest evaluation metrics.

The model appears to be overfitting ; the training and validation loss have started to diverge. The training loss continues to decrease as the validation loss stagnates / begins to increase. This performance deterioration can also be seen in the evaluation metrics (2 graphs on the right).

The training/validation loss curve above appears to have similar characteristics to the following example of an overfitting learning curve:

```
In [36]: from IPython.display import Image
         Image(filename="./images/overfitting_example.png", width="500")
```

Out[36]:

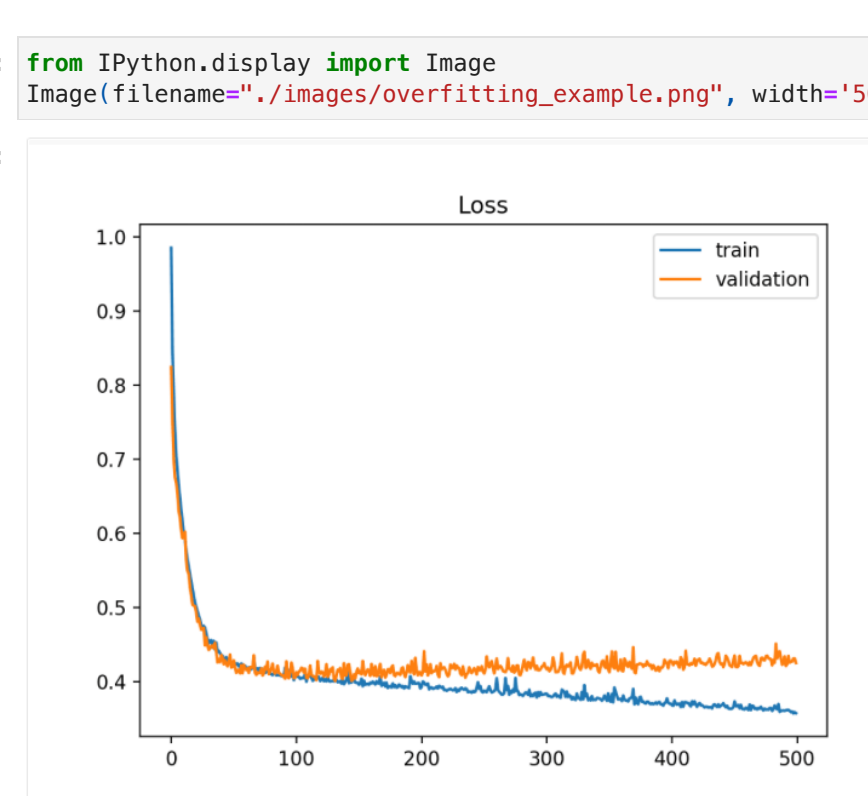


Image source: (Brownlee, 2019)

```
In [36]: stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 100,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(600, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         stage_6_num_units = 600
```

Section 7 Training Log: Model Regularization and hyperparameter tuning (based on validation data performance)

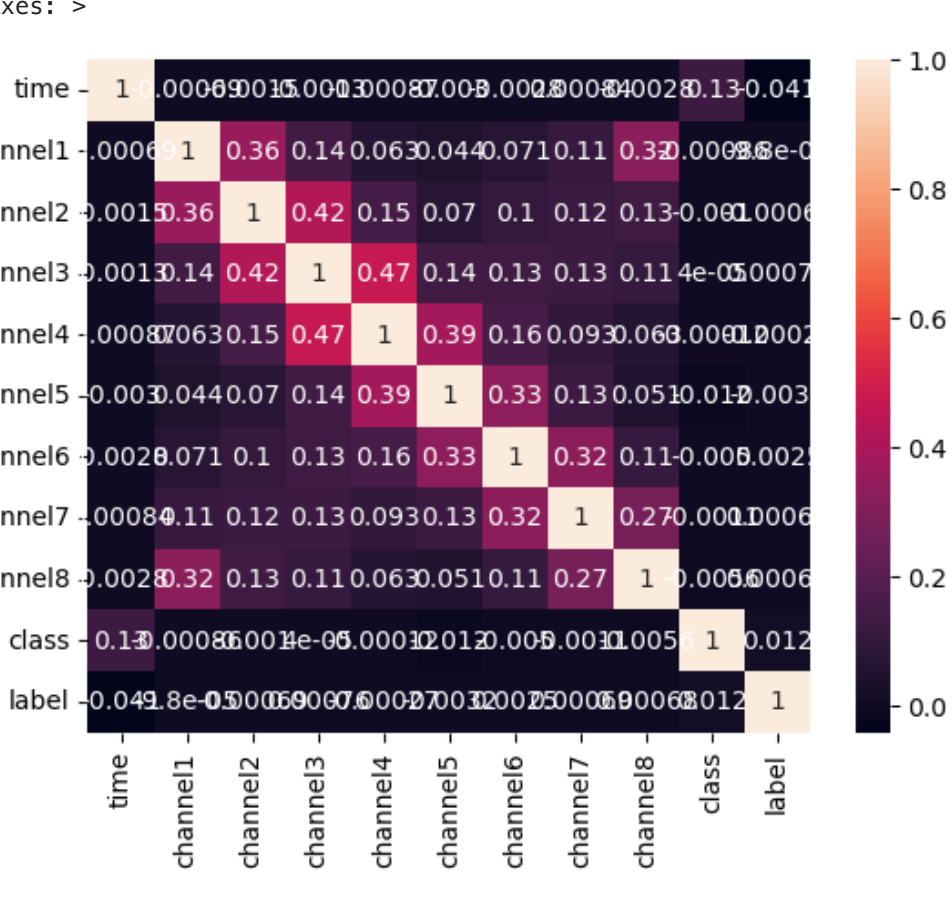
Model Regularization:

L1 , L2 Regularization:

It seems that Elastic Net Regularization (L1 and L2 regularization) is generally applied to models with high collinearity or when there are more features than observations. (Mayank, 2023) On examination this dataset does not fit either of these criteria ; there are 9 features and over 1 million observations. A collinearity plot is shown below:

```
In [38]: import seaborn as sns
         # calculate the correlations using pandas corr() - this method uses the Pearson correlation by default
         #Pearsons measures the strength of the linear relationship between 2 variables

         featureCorrelations = data.corr()
         featureNames = data.columns
         #plot the heatmap
         sns.heatmap(featureCorrelations, xticklabels=featureNames, yticklabels=featureNames, annot=True)
```

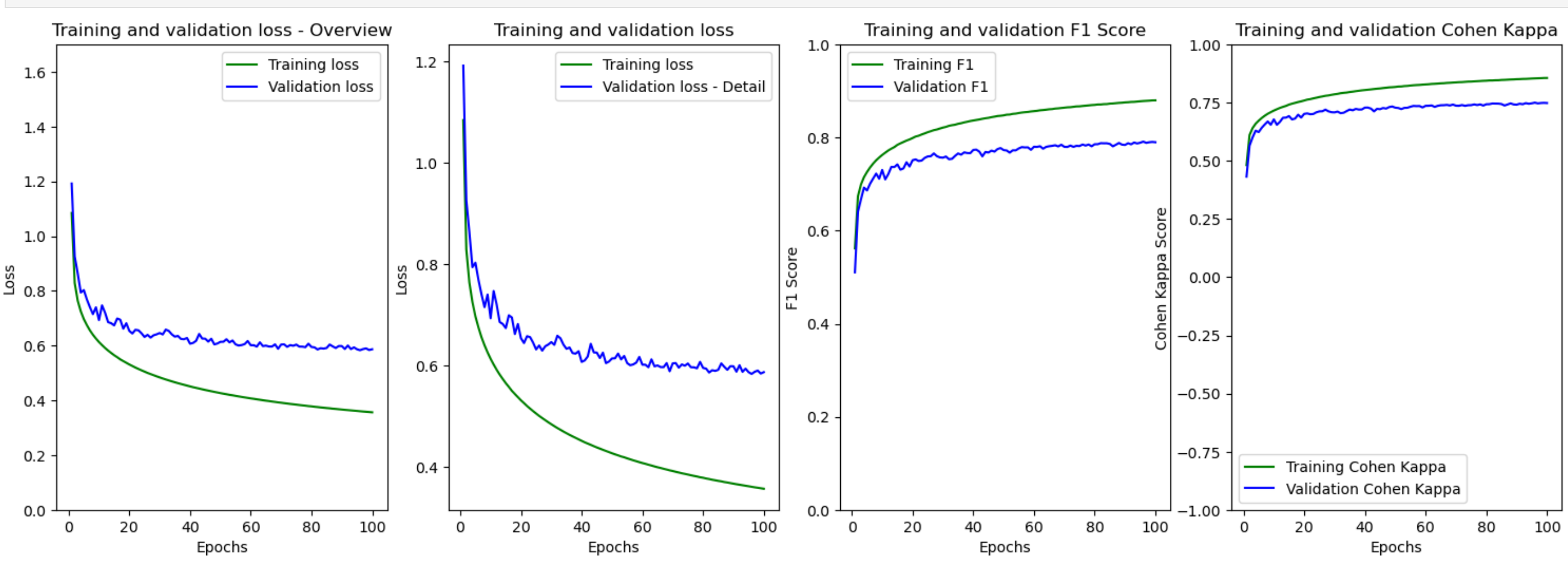
```
Out[38]: <Axes: >
```



L1/L2 ; weights are incorporated into the loss function, meaning that the model is penalised for large weights

L2 Regularization:

```
In [52]: from keras import regularizers
         stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 100,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L2(1e-4)),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L2(1e-4)),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L2(1e-4)),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L2(1e-4)),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(stage_6_layers)
         model_2_hidden_layers.set_hyperparameters(stage_6_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```

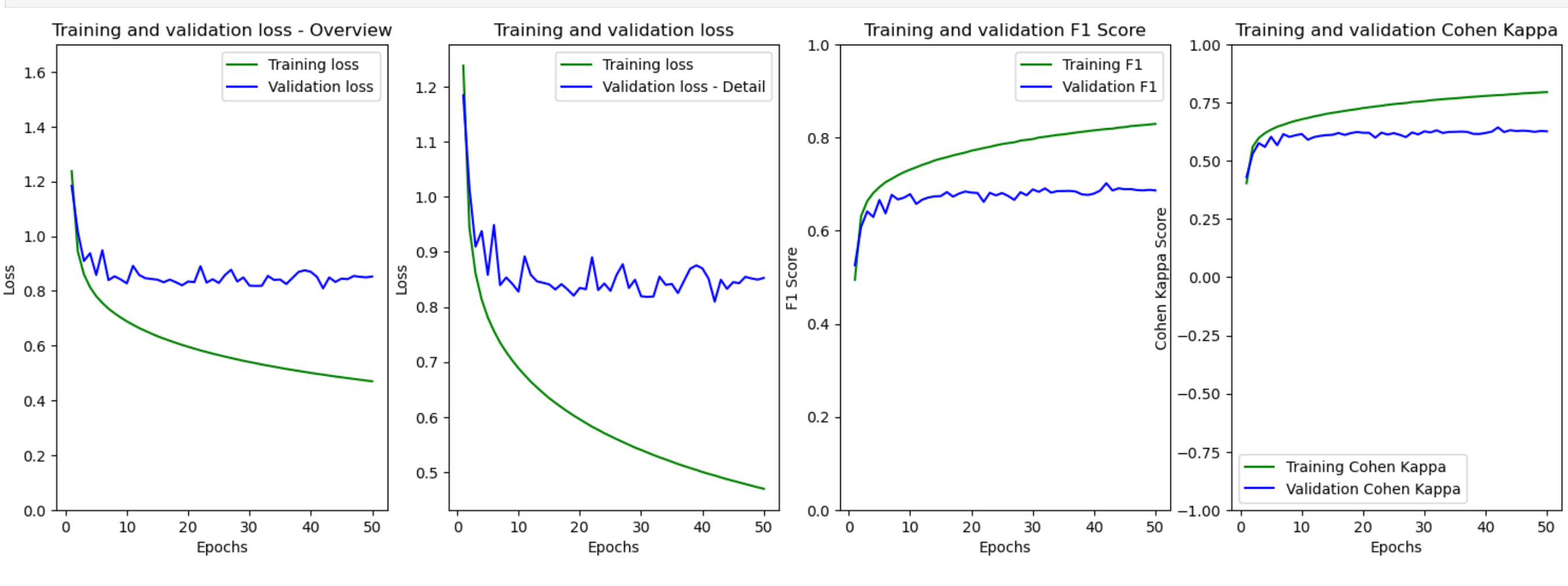


^ 4 Hidden layers, 100 epochs, [600, 600, 600, 600, 600] units ^

L2 Regularization does not appear to have significantly reduce overfitting (at an alpha value of 1e-4)

L1 Regularization:

```
In [42]: stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 50,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(600, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(600, activation='relu', bias_regularizer=regularizers.L1(1e-4)),
                 layers.Dense(600, activation='relu', bias_regularizer=regularizers.L1(1e-4)),
                 layers.Dense(600, activation='relu', bias_regularizer=regularizers.L1(1e-4)),
                 layers.Dense(600, activation='relu', bias_regularizer=regularizers.L1(1e-4)),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(stage_6_layers)
         model_2_hidden_layers.set_hyperparameters(stage_6_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```

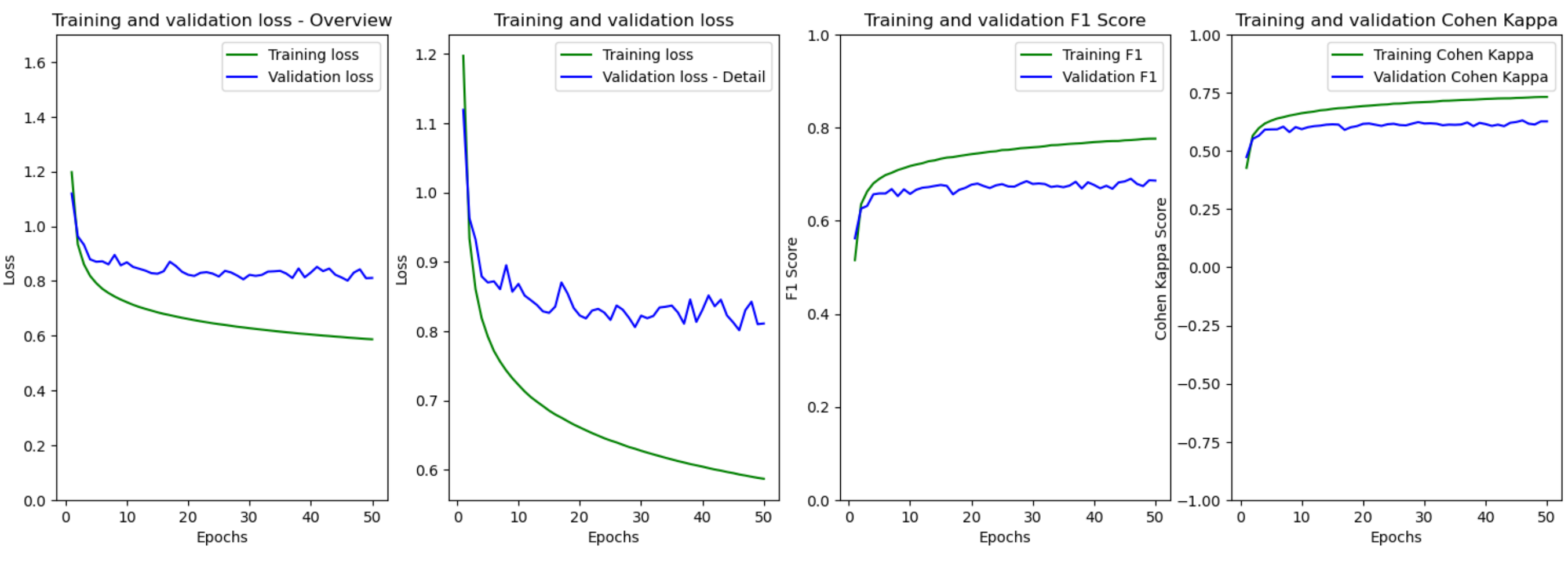


^ 4 Hidden layers, 50 epochs, [600, 600, 600, 600, 600] units ^

L1 regularization does not seem to have closed the gap between training and validation loss ; the model still appears to be overfitting.

Removal of Layers:

```
In [22]: stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 50,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(600, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(600, activation='relu'),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(stage_6_layers)
         model_2_hidden_layers.set_hyperparameters(stage_6_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



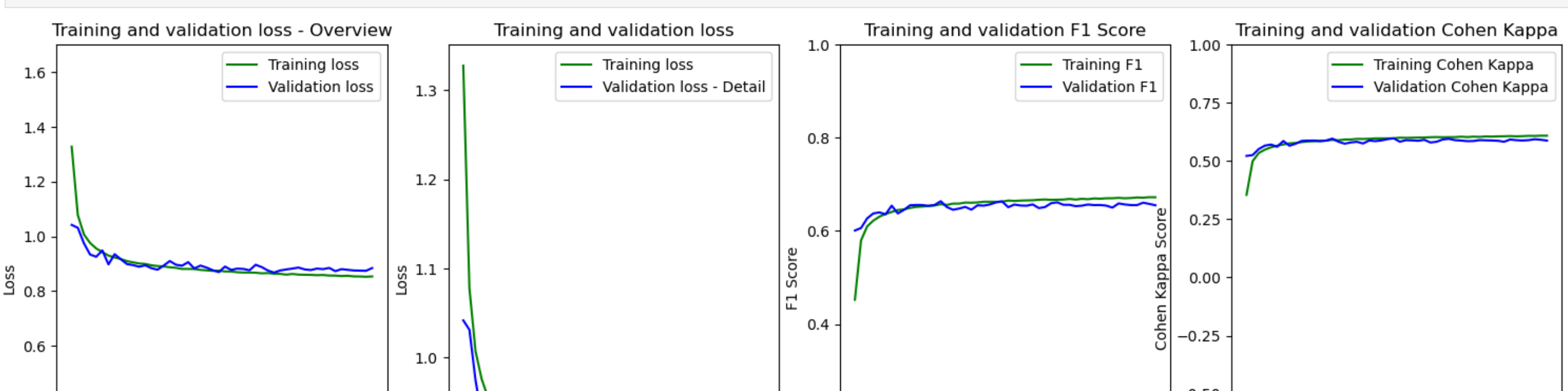
^ 3 Hidden layers, 50 epochs, [600, 600, 600, 600] units ^

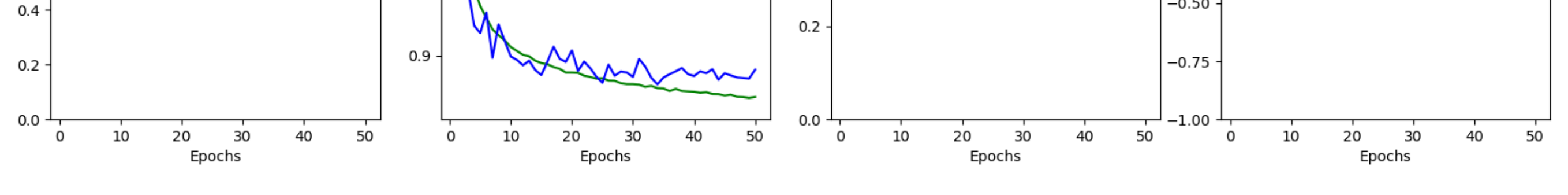
Looking at the graph above (second column), removing 1 Layer did not appear to make any significant reductions in overfitting.

Adding Dropout Layers:

I added drop out layers after every hidden layer:

```
In [64]: stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 50,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6)),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6)),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6)),
                 layers.Dropout(0.4),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6)),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_2_hidden_layers = NeuralNet(stage_6_layers)
         model_2_hidden_layers.set_hyperparameters(stage_6_hyperparameters)
         model_2_hidden_layers.compile_model()
         model_2_hidden_layers.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```





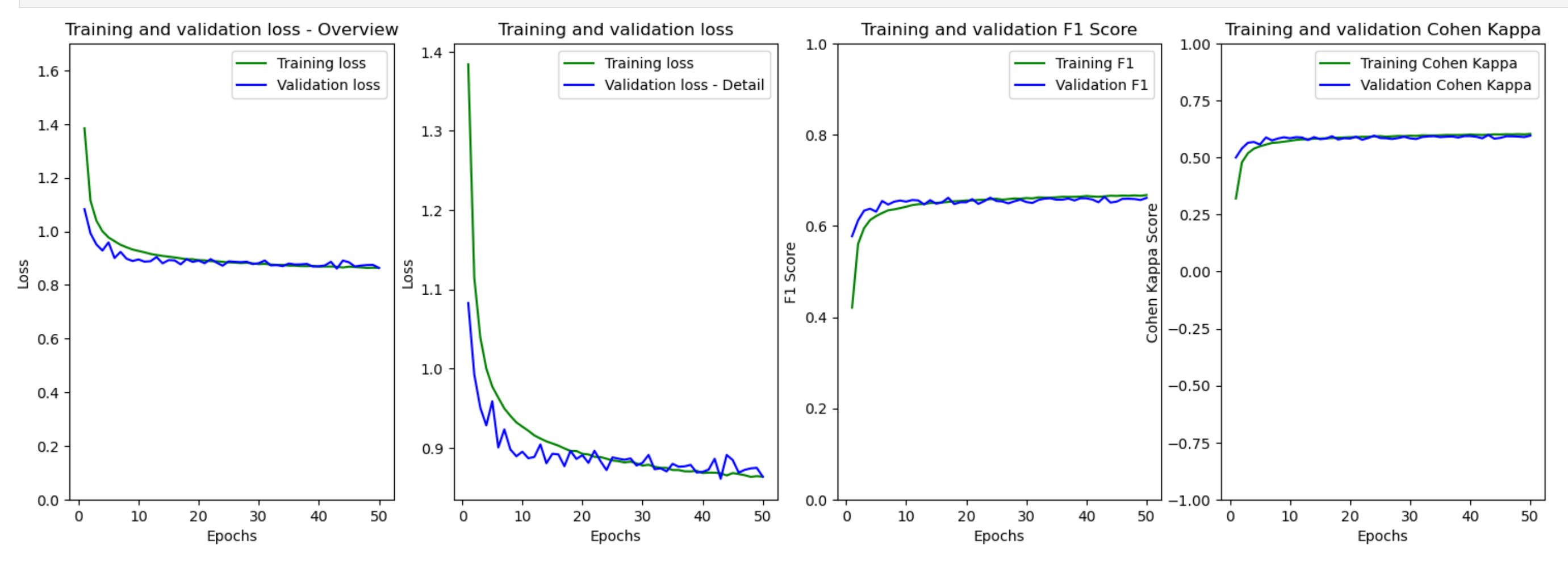
^ 8 Hidden layers, 50 epochs, [600, 600, 600, 600, 600] units ^

The training/validation loss curve above shows a considerable improvement ; there is no longer a sizeable gap between training and validation loss.

Weight initialisation:

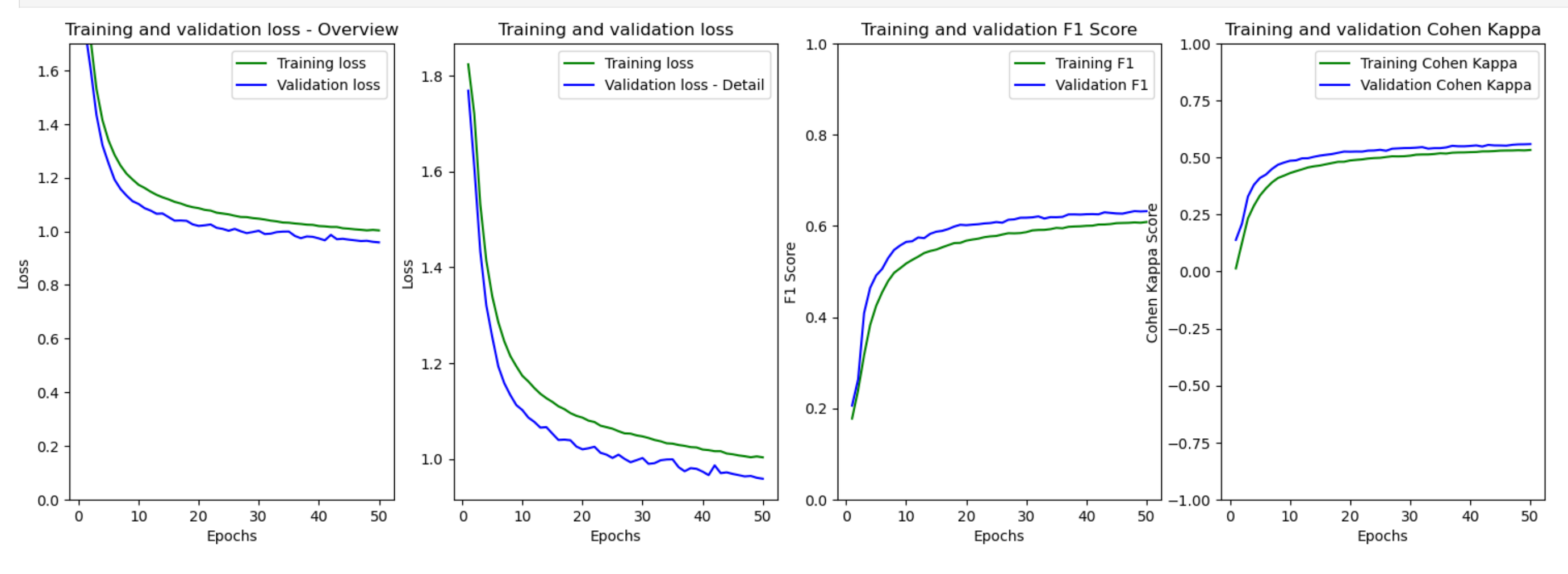
He initialization is commonly used with relu activation functions (Katanforoosh, 2019): I conducted the following experiments with He initialization:
Keras offers 2 forms of He initialization: HeNormal and HeUniform

```
In [69]: from keras import initializers
         stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 50,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_dropout = NeuralNet(stage_6_layers)
         model_dropout.set_hyperparameters(stage_6_hyperparameters)
         model_dropout.compile_model()
         model_dropout.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 8 Hidden layers, 50 epochs, [600, 600, 600, 600, 600] units ^

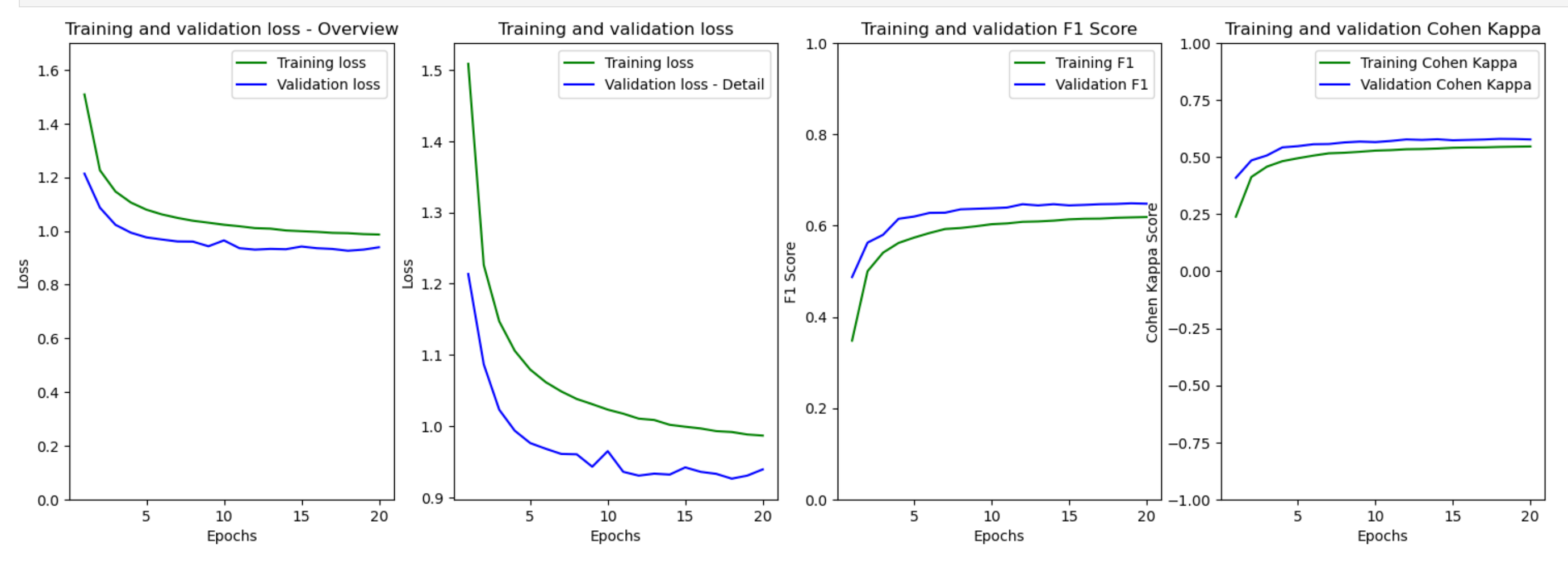
```
In [70]: stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.4),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_dropout = NeuralNet(stage_6_layers)
         model_dropout.set_hyperparameters(stage_6_hyperparameters)
         model_dropout.compile_model()
         model_dropout.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 8 Hidden layers, 50 epochs, [600, 600, 600, 600, 600] units ^

Comparing the 2 rows of graphs above ; the model initialized using HeNormal acheived a lower training and validation loss. I will use HeNormal initialization in the next set of experiments.

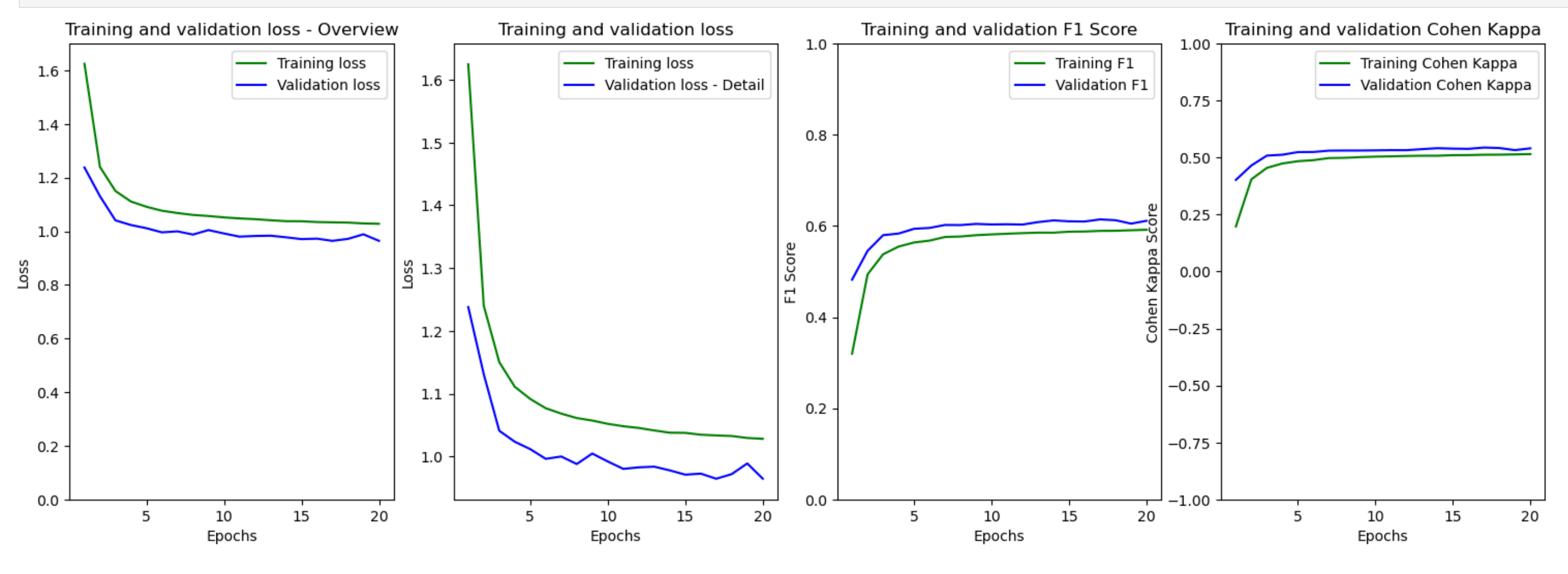
```
In [71]: stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 20,
                     "loss_function": 'categorical_crossentropy'
                 }

         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.8),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeUniform()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_dropout = NeuralNet(stage_6_layers)
         model_dropout.set_hyperparameters(stage_6_hyperparameters)
         model_dropout.compile_model()
         model_dropout.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 8 Hidden layers, 20 epochs, [600, 600, 600, 600, 600] units ^

Change activation function to Exponential Linear Units (ELU):

```
In [78]: from keras import initializers
         stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 20,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='elu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='elu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.8),
                 layers.Dense(stage_6_num_units, activation='elu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='elu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='elu', bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_dropout = NeuralNet(stage_6_layers)
         model_dropout.set_hyperparameters(stage_6_hyperparameters)
         model_dropout.compile_model()
         model_dropout.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```

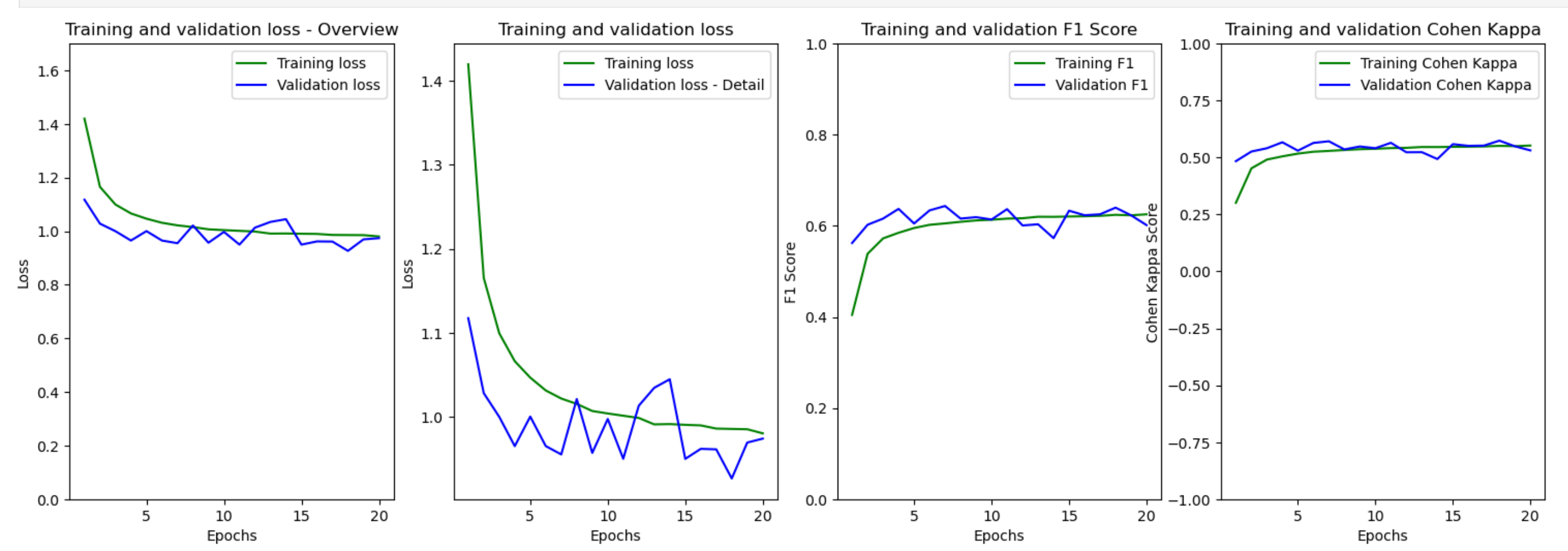


^ 8 Hidden layers, 20 epochs, [600, 600, 600, 600, 600] units ^

The loss appears to have increased from 0.9-1.0 after change 'relu' to 'elu'.

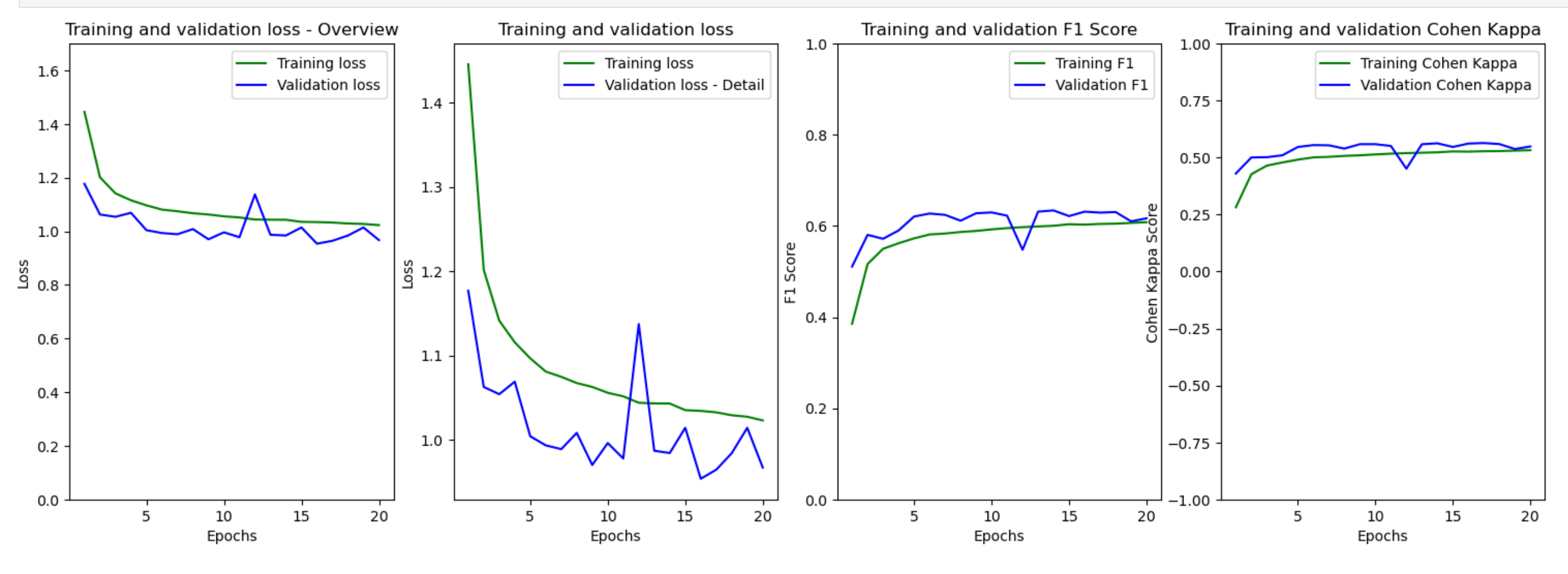
Change optimization function to ADAM variants:

```
In [81]: from keras import initializers
         stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.Nadam(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 20,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_nadam = NeuralNet(stage_6_layers)
         model_nadam.set_hyperparameters(stage_6_hyperparameters)
         model_nadam.compile_model()
         model_nadam.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 8 Hidden layers, 20 epochs, [600, 600, 600, 600, 600] units ^

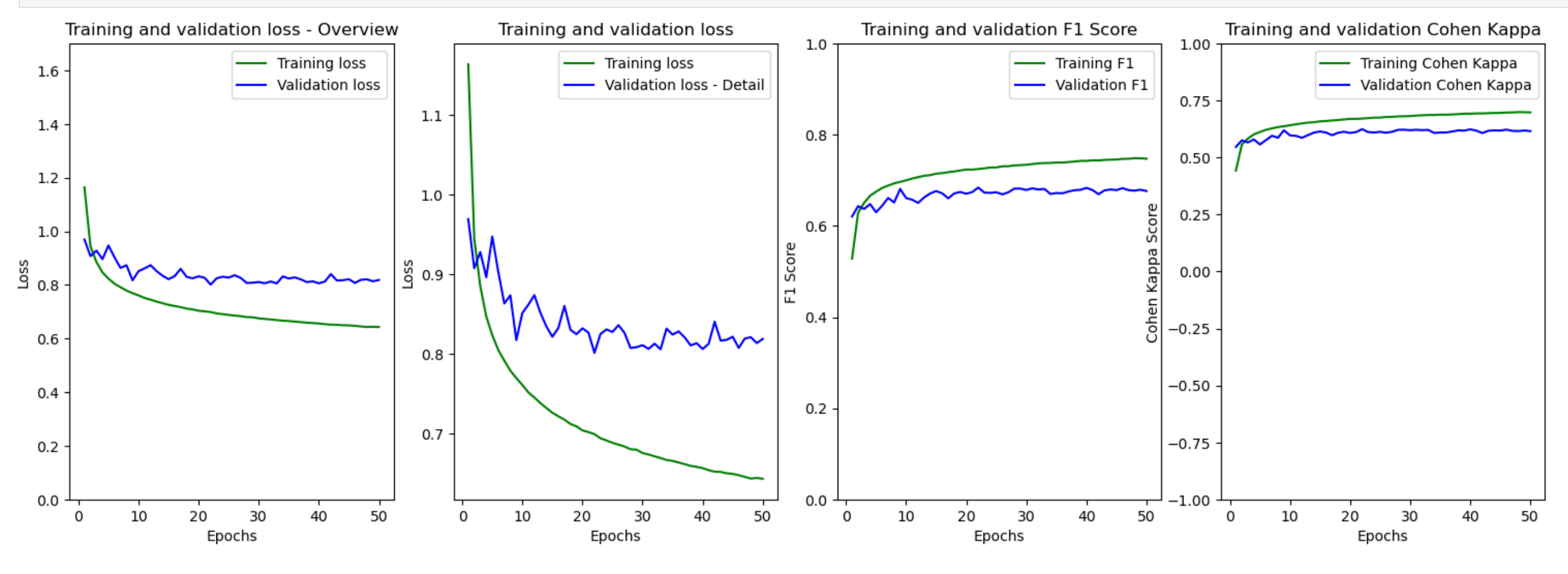
```
In [85]: from keras import initializers
         stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.Adam(decay=1e-6), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 20,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_nadam = NeuralNet(stage_6_layers)
         model_nadam.set_hyperparameters(stage_6_hyperparameters)
         model_nadam.compile_model()
         model_nadam.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 8 Hidden layers, 20 epochs, [600, 600, 600, 600, 600] units ^

Changing the optimizer to Adam and Nadam has not furhter decreased training/validation loss.

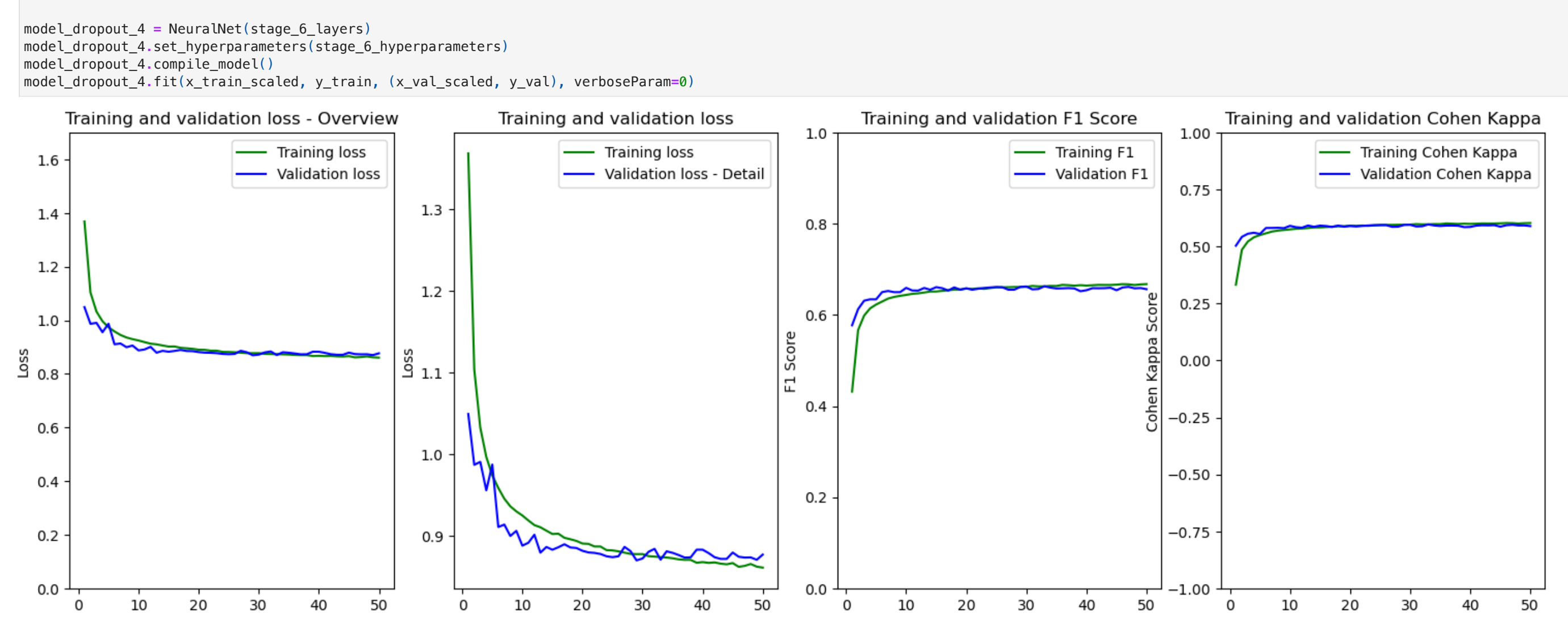
```
In [86]: from keras import initializers
         stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.Adam(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 50,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.2),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.2),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_nadam = NeuralNet(stage_6_layers)
         model_nadam.set_hyperparameters(stage_6_hyperparameters)
         model_nadam.compile_model()
         model_nadam.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 6 Hidden layers, 50 epochs, [600, 600, 600, 600, 600] units ^

Best model so far:

```
In [38]: from keras import initializers, regularizers
         stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3), #tweaked the rate of decay to best suit this trial
                     "learning_rate": None,
                     "batch_size": 128,
                     "epochs_number": 50,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         model_dropout_4 = NeuralNet(stage_6_layers)
         model_dropout_4.set_hyperparameters(stage_6_hyperparameters)
         model_dropout_4.compile_model()
         model_dropout_4.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```



^ 8 Hidden layers, 50 epochs, [600, 600, 600, 600, 600] units ^

There appears to be little improvement in the training/validation loss and evaluation metrics after 30 epochs - this seems to be the ideal number of epochs for this set of hyperparameters.

Reflecting on the previous iterations of hyperparamter training - it seemed that the training/validation loss, F1 score and Cohen Kappa were relatively stagnant at ~0.9, ~0.6 and ~0.5. I turned my research to identify more effective methods of searching the hyperparameter space

```
In [39]: from IPython.display import Image
         Image(filename="./images/saddle_local_optima.png", width='400')
```

Out[39]:

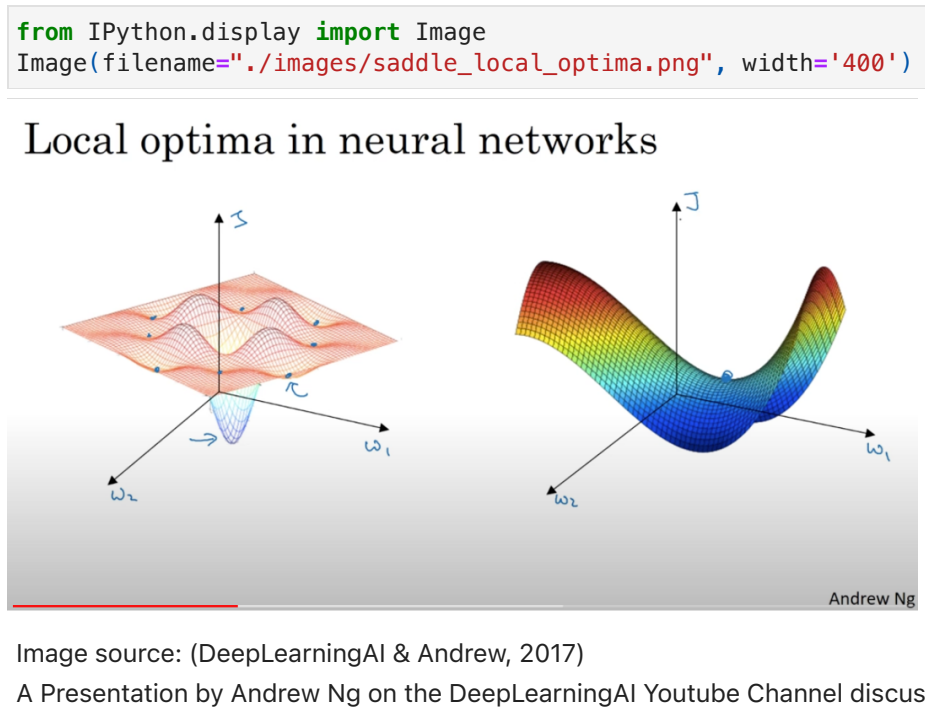


Image source: (DeepLearningAI & Andrew, 2017)
A Presentation by Andrew Ng on the DeepLearningAI Youtube Channel discussed that most points of zero gradient are not local optima (as pictured on the left) but are 'saddle' points (pictured on the right) Andrew Ng presents that plateaus are regions in which the derivative is close to 0 for a long time - this can slow down learning;

```
In [41]: from IPython.display import Image
         Image(filename="./images/Plateaus.png", width='400')
```

Out[41]:

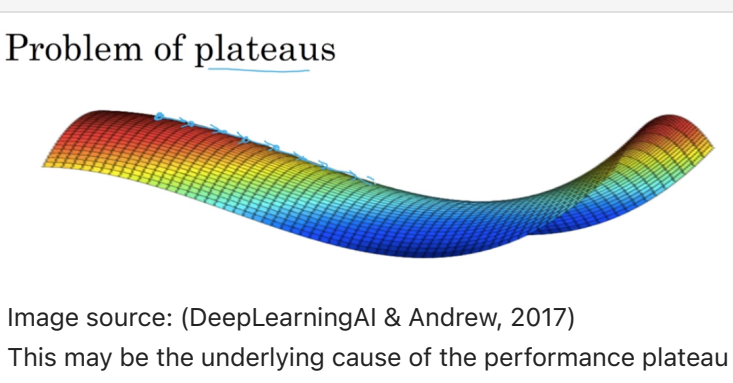


Image source: (DeepLearningAI & Andrew, 2017)
This may be the underlying cause of the performance plateau at F1 ~0.6 and Cohen Kappa ~0.5

Keras Tuner:

The keras_tuner module offers the facility to tune hyperparameters using Bayesian search ; Keras also offers random search, however Bayesian search uses the results of the previous hyperparameter sample to inform the next sampling. This suggests that Bayesian search may be offer more insightful results in less computation time.

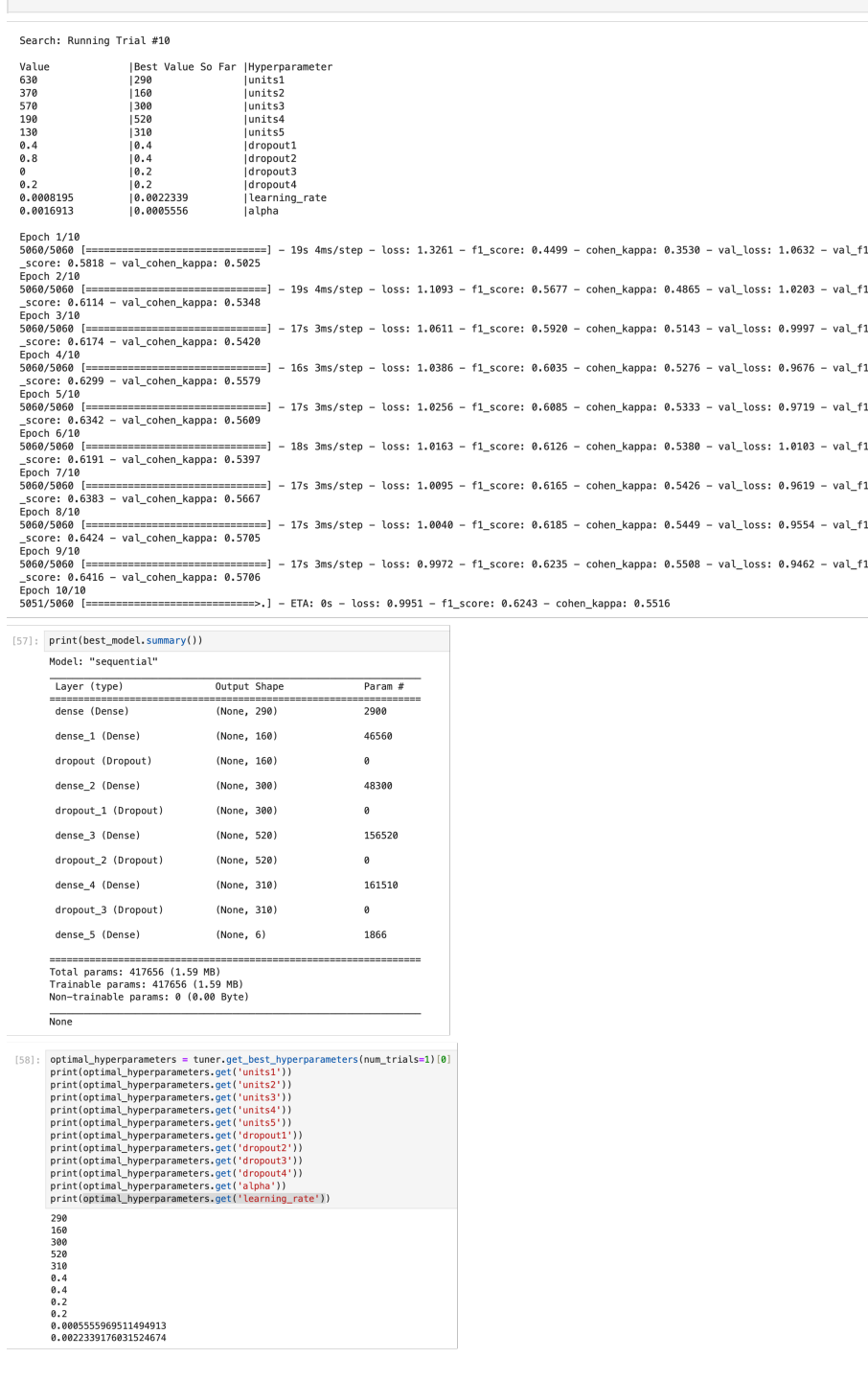
```
In [ ]: from keras import regularizers, initializers
        import tensorflow as tf

        def build_model(hp):
            neurons_1 = hp.Int("units1", min_value=30, max_value=650, step=10)
            neurons_2 = hp.Int("units2", min_value=30, max_value=650, step=10)
            neurons_3 = hp.Int("units3", min_value=30, max_value=650, step=10)
            neurons_4 = hp.Int("units4", min_value=30, max_value=650, step=10)
            neurons_5 = hp.Int("units5", min_value=100, max_value=650, step=10)
            dropout_1 = hp.Float("dropout1", min_value=0, max_value=1, step=0.2)
            dropout_2 = hp.Float("dropout2", min_value=0, max_value=1, step=0.2)
            dropout_3 = hp.Float("dropout3", min_value=0, max_value=1, step=0.2)
            dropout_4 = hp.Float("dropout4", min_value=0, max_value=1, step=0.2)
            lr = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2, sampling="log")
            alpha = hp.Float("alpha", min_value=1e-4, max_value=1e-2, sampling="log")

            stage_6_hyperparameters = {
                    #a tensorflow warning suggested using the legacy version of M1/M2 chips
                    "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3, learning_rate=lr), #tweaked the rate of decay to best suit this trial
                    "learning_rate": None,
                    "batch_size": 128,
                    "epochs_number": 10,
                    "loss_function": 'categorical_crossentropy'
                }


            stage_6_layers = [
                layers.Dense(neurons_1, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                layers.Dense(neurons_2, activation='relu', bias_regularizer=regularizers.L1(alpha),
                             kernel_initializer=initializers.HeNormal()),
                layers.Dropout(dropout_1),
                layers.Dense(neurons_3, activation='relu', bias_regularizer=regularizers.L1(alpha),
                             kernel_initializer=initializers.HeNormal()),
                layers.Dropout(dropout_2),
                layers.Dense(neurons_4, bias_regularizer=regularizers.L1(alpha),
                             kernel_initializer=initializers.HeNormal()),
                layers.Dropout(dropout_3),
                layers.Dense(neurons_5, bias_regularizer=regularizers.L1(alpha),
                            kernel_initializer=initializers.HeNormal()),
                layers.Dropout(dropout_4),
                layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                ]

            model = NeuralNet(stage_6_layers)
            model.set_hyperparameters(stage_6_hyperparameters)
            model.compile_model()
            #the use of .model is due to the use of custom class NeuralNet
            return model.model
```

```
In [ ]: import keras_tuner as kt
        tuner = kt.BayesianOptimization(build_model, objective='val_loss', max_trials=10, overwrite=True)
```

```
In [ ]: tuner.search(x_train_scaled, y_train, epochs=10, validation_data=(x_val_scaled, y_val), batch_size=32)
        best_models = tuner.get_best_models()
        best_model = best_models[0]
```

```
In [87]: from IPython.display import Image
         Image(filename="./images/keras_tuner_2.png", width='500')
```

Out[87]:



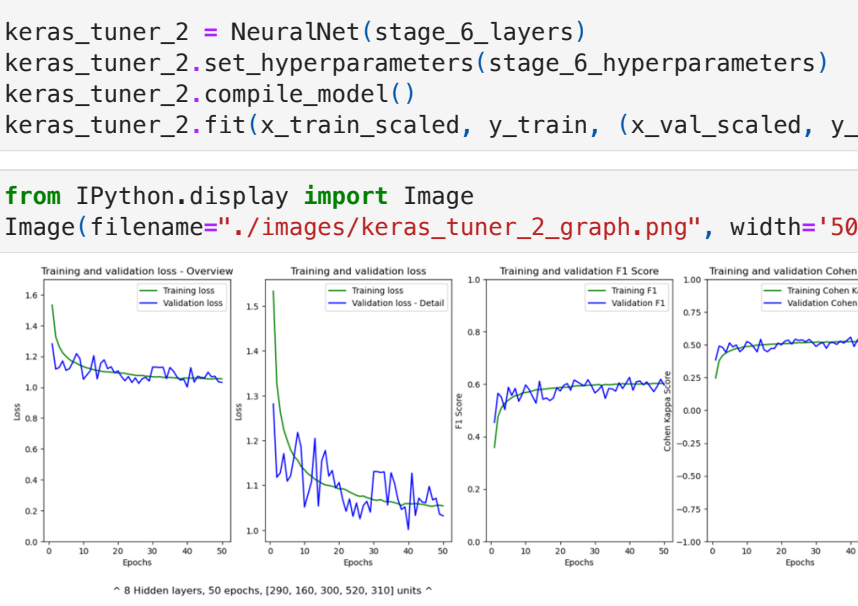
```
Search: Running Trial #10

Value             |Best Value So Far |Hyperparameter
630               |290               |units1
370               |160               |units2
570               |300               |units3
190               |520               |units4
130               |310               |units5
0.4               |0.4               |dropout1
0.8               |0.4               |dropout2
0                 |0.2               |dropout3
0.2               |0.2               |dropout4
0.0008195         |0.0022339         |learning_rate
0.0016913         |0.0005556         |alpha

Epoch 1/10
5060/5060 [==============================] - 19s 4ms/step - loss: 1.3261 - f1_score: 0.4499 - cohen_kappa: 0.3530 - val_loss: 1.0632 - val_f1
_score: 0.5818 - val_cohen_kappa: 0.5025
Epoch 2/10
5060/5060 [==============================] - 19s 4ms/step - loss: 1.1093 - f1_score: 0.5677 - cohen_kappa: 0.4865 - val_loss: 1.0203 - val_f1
_score: 0.6114 - val_cohen_kappa: 0.5348
Epoch 3/10
5060/5060 [==============================] - 17s 3ms/step - loss: 1.0611 - f1_score: 0.5920 - cohen_kappa: 0.5143 - val_loss: 0.9997 - val_f1
_score: 0.6174 - val_cohen_kappa: 0.5420
Epoch 4/10
5060/5060 [==============================] - 16s 3ms/step - loss: 1.0386 - f1_score: 0.6035 - cohen_kappa: 0.5276 - val_loss: 0.9676 - val_f1
_score: 0.6299 - val_cohen_kappa: 0.5579
Epoch 5/10
5060/5060 [==============================] - 17s 3ms/step - loss: 1.0256 - f1_score: 0.6085 - cohen_kappa: 0.5333 - val_loss: 0.9719 - val_f1
_score: 0.6342 - val_cohen_kappa: 0.5609
Epoch 6/10
5060/5060 [==============================] - 18s 3ms/step - loss: 1.0163 - f1_score: 0.6126 - cohen_kappa: 0.5380 - val_loss: 1.0103 - val_f1
_score: 0.6191 - val_cohen_kappa: 0.5397
Epoch 7/10
5060/5060 [==============================] - 17s 3ms/step - loss: 1.0095 - f1_score: 0.6165 - cohen_kappa: 0.5426 - val_loss: 0.9619 - val_f1
_score: 0.6383 - val_cohen_kappa: 0.5667
Epoch 8/10
5060/5060 [==============================] - 17s 3ms/step - loss: 1.0040 - f1_score: 0.6185 - cohen_kappa: 0.5449 - val_loss: 0.9554 - val_f1
_score: 0.6424 - val_cohen_kappa: 0.5705
Epoch 9/10
5060/5060 [==============================] - 17s 3ms/step - loss: 0.9972 - f1_score: 0.6235 - cohen_kappa: 0.5508 - val_loss: 0.9462 - val_f1
_score: 0.6416 - val_cohen_kappa: 0.5706
Epoch 10/10
5051/5060 [============================>.] - ETA: 0s - loss: 0.9951 - f1_score: 0.6243 - cohen_kappa: 0.5516

[57]: print(best_model.summary())
      Model: "sequential"
      _________________________________________________________________
       Layer (type)                Output Shape              Param #
      =================================================================
       dense (Dense)               (None, 290)               2900

       dense_1 (Dense)             (None, 160)               46560

       dropout (Dropout)           (None, 160)               0

       dense_2 (Dense)             (None, 300)               48300

       dropout_1 (Dropout)         (None, 300)               0

       dense_3 (Dense)             (None, 520)               156520

       dropout_2 (Dropout)         (None, 520)               0

       dense_4 (Dense)             (None, 310)               161510

       dropout_3 (Dropout)         (None, 310)               0

       dense_5 (Dense)             (None, 6)                 1866

      =================================================================
      Total params: 417656 (1.59 MB)
      Trainable params: 417656 (1.59 MB)
      Non-trainable params: 0 (0.00 Byte)
      _________________________________________________________________
      None

[58]: optimal_hyperparameters = tuner.get_best_hyperparameters(num_trials=1)[0]
      print(optimal_hyperparameters.get('units1'))
      print(optimal_hyperparameters.get('units2'))
      print(optimal_hyperparameters.get('units3'))
      print(optimal_hyperparameters.get('units4'))
      print(optimal_hyperparameters.get('units5'))
      print(optimal_hyperparameters.get('dropout1'))
      print(optimal_hyperparameters.get('dropout2'))
      print(optimal_hyperparameters.get('dropout3'))
      print(optimal_hyperparameters.get('dropout4'))
      print(optimal_hyperparameters.get('alpha'))
      print(optimal_hyperparameters.get('learning_rate'))

      290
      160
      300
      520
      310
      0.4
      0.4
      0.2
      0.2
      0.0005555969511494913
      0.0022339176031524674
```

I then trained this model for 50 epochs in order to get a better overivew of it's capabilities:

```
In [ ]: from keras import initializers
        stage_6_hyperparameters = {
                    #a tensorflow warning suggested using the legacy version of M1/M2 chips
                    "optimizer": tf.keras.optimizers.legacy.RMSprop(learning_rate=optimal_hyperparameters.get('learning_rate')),
                    "learning_rate": None,
                    "batch_size": 128,
                    "epochs_number": 50,
                    "loss_function": 'categorical_crossentropy'
                }


        stage_6_layers = [
                layers.Dense(optimal_hyperparameters.get('units1'), activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                layers.Dense(optimal_hyperparameters.get('units2'), activation='relu', bias_regularizer=regularizers.L1(optimal_hyperparameters.get('alpha')),
                             kernel_initializer=initializers.HeNormal()),
                layers.Dropout(optimal_hyperparameters.get('dropout1')),
                layers.Dense(optimal_hyperparameters.get('units3'), activation='relu', bias_regularizer=regularizers.L1(optimal_hyperparameters.get('alpha')),
                             kernel_initializer=initializers.HeNormal()),
                layers.Dropout(optimal_hyperparameters.get('dropout2')),
                layers.Dense(optimal_hyperparameters.get('units4'), bias_regularizer=regularizers.L1(optimal_hyperparameters.get('alpha')),
                             kernel_initializer=initializers.HeNormal()),
                layers.Dropout(optimal_hyperparameters.get('dropout3')),
                layers.Dense(optimal_hyperparameters.get('units5'), bias_regularizer=regularizers.L1(optimal_hyperparameters.get('alpha')),
                            kernel_initializer=initializers.HeNormal()),
                layers.Dropout(optimal_hyperparameters.get('dropout4')),
                layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                ]

        keras_tuner_2 = NeuralNet(stage_6_layers)
        keras_tuner_2.set_hyperparameters(stage_6_hyperparameters)
        keras_tuner_2.compile_model()
        keras_tuner_2.fit(x_train_scaled, y_train, (x_val_scaled, y_val), verboseParam=0)
```

```
In [88]: from IPython.display import Image
         Image(filename="./images/keras_tuner_2_graph.png", width='500')
```

Out[88]:

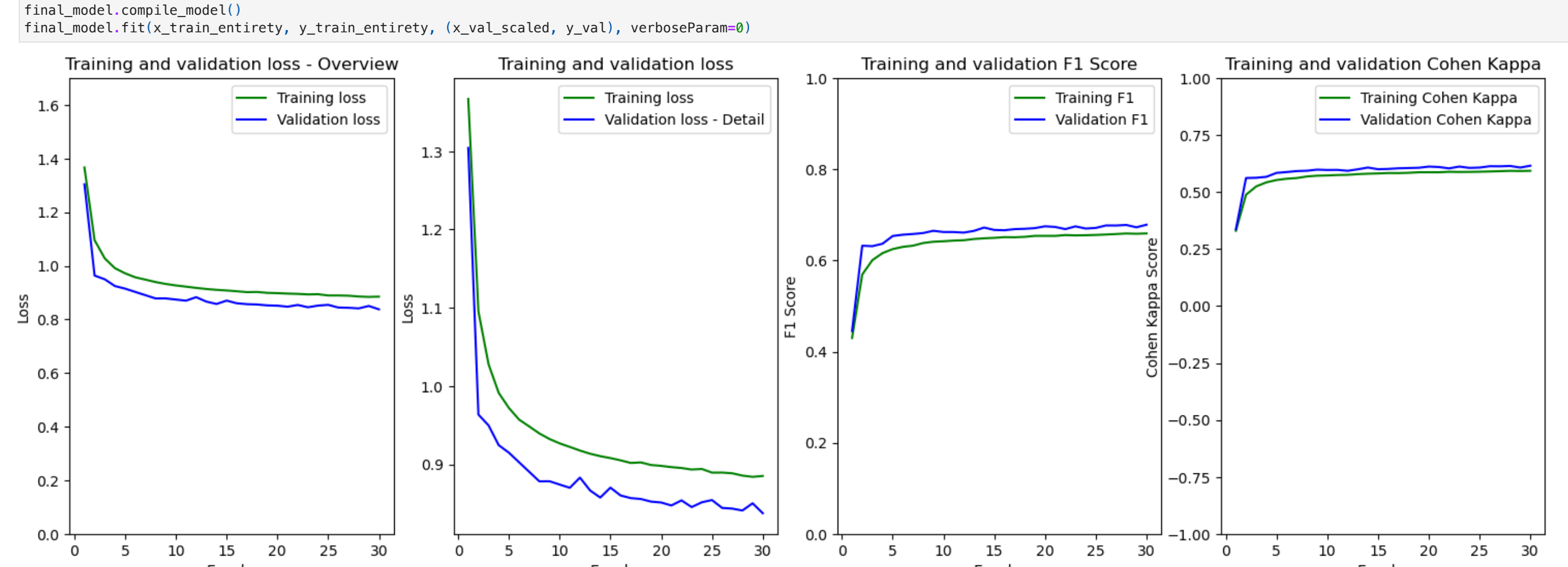


Retrain and testing of final model

Merge the validation and training sets, retrain for optimal epochs and evaluate on the test set:

```
In [34]: x_train_entirety = tf.concat([x_train_scaled, x_val_scaled], axis=0)
         y_train_entirety = tf.concat([y_train, y_val], axis=0)
```

```
In [58]: stage_6_hyperparameters = {
                     #a tensorflow warning suggested using the legacy version of M1/M2 chips
                     "optimizer": tf.keras.optimizers.legacy.RMSprop(decay=1e-3),
                     "batch_size": 128,
                     "epochs_number": 30,
                     "loss_function": 'categorical_crossentropy'
                 }


         stage_6_layers = [
                 layers.Dense(stage_6_num_units, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, activation='relu', bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.6),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                              kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(stage_6_num_units, bias_regularizer=regularizers.L1(1e-6),
                             kernel_initializer=initializers.HeNormal()),
                 layers.Dropout(0.4),
                 layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
                 ]

         final_model = NeuralNet(stage_6_layers)
         final_model.set_hyperparameters(stage_6_hyperparameters)
         final_model.compile_model()
         final_model.fit(x_train_entirety, y_train_entirety, (x_val_scaled, y_val), verboseParam=0)
```



^ 8 Hidden layers, 30 epochs, [600, 600, 600, 600, 600] units ^

In [59]:

```
(test_loss, test_acc, cohen_kappa) = final_model.test(x_test_scaled, y_test)
print(test_acc)
```

```
563/563 [==============================] - 1s 922us/step - loss: 0.8182 - f1_score: 0.6794 - cohen_kappa: 0.6169
[0.9143597  0.63447523 0.61759543 0.67940336 0.6351655  0.5951203 ]
```

Test F1 Score:

In [62]:

```
print(np.mean(test_acc))
```

```
0.67935324
```

Test Loss:

In [60]:

```
print(test_loss)
```

```
0.8182498812675476
```

Test Cohen Kappa:

In [61]:

```
print(cohen_kappa)
```

```
0.6168555617332458
```

Results

In [12]:

```
final_results = pd.read_csv('final_results.csv')
final_results
```

Out[12]:

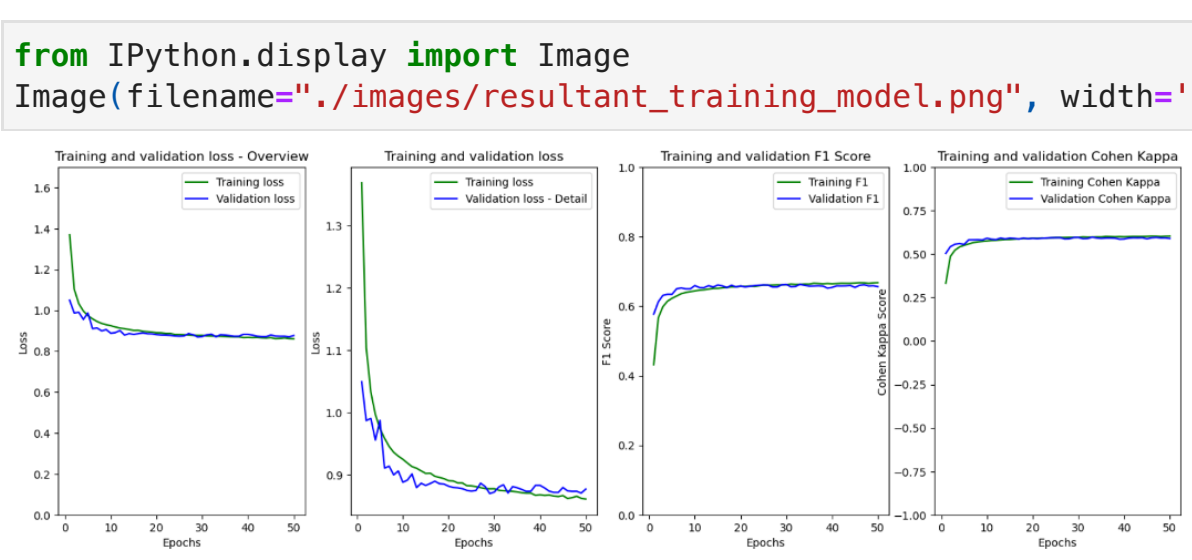
	test_loss	val_f1_score	test_cohen_kappa
0	0.818	0.679	0.617

The final model added (0.679-0.25) = 0.429 to the F1 statistical baseline.

In [13]:

```
from IPython.display import Image
Image(filename="./images/resultant_training_model.png", width='500')
```

Out[13]:



^ 8 Hidden layers, 50 epochs, [600, 600, 600, 600, 600] units ^

The test loss, validation F1 Score, validation Cohen kappa seem to be relatively on par with the estimated model performance during training.

In [68]:

```
stage_6_results.sort_values(by='val_f1_score', ascending=False).head()
```

Out[68]:

	num_hidden_layers	num_units	optimizer	learning_rate	activation_function	batch_size	num_epochs	regularization	val_f1_score
11	1	600	rmsprop	0.001	relu	128	100	False	0.702647
14	1	600	rmsprop	0.001	relu	128	100	L2, alpha:1e-4	0.698807
25	4	600	rmsprop	0.001	relu	128	100	False	0.695391
19	4	600	rmsprop	0.001	relu	128	100	L1, alpha:1e-4	0.689719
22	4	600	rmsprop	0.001	relu	128	100	L2, alpha:1e-4	0.688726

Comparing the final model to the stage 6 and 7 results, there is a discrepancy of ~0.03 between validation f1 score and test f1 score. This seems a relatively small difference that suggests the validation evaluation procedure was functioning effectively.

Throughout stages 5-7 I tested 55 combination of hyperparameters:

In [67]:

```
stage_5_results.shape[0] + stage_6_results.shape[0]
```

Out[67]:

```
55
```

Evaluation

'As a general guideline, an F1 score of 0.7 or higher is often considered good.' (Logunova & guide, 2023) - Based on this assessment is acceptable but could be improved:

Identifying areas for improvement:

I could further explore the application of L1 and L2 regularizers ; the Keras documentation states that L1 and L2 can be supplied to the following parameters:

- kernel_regularizer: Regularizer to apply a penalty on the layer's kernel
- bias_regularizer: Regularizer to apply a penalty on the layer's bias
- activity_regularizer: Regularizer to apply a penalty on the layer's output

Source (Keras, 2024)

I think it would be beneficial to test a wider range of alpha values for L1 and L2 regularization. Part of the methodoloy of this coursework has been to identify optimal hyperparameters in isolation ; it may be more efficient to perform a grid search perhaps as the best performing model may require each hyperparameter to compromise / balance with the other hyperparameters.

I note that the TensorFlow library offers may learning rate schedulers, such as ExponentialDecay, CosineDecay etc - these are additional hyperparameter values for consideration.

Perhaps an alternative approach to hyperparameter tuning could be to retrieve a smaller stratified sample to perform a grid search; this could potentially establishing an initial search space in a relatively short amount of time (and in a systematic manner).

I would like to further explore the cause of performance plateau (eg perhaps a neural network 'saddle point').

Identifying areas for extension:

I am curious to investigate the importance of each hyperparameter for this dataset and model. An article published on website 'Weights & Biases ' (an online applied machine learning community) discussed using random forests to calculate an 'importance' metric for each hyperparameter. (Shukla, 2023) The suggestion was that a random forest was passed hyperparameter values as inputs and the metric (eg loss) as the target output. Calculating an 'importance metric' for each hyperparameter could reduce hyperparameter training time, with greater performance gains made in less time.
This point is further expanded by (Coelho, 2020); 'For the most important hyperparameters we hope to reduce the search range and focus on the best values, while for the least important ones, we could fix them to a single value and thus remove entire dimensions from a grid search.'

I have recently read about neural network architectures that are designed to handle gradient-related problems (such as vanishing and exploding gradient). An example of this are 'Residual connections' ; these allow a network to 'skip some layers and add the output of a previous layer to the input of a later layer. This can create a shortcut for the gradients to flow through the network and avoid being diluted by multiple layers.' . By 'skipping' some layers a neural network can combine levels of abstraction; making it more capable of learning complex, non-linear patterns. (Cisco, 2023) Implementing techniques such as these could make for a more resilient model during training.

Appendices

Appendix A: Code to implement SMOTETomek

I chose to use a combination of SMOTE and Tomek Links as it effectively 'cleans' the dataset whilst oversampling (SMOTE) ; ambiguous points 'Tomek Links' are removed as part of the SMOTETomek process. (Truong, 2022)

I split the data into training, validation and test set before applying SMOTETomek, this decision was influenced by the following research; "If you are going to use SMOTE, it should only be applied to the training data. This is because you are using SMOTE to gain an improvement in operational performance, and both the validation and test sets are there to provide an estimate of operational performance" (Cawley, 2022)

For this reason I did not use the 'validation_split' parameter available for keras model.fit() https://keras.io/api/models/model_training_apis/

Random oversampling can decrease the variance of the dataset which can potentially lead to an overfitting model https://www.youtube.com/watch?v=8_4Ls7k1wyw Synthetic minority oversampling technique (SMOTE) developed by Chawla et al. (2002) - 'generates a new observation that exists on a line between observations' - there are 3 common extensions to SMOTE

Random undersampling can potentially remove key characteristics from the dataset https://www.youtube.com/watch?v=8_4Ls7k1wyw Tomek-links algorithm - 'takes 2 observations of different classes and whose distance is shortest using a form of k nearest neighbours - the algorithm then deletes the majority class observation'

the class imbalance suggests that accuracy is not an appropriate metric ; as a random model could potentially acheive a high accuracy by only predicting the majority class [https://machinelearningmastery.com/tour-of-evaluation-metrics-for-imbalanced-classification/]

for this reason I decided not to use the keras categorical_accuracy metric ; the keras documentation states that this metric 'Calculates how often predictions match one-hot labels' (Keras, 2024); if the validation/test set is imbalanced (ie with a majority class), a high accuracy could be acheived by a model that predicts the majority class every time.

I applied SMOTETomek to the training data ; this resulted in a balanced class distribution to train the model. However I did not apply SMOTETomek to the validation and testing data as it would have introduced artificial data (Nair, 2022) - this means that the validation and test sets are imbalanced. This imbalance affects the choice of evaluation metric:

My next step was to research evaluation metrics that are not impacted by class imbalance:

Candidates identified:

- F1 Score: appears to be appropraite for imbalanced datasets https://thedatascientist.com/f-1-measure-useful-imbalanced-class-problems/ 'Maximizing for the F1 score implies simultaneously maximizing for both precision and recall' (Logunova & guide, 2023)

a noted limitation of the F1 score is that it 'disregards true negatives, making it misleading for imbalanced classes' (Logunova & guide, 2023)

- Cohen Kappa ; 'normalized by the imbalance of the classes in the data.' (Brownlee, 2020) ; scores range from [-1, 1]. Negative scores indicate a disagreement between two raters; positive scores indicate agreement between. A score of 0 indicates agreement by chance. (TensorFlow, 2023)
- Area Under the Precision-Recall Curve (AUPRC) ; Machine learning researcher Daniel Rosenberg (MSc) suggested that AUPRC may offer a more accuracate performance estimate of models trained on imbalanced data (compared to ROC-AUC). Rosenberg attributes this to the difference in x-axis ; the ROC curve uses FPR (false positive rate), whilst AUPRC uses recall. In the context of unbalanced data, the rate of change of FPR is slower than recall. (Rosenberg, 2022)

In [ ]:

```
# from imblearn.combine import SMOTETomek
# from imblearn.under_sampling import TomekLinks
# #imblearn appears to be compatible with scikit-learn 1.2.2 - versions higher than this appear to be incompatible
# #as noted by other developers
# #https://stackoverflow.com/questions/76593906/how-to-resolve-cannot-import-name-missingvalues-from-sklearn-utils-param-v


# #this approach has been applied from
# #https://www.aitude.com/multiclass-classification-on-highly-imbalanced-dataset/
# resample=SMOTETomek(tomek=TomekLinks(sampling_strategy='majority'))
# x_train_SMOTETomek, y_train_SMOTETomek = resample.fit_resample(x_train, y_train)
```

In [ ]:

```
# y_train_SMOTETomek.hist()
```

The combined approach of SMOTE and Tomek appears to have rebalanced the classes.

In [ ]:

```
# from collections import Counter
# new_class_counts = Counter(y_train_SMOTETomek)
# min_class_count_ID = min(new_class_counts)
# percentage_of_dataset = new_class_counts[min_class_count_ID] / len(y_train_SMOTETomek) *100
# percentage_of_dataset
```

In [ ]:

```
# for key in new_class_counts:
#     print(new_class_counts[key] / len(y_train_SMOTETomek) *100)
```

The new data distribution can be classified as balanced ; each class represents approximately 14% of the dataset

A graph of the samples generated by SMOTE (shown in green):

In [ ]:

```
# import matplotlib.pyplot as plt

# classes = list(set(y))

# plt.bar(classes, height=y.value_counts().to_list(), color='cornflowerblue')
# plt.bar(classes, height=y_train_SMOTETomek.value_counts().to_list(), color='lime')
# plt.show()
```

Values have been generated for class 7 - values have been removed from classes 1-6 as part of the rebalancing process

In [ ]:

```
# plt.scatter(x_train_SMOTETomek['channel2'], y_train_SMOTETomek, s=10, c='lime', marker="s", label='SMOTE-generated')
# plt.scatter(x_train['channel2'], y_train, s=10, c='b', marker="s", label='original')
# plt.legend(loc='upper left')
# plt.show()
```

Looking at the data sample visualised above ; there do not appear to be any SMOTE generate values outside the expected range - (SMOTE generated examples are shown in green)

Scaling on SMOTETomek Data

In [ ]:

```
# from sklearn.preprocessing import MinMaxScaler
# #scaler = MinMaxScaler(feature_range=(-1, 1))
# scaler = MinMaxScaler()
# x_train_SMOTETomek_scaled = scaler.fit_transform(x_train_SMOTETomek)
# x_train_SMOTETomek_scaled
```

Convert to SMOTETomek data to tensor form

In [ ]:

```
# import tensorflow as tf
# #The DataFrame can be converted to a NumPy array using the DataFrame.values property or numpy.array(df).
# #To convert it to a tensor, use tf.convert_to_tensor:
# #https://www.tensorflow.org/tutorials/load_data/pandas_dataframe
# x_train_SMOTETomek_scaled = tf.convert_to_tensor(x_train_SMOTETomek_scaled)
# x_train_SMOTETomek_scaled
```

One hot encode SMOTETomek data

In [ ]:

```
# new_SMOTETomek_train_classes = [x-1 for x in list(y_train_SMOTETomek)]
```

In [ ]:

```
# y_train = to_categorical(new_SMOTETomek_train_classes)
# y_train_SMOTETomek = tf.convert_to_tensor(y_train)
# y_train_SMOTETomek
```

Calcuation of majority class baseline on SMOTETomek data

In [ ]:

```
# SMOTETomek_class_count = Counter(new_SMOTETomek_train_classes)
# SMOTETomek_class_count
```

In [ ]:

```
# max_class_ID_SMOTETomek = max(SMOTETomek_class_count)
# max_class_count_SMOTETomek = SMOTETomek_class_count[max_class_ID_SMOTETomek]
# total_class_count_SMOTETomek = len(y_train_SMOTETomek)
# statistical_baseline_SMOTETomek = max_class_count_SMOTETomek / total_class_count_SMOTETomek * 100
# statistical_baseline_SMOTETomek
```

On reflection and further research I realised that the above was not directly comparable to the F1 evaluation of the model; Further research identified that the common sense baseline is calculated on the test set (Brownlee, 2020)

With this in mind; I calculated the following common sense baseline:

In [ ]:

```
# #majority class baseline: f1 score
# from sklearn.dummy import DummyClassifier
# from sklearn.metrics import classification_report
# from sklearn.metrics import precision_recall_fscore_support as score

# dummy_clf = DummyClassifier(strategy="most_frequent")
# dummy_clf.fit(x_train_SMOTETomek_scaled, y_train_SMOTETomek)
# y_pred_dummy = dummy_clf.predict(x_test)
# precision,recall,fscore,support=score(y_test,y_pred_dummy,average='macro')
# fscore
```

In [ ]:

```
# #random baseline: calculated on validation data
# dummy_clf = DummyClassifier(strategy="uniform")
# #the sklearn docs define 'uniform' as follows:
# #'generates predictions uniformly at random from the list of unique classes observed in y, i.e. each class has equal probability.'
# #(scikit learn, 2024)
# dummy_clf.fit(x_train_SMOTETomek_scaled, y_train_SMOTETomek)
# y_pred_dummy = dummy_clf.predict(x_val)
# precision,recall,fscore,support=score(y_val,y_pred_dummy,average='macro')
# val_common_sense_baseline = fscore
# val_common_sense_baseline
```

In [ ]:

```
# #random baseline: calculated on test data
# dummy_clf = DummyClassifier(strategy="uniform")
# #the sklearn docs define 'uniform' as follows:
# #'generates predictions uniformly at random from the list of unique classes observed in y, i.e. each class has equal probability.'
# #(scikit learn, 2024)
# dummy_clf.fit(x_train_SMOTETomek_scaled, y_train_SMOTETomek)
# y_pred_dummy = dummy_clf.predict(x_test)
# precision,recall,fscore,support=score(y_test,y_pred_dummy,average='macro')
# test_common_sense_baseline = fscore
# test_common_sense_baseline
```

Train the model on SMOTETomek data

In [ ]:

```
# initial_hyperparameters = {
#             "optimizer": 'rmsprop',
#             "learning_rate": None,
#             "batch_size": 128,
#             "epochs_number": 5,
#             "loss_function": 'categorical_crossentropy',
#             #use categorical cross entropy as targets are one-hot encoded
#         }
# initial_layers = [
#     layers.Dense(10, activation='relu', input_shape=(9,)), #input shape == x_train_tensor[0].shape
#     layers.Dense(num_classes, activation='softmax') #7 output neurons as there are 7 classes
#     ]

# model_1 = NeuralNet(initial_layers)
# model_1.set_hyperparameters(initial_hyperparameters)
# model_1.compile_model()
# model_1.fit(x_train_SMOTETomek_scaled, y_train_SMOTETomek, (x_val_scaled, y_val))
```

Appendix B: Comparing the performance of a model trained on non-SMOTETomek data

Model definition 2: Trained on non-SMOTETomek Data

(compare to the model trained on SMOTETomek data)

In [ ]:

```
# model_2 = NeuralNet(initial_layers)
# model_2.set_hyperparameters(initial_hyperparameters)
# model_2.compile_model()
# model_2.fit(x_train_scaled, y_train, (x_val_scaled, y_val))
```

In the graphs above, the model is performing better on the validation data ; this indicates that the training data is harder to model compared to the validation data (Bhattiprolu, 2020) - this suggests there is an unrepresentative split between training and validation. (possibly as a result of not applying SMOTETomek)

The training / validation loss curve above is similar to the following graph ; (Bhattiprolu, 2020)

Image source: (Bhattiprolu, 2020)

The val_f1score of the model trained on the non-SMOTETomek data appears to be lower than the val_f1score of the model trained on SMOTETomek data ; for this reason I will further develop the model trained on SMOTETomek data (model_1)

References

Arie, B. D. (2008, January 6). Comparison of classification accuracy using Cohen's Weighted Kappa. Science Direct. Retrieved March 11, 2024, from https://www.sciencedirect.com/science/article/abs/pii/S0957417406003435

Awan, A. A. (2023). What is Online Machine Learning? DataCamp. Retrieved March 10, 2024, from https://www.datacamp.com/blog/what-is-online-machine-learning

Bhattiprolu, S. (2020, September 1). 154 - Understanding the training and validation loss curves. Wikipedia. Retrieved March 6, 2024, from https://www.youtube.com/watch?v=p3CcfIjycBA&t=666s

Brownlee, J. (2019, August 6). How to use Learning Curves to Diagnose Machine Learning Model Performance - MachineLearningMastery.com. Machine Learning Mastery. Retrieved March 7, 2024, from https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

Brownlee, J. (2020, Jan 2). How To Get Baseline Results And Why They Matter - MachineLearningMastery.com. Machine Learning Mastery. Retrieved March 6, 2024, from https://machinelearningmastery.com/how-to-get-baseline-results-and-why-they-matter/

Brownlee, J. (2020, August 15). 8 Tactics to Combat Imbalanced Classes in Your Machine Learning Dataset - MachineLearningMastery.com. Machine Learning Mastery. Retrieved March 6, 2024, from https://machinelearningmastery.com/tactics-to-combat-imbalanced-classes-in-your-machine-learning-dataset/

Brownlee, J. (2021, January 22). How to Choose an Activation Function for Deep Learning - MachineLearningMastery.com. Machine Learning Mastery. Retrieved March 7, 2024, from https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/

Brownlee, J. (2021, May 1). Tour of Evaluation Metrics for Imbalanced Classification - MachineLearningMastery.com. Machine Learning Mastery. Retrieved March 11, 2024, from https://machinelearningmastery.com/tour-of-evaluation-metrics-for-imbalanced-classification/

Buhl, N. (2023, July 18). F1 Score in Machine Learning Explained. Encord. Retrieved March 6, 2024, from https://encord.com/blog/f1-score-in-machine-learning/

Cawley, G. C. (2022, January 13). How to use SMOTE on the final model training? Cross Validated. Retrieved March 4, 2024, from https://stats.stackexchange.com/questions/560321/how-to-use-smote-on-the-final-model-training

Ch, V. (2023, July 13). What unit should I put for my Dense? TensorFlow Forum. Retrieved March 8, 2024, from https://discuss.tensorflow.org/t/what-unit-should-i-put-for-my-dense/18173/2

Chollet, F. (2018). Deep Learning with Python. Manning. https://livebook.manning.com/book/deep-learning-with-python/chapter-3/163

Cisco. (2023, November 27). How to Prevent Vanishing Gradients in Neural Networks. LinkedIn. Retrieved March 11, 2024, from https://www.linkedin.com/advice/3/what-best-practices-preventing-vanishing-bmaef

Coelho, A. (2020, June 17). Narrowing the Search: Which Hyperparameters Really Matter? Dataiku blog. Retrieved March 11, 2024, from https://blog.dataiku.com/narrowing-the-search-which-hyperparameters-really-matter

Shukla, L. (2023, September 18). Find The Most Important Hyperparameters In Seconds | articles – Weights & Biases. Weights & Biases. Retrieved March 11, 2024, from https://wandb.ai/wandb_fc/articles/reports/Find-The-Most-Important-Hyperparameters-In-Seconds--Vmlldzo1NDMxNTQz

Crêteur, S. (2022, December 16). Machine learning: one-hot encoding vs integer encoding | by Stéphanie Crêteur | Geek Culture. Medium. Retrieved March 10, 2024, from https://medium.com/geekculture/machine-learning-one-hot-encoding-vs-integer-encoding-f180eb831cf1

Cisco. (2023, November 27). How to Prevent Vanishing Gradients in Neural Networks. LinkedIn. Retrieved March 11, 2024, from https://www.linkedin.com/advice/3/what-best-practices-preventing-vanishing-bmaef

DeepLearningAI & Andrew, N. (2017, August 25). The Problem of Local Optima (C2W3L10). YouTube. Retrieved March 11, 2024, from https://www.youtube.com/watch?v=fODpu1-lNTw

Engati. (2024). Vanishing gradient problem | Engati. Engati chatbot. Retrieved March 7, 2024, from https://www.engati.com/glossary/vanishing-gradient-problem

Google. (2023, June 9). Imbalanced Data | Machine Learning. Google for Developers. Retrieved March 6, 2024, from https://developers.google.com/machine-learning/data-prep/construct/sampling-splitting/imbalanced-data

iguazio. (2024). What is Baseline Models. Iguazio. Retrieved March 6, 2024, from https://www.iguazio.com/glossary/baseline-models/ John Hopkins Medicine. (2024). Electromyography (EMG). Johns Hopkins Medicine. Retrieved March 10, 2024, from https://www.hopkinsmedicine.org/health/treatment-tests-and-therapies/electromyography-emg

Katanforoosh, K. (2019). Initializing neural networks - deeplearning.ai. DeepLearning.AI. Retrieved March 11, 2024, from https://www.deeplearning.ai/ai-notes/initialization/index.html

Keras. (2024). Accuracy metrics. Keras. Retrieved March 6, 2024, from https://keras.io/api/metrics/accuracy_metrics/#categoricalaccuracy-class

Keras. (2024). Layer weight regularizers. Keras. Retrieved March 11, 2024, from https://keras.io/api/layers/regularizers/

Logunova, I., & guide, s. (2023, July 10). F1 Score in Machine Learning. Serokell. Retrieved March 5, 2024, from https://serokell.io/blog/a-guide-to-f1-score#

Mayank, K. (2023, April 4). BxD Primer Series: Elastic Net Regression; combining L1 and L2 regularization. LinkedIn. Retrieved March 11, 2024, from https://www.linkedin.com/pulse/bxd-primer-series-elastic-net-regression-combining-l1-mayank-k-

McGill, S. (1999, September 22). The importance of normalization in the interpretation of surface electromyography: a proof of principle. PubMed. Retrieved February 23, 2024, from https://pubmed.ncbi.nlm.nih.gov/10519560/

Mohd, F. (2006). Techniques of EMG signal analysis: detection, processing, classification and applications. NCBI. Retrieved March 10, 2024, from https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1455479/

Morgunov, A. (2023, August 23). Keras Tuner: Lessons Learned From Tuning Hyperparameters of a Real-Life Deep Learning Model. neptune.ai. Retrieved March 8, 2024, from https://neptune.ai/blog/keras-tuner-tuning-hyperparameters-deep-learning-model

Nair, A. (2022, January 24). Create Artificial Data With SMOTE | by Aashish Nair. Towards Data Science. Retrieved March 6, 2024, from https://towardsdatascience.com/create-artificial-data-with-smote-2a31ee855904

Pao, S. (2021, April 11). Sampling Data with SMOTE, Tomek Links, and Nearmiss in R. YouTube. Retrieved February 22, 2024, from https://www.youtube.com/watch?v=8_4Ls7k1wyw

Rosebrock, A. (2019, October 14). Why is my validation loss lower than my training loss? PyImageSearch. Retrieved March 10, 2024, from https://pyimagesearch.com/2019/10/14/why-is-my-validation-loss-lower-than-my-training-loss/

Rosenberg, D. (2022, June 6). Imbalanced Data? Stop Using ROC-AUC and Use AUPRC Instead. Towards Data Science. Retrieved February 22, 2024, from https://towardsdatascience.com/imbalanced-data-stop-using-roc-auc-and-use-auprc-instead-46af4910a494

Saturn Cloud. (2023, July 6). What Is 'random_state' in sklearn.model_selection.train_test_split Example? Saturn Cloud. Retrieved March 5, 2024, from https://saturncloud.io/blog/what-is-randomstate-in-sklearnmodelselectiontraintestsplit-example/

scikit learn. (2024). sklearn.dummy.DummyClassifier — scikit-learn 1.4.1 documentation. Scikit-learn. Retrieved March 6, 2024, from https://scikit-learn.org/stable/modules/generated/sklearn.dummy.DummyClassifier.html

TensorFlow. (2023, May 25). tfa.metrics.CohenKappa. TensorFlow. Retrieved March 6, 2024, from https://www.tensorflow.org/addons/api_docs/python/tfa/metrics/CohenKappa

Truong, A. (2022, June 27). Imbalanced Data ML: SMOTE and its variants | by An Truong | TotalEnergies Digital Factory. Medium. Retrieved March 10, 2024, from https://medium.com/totalenergies-digital-factory/imbalanced-data-ml-smote-and-its-variants-c69a4b32f7e7

Turp, M. (2022, Nov). How to Make Neural Networks Train Faster on Keras. YouTube: Home. Retrieved March 10, 2024, from https://www.youtube.com/watch?v=NBhas7xb3h4&t=304s

Turp, M. (2022, November 9). How to Choose an Activation Function for Neural Networks. YouTube. Retrieved March 8, 2024, from https://www.youtube.com/watch?v=_UjA9sk_fK4

UC Irvine Machine Learning Repository. (2019, June 1). EMG data for gestures. UCI Machine Learning Repository. Retrieved March 10, 2024, from https://archive.ics.uci.edu/dataset/481/emg+data+for+gestures

Wang, A. (2022, August 5). Why Using Learning Rate Schedulers In NNs May Be a Waste of Time. Towards Data Science. Retrieved March 10, 2024, from https://towardsdatascience.com/why-using-learning-rate-schedulers-in-nns-may-be-a-waste-of-time-8fa20339002b

Wang, C.-F. (2019, January 8). The Vanishing Gradient Problem. The Problem, Its Causes, Its... | by Chi-Feng Wang. Towards Data Science. Retrieved March 7, 2024, from https://towardsdatascience.com/the-vanishing-gradient-problem-69bf08b15484

In [ ]: